

привет, помоги с кодом!

для начала контекст:

 BB+ (Baldi's Basics Plus) — Полный разбор

Что такое BB+?

18

Baldi's Basics Plus (стилизованно как *Baldi's BASICS +*) — это полноценная игра серии Baldi's Basics с атмосферой образовательных игр конца 90-х, элементами хоррора и сюрреализма, созданная студией Basically Games.

19

Это активный стелс-рогалик, который пародирует дешёвые образовательные игры 90-х с тонким хоррор-твистом.

 Основная цель

16

В BB+ твоя цель — собрать все тетради, разбросанные по школе, и добраться до выхода, не позволив Балди поймать тебя.

20

Балди отслеживает игрока через звук — неосторожное движение быстро выдаёт твою позицию. Управление выносливостью становится важным, потому что слишком частый бег делает игрока уязвимым во время погони.

 Геймплей и механики

17

BB+ играется похоже на оригинал — игрок должен исследовать Супер Школу и собирать тетради, избегая Балди. Однако Plus — значительно более масштабная игра: больше персонажей, новые механики, случайные внутриигровые события и несколько новых бонусных возможностей.

17

В игре четыре режима: Hide-and-Seek (Прятки), Endless Mode (Бесконечный режим), Challenges (Испытания) и Field Trips (Полевые поездки).

17

Игрок начинает с тремя жизнями (называемыми «power tubes»). Если Балди поймает тебя, теряешь одну жизнь и вынужден начинать этаж заново. Если потеряешь все — это Game Over, и нужно начинать режим с самого начала.

 Рандомизация и реиграбельность

19

BB+ рандомизирует огромное количество вещей — каждое прохождение будет уникальным опытом. Уровни меняются каждый раз: изменяется не только планировка, но и типы комнат и препятствий.

19

Случайные события могут произойти в любой момент: школа может затопиться или перевернуться вверх ногами, держа тебя в постоянном напряжении.

19

Каждый раз встречается случайная комбинация персонажей, у каждого — уникальное поведение. Когда разные персонажи сталкиваются друг с другом, могут возникать уникальные ситуации.

 Структура уровней

20

Игра включает несколько этажей, соединённых лифтами и системами прогресса. Каждый этаж вводит разные расположения комнат и опасности, делая навигацию всё сложнее по ходу прохождения.

18

Игрок может встретить новых персонажей, таких как Dr. Reflex и The Test, а также случайные события: Student Shuffle, Balder Dash и Gravity Chaos.

 Plus Level Studio — Полный разбор

Что такое Plus Level Studio?

9

Baldi's Basics Plus Level Editor (Plus Level Studio) — это мод, созданный MissingTextureMan101. В нём можно создавать кастомные схемы школ, расставлять спавны персонажей, тетрадей и многое другое.

6

Мод относится к категории Plug-ins на GameBanana и размещён автором под ником MissingTextureMeow101.

Актуальная версия мода — (0.14.X) Plus Level Studio (BETA).

 Что можно создавать?

9

В редакторе можно размещать коридоры/комнаты, персонажей, двери, активности, математические машины, лифты и многое другое, чтобы собрать собственную школу.

Типы комнат (тайлы):

9

H — Коридор (Hallway)

C — Классная комната (Classroom)

F — Комната только для персонала (Faculty Only)

O — Офис (Office)

CL — Шкаф/Чулан (Closet)

DR — Клиника Dr. Reflex

LB — Библиотека (Library)

CF — Кафетерий (Cafeteria)

OUT — Улица (Outside)

 Сохранение и загрузка

9

В редакторе есть кнопка сохранения, загрузки и кнопка Play. Play загружает твою карту, Save сохраняет её в `AppData\LocalLow\Basically Games\Baldi's Basics Plus\CustomLevels`, а Load загружает сохранённый уровень.

 Расширения Plus Level Studio

Экосистема мода включает несколько расширений:

Plus Level Studio Pro —

7

расширенный редактор уровней,

7

созданный пользователем GrandGames на GameBanana.

Plus Level Studio Extension: Custom Runs —

8

позволяет сыграть все кастомные уровни в одном заходе Hide & Seek.

BB Plus Level Loader System —

13

дополнительный плагин от того же автора (MissingTextureMeow101) для загрузки уровней.

📦 Установка

14

Plus Level Studio устанавливается через GameBanana, а для работы требует BeplnEx. Также доступен отдельный инструмент — BB Plus Mod Manager

—

3

для удобного управления модами в BB+.

📖 История версий

Мод изначально назывался просто «Baldi's Basics Plus Level Editor».

9

Он был прекращён, когда автор перешёл на новый мод — Plus Level Studio, начиная с версии игры 0.12.X.

Сейчас актуальная версия мода поддерживает BB+ 0.14.X.

📄 Итоговый контекст для промпта

ПараметрBB+Plus Level Studio

ТипИгра (стелс-рогалик)Мод/плагин для BB+

РазработчикBasically Games (mystman12)MissingTextureMeow101 / GrandGames

ПлатформаWindows, macOS, LinuxGameBanana (BeplnEx плагин)

Версия0.14.X(0.14.X) BETA

СутьСобирать тетради, избегать БалдиСоздавать кастомные уровни для BB+

РандомизацияДа, процедурная генерацияНет — уровень создаётся вручную

Расширения—Pro, Custom Runs, Level Loader

вот мой код:

```
using PlusLevelStudio.Editor;
using System.Collections.Generic;
using UnityEngine;
namespace KnoxumPLSExtension.Features
{
    public static class HighWallsController
    {
        public static Dictionary<int, int> roomHeights = new Dictionary<int, int>();
        public static Dictionary<int, int> roomYOffsets = new Dictionary<int, int>();
        public static Dictionary<(int x, int z), ushort> runtimeCellRoomIdMap
            = new Dictionary<(int x, int z), ushort>();
        public static bool runtimeMapPopulated = false;
        public static bool isRampToolActive = false;
        public static float currentEditorRampHeightSelection = 1f;
        public static int GetRoomHeight(int roomId)
        {
            if (roomId <= 0) return 1;
            if (roomHeights.TryGetValue(roomId, out int height)) return height;
            return 1;
        }
        public static int GetRoomYOffset(int roomId)
        {
            if (roomId <= 0) return 0;
            if (roomYOffsets.TryGetValue(roomId, out int offset)) return offset;
            return 0;
        }
        public static void RegisterRoom3D(int roomId, int height, int offset, bool refreshCells)
        {
            if (roomId <= 0)
            {
                Debug.LogWarning($"[Knoxum] RegisterRoom3D вызван с некорректным roomId={roomId}, игнорируется.");
                return;
            }
            roomHeights[roomId] = Mathf.Clamp(height, 1, 7);
            roomYOffsets[roomId] = Mathf.Max(0, offset);
            Debug.Log($"[Knoxum] RegisterRoom3D: roomId={roomId} height={roomHeights[roomId]} yOffset={roomYOffsets[roomId]}");
            if (refreshCells && Singleton<EditorController>.Instance != null)
            {
                Singleton<EditorController>.Instance.RefreshCells(true, true);
                var rooms = Singleton<EditorController>.Instance.levelData.rooms;
                int idx = roomId - 1;
                if (idx >= 0 && idx < rooms.Count)
                {
                    Singleton<EditorController>.Instance.UpdateVisualsForRoom(rooms[idx]);
                }
            }
        }
        public static void ResetAll()
        {
            roomHeights.Clear();
            roomYOffsets.Clear();
            runtimeCellRoomIdMap.Clear();
            runtimeMapPopulated = false;
            Debug.Log($"[Knoxum] HighWallsController.ResetAll() выполнен.");
        }
        public static void DebugDumpState(string context)
```

```

    {
        Debug.Log($"[Knoxum][DUMP:{context}] roomHeights.Count={roomHeights.Count}, " +
            $"runtimeCellRoomIdMap.Count={runtimeCellRoomIdMap.Count}, " +
            $"runtimeMapPopulated={runtimeMapPopulated}, " +
            $"isEditorAlive={Singleton<EditorController>.Instance != null}");
        foreach (var kv in roomHeights)
        {
            Debug.Log($"[Knoxum][DUMP:{context}] roomId={kv.Key} height={kv.Value} yOffset={GetRoomYOffset(kv.Key)}");
        }
    }
}
}
using HarmonyLib;
using System;
using System.IO;
using System.Collections.Generic;
using PlusLevelStudio.UI;
using TMPro;
using UnityEngine;
using UnityEngine.UI;
using PlusLevelStudio.Editor;
using PlusLevelStudio.Editor.SettingsUI;
namespace KnoxumPLSExtension.Features
{
    public class KnoxumSelectionUI : MonoBehaviour
    {
        private static KnoxumSelectionUI instance;
        public EditorRoom targetRoom;
        private GameObject internalPanel;
        public Image heightUpImg;
        public Image heightDownImg;
        private TextMeshProUGUI heightText;
        public Image yPosUpImg;
        public Image yPosDownImg;
        private TextMeshProUGUI yPosValueText;
        public Sprite spUpNormal;
        public Sprite spUpHover;
        public Sprite spUpLimit;
        public Sprite spDownNormal;
        public Sprite spDownHover;
        public Sprite spDownLimit;
        public bool isHUpL; public bool isHDownL;
        public bool isYUpL; public bool isYDownL;
        private static Dictionary<EditorRoom, int> roomFallbackCache = new Dictionary<EditorRoom, int>();
        public static KnoxumSelectionUI Instance
        {
            get
            {
                if (instance == null)
                {
                    GameObject go = new GameObject("KnoxumUI_Global");
                    instance = go.AddComponent<KnoxumSelectionUI>();
                    DontDestroyOnLoad(go);
                }
                return instance;
            }
        }

        private int GetEditorRoomId(EditorRoom room)
        {
            if (room == null) return -1;
            if (Singleton<EditorController>.Instance != null && Singleton<EditorController>.Instance.levelData != null)
            {
                int index = Singleton<EditorController>.Instance.levelData.rooms.IndexOf(room);
                if (index >= 0)
                {
                    int roomId = index + 1;
                    roomFallbackCache[room] = roomId;
                    return roomId;
                }
            }
            if (roomFallbackCache.TryGetValue(room, out int cachedId))
            {
                return cachedId;
            }

            var field = typeof(EditorRoom).GetField("id", System.Reflection.BindingFlags.Public | System.Reflection.BindingFlags.NonPublic |
                System.Reflection.BindingFlags.Instance)
                ?? typeof(EditorRoom).GetField("roomId", System.Reflection.BindingFlags.Public | System.Reflection.BindingFlags.NonPublic |
                System.Reflection.BindingFlags.Instance)

```

```

        ?? typeof(EditorRoom).GetField("roomId", System.Reflection.BindingFlags.Public | System.Reflection.BindingFlags.NonPublic |
System.Reflection.BindingFlags.Instance);
        if (field != null)
        {
            int rawId = (int)field.GetValue(room);
            int roomId1based = rawId + 1;
            roomFallbackCache[room] = roomId1based;
            return roomId1based;
        }
        return -1;
    }
    private Sprite LoadSprite(string path)
    {
        if (!File.Exists(path)) return Sprite.Create(Texture2D.whiteTexture, new Rect(0, 0, 4, 4), Vector2.one * 0.5f);
        byte[] fileBytes = File.ReadAllBytes(path);
        Texture2D tex = new Texture2D(2, 2);
        if (ImageConversion.LoadImage(tex, fileBytes))
        {
            tex.filterMode = FilterMode.Point;
            return Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), Vector2.one * 0.5f);
        }
        return null;
    }
    public void InjectToHandler(RoomSettingsExchangeHandler handler, EditorRoom room)
    {
        this.targetRoom = room;
        if (internalPanel != null)
        {
            internalPanel.transform.SetParent(handler.transform, false);
            internalPanel.SetActive(true);
            RefreshSprites();
            return;
        }
        string baseDir = Path.Combine(Application.streamingAssetsPath, "Modded", "knoxum.moddinghq.plsextension", "UI", "Editor");
        spUpNormal = LoadSprite(Path.Combine(baseDir, "UpArrow.png"));
        spUpHover = LoadSprite(Path.Combine(baseDir, "UpArrow_Hover.png"));
        spUpLimit = LoadSprite(Path.Combine(baseDir, "UpArrow_Limit.png"));
        spDownNormal = LoadSprite(Path.Combine(baseDir, "DownArrow.png"));
        spDownHover = LoadSprite(Path.Combine(baseDir, "DownArrow_Hover.png"));
        spDownLimit = LoadSprite(Path.Combine(baseDir, "DownArrow_Limit.png"));
        internalPanel = new GameObject("KnoxumInjectedPanel");
        internalPanel.transform.SetParent(handler.transform, false);
        RectTransform pRt = internalPanel.AddComponent<RectTransform>();
        pRt.anchorMin = new Vector2(0.5f, 0.5f);
        pRt.anchorMax = new Vector2(0.5f, 0.5f);
        pRt.pivot = new Vector2(0.5f, 0.5f);
        pRt.anchoredPosition = new Vector2(-285f, 0f);
        pRt.sizeDelta = new Vector2(130f, 200f);
        CreateTip(internalPanel.transform, "H_Tip", LoadSprite(Path.Combine(baseDir, "RoomHeightTip.png")), 45f);
        CreateTip(internalPanel.transform, "Y_Tip", LoadSprite(Path.Combine(baseDir, "RoomYPosTip.png")), -45f);
        heightText = CreateText(internalPanel.transform, "H_Val", 45f);
        yPosValueText = CreateText(internalPanel.transform, "Y_Val", -45f);
        heightUpImg = CreateArrow(internalPanel.transform, "H_Up", 45f, 24f, true, true);
        heightDownImg = CreateArrow(internalPanel.transform, "H_Down", 45f, -24f, true, false);
        yPosUpImg = CreateArrow(internalPanel.transform, "Y_Up", -45f, 24f, false, true);
        yPosDownImg = CreateArrow(internalPanel.transform, "Y_Down", -45f, -24f, false, false);
        RefreshSprites();
    }
    private void CreateTip(Transform p, string n, Sprite s, float y)
    {
        GameObject g = new GameObject(n); g.transform.SetParent(p, false);
        Image i = g.AddComponent<Image>(); i.sprite = s;
        i.rectTransform.sizeDelta = new Vector2(32f, 32f);
        i.rectTransform.anchoredPosition = new Vector2(-10f, y);
    }
    private TextMeshProUGUI CreateText(Transform p, string n, float y)
    {
        GameObject g = new GameObject(n); g.transform.SetParent(p, false);
        TextMeshProUGUI t = g.AddComponent<TextMeshProUGUI>();
        t.fontSize = 20f; t.alignment = TextAlignmentOptions.Center;
        t.rectTransform.sizeDelta = new Vector2(40f, 30f);
        t.rectTransform.anchoredPosition = new Vector2(30f, y);
        t.color = Color.black;
        return t;
    }
    private Image CreateArrow(Transform p, string n, float yBase, float yOff, bool isH, bool isUp)
    {
        GameObject g = new GameObject(n); g.transform.SetParent(p, false);
        g.AddComponent<CanvasRenderer>();
    }

```

```

g.tag = "Button";
Image i = g.AddComponent<Image>();
i.rectTransform.sizeDelta = new Vector2(20f, 20f);
i.rectTransform.anchoredPosition = new Vector2(30f, yBase + yOff);
Button b = g.AddComponent<Button>();
b.targetGraphic = i;
b.transition = Button.Transition.None;
b.onClick.AddListener(delegate {
    if (targetRoom == null) return;
    int roomId = GetEditorRoomId(targetRoom);
    if (roomId <= 0) return;
    int h = HighWallsController.GetRoomHeight(roomId);
    int y = HighWallsController.GetRoomYOffset(roomId);
    if (isH) h = Mathf.Clamp(h + (isUp ? 1 : -1), 1, 7);
    else y = Mathf.Clamp(y + (isUp ? 1 : -1), 0, 10);
    HighWallsController.RegisterRoom3D(roomId, h, y, true);
    RefreshSprites();
});
return i;
}
public void RefreshSprites()
{
    if (targetRoom == null || Singleton<EditorController>.Instance == null) return;
    int roomId = GetEditorRoomId(targetRoom);
    if (roomId <= 0) return;
    int h = HighWallsController.GetRoomHeight(roomId);
    int y = HighWallsController.GetRoomYOffset(roomId);
    if (heightText != null) heightText.text = h.ToString();
    if (yPosValueText != null) yPosValueText.text = y.ToString();
    isHUpL = h >= 7; isHDownL = h <= 1; isYUpL = y >= 10; isYDownL = y <= 0;
    if (heightUpImg != null) heightUpImg.sprite = isHUpL ? spUpLimit : spUpNormal;
    if (heightDownImg != null) heightDownImg.sprite = isHDownL ? spDownLimit : spDownNormal;
    if (yPosUpImg != null) yPosUpImg.sprite = isYUpL ? spUpLimit : spUpNormal;
    if (yPosDownImg != null) yPosDownImg.sprite = isYDownL ? spDownLimit : spDownNormal;
}
public void Hide() { if (internalPanel != null) internalPanel.SetActive(false); }
}
[HarmonyPatch(typeof(RoomSettingsExchangeHandler), "UpdateTextures")]
class SelectionFixPatch
{
    static void Postfix(RoomSettingsExchangeHandler __instance)
    {
        EditorRoom r = (EditorRoom)AccessTools.Field(typeof(RoomSettingsExchangeHandler), "myRoom").GetValue(__instance);
        if (r != null) KnoxumSelectionUI.Instance.InjectToHandler(__instance, r);
    }
}
[HarmonyPatch(typeof(RoomSettingsExchangeHandler), "OnExit")]
class SelectionExitFixPatch
{
    static void Postfix() { KnoxumSelectionUI.Instance.Hide(); }
}
[HarmonyPatch(typeof(CursorController), "Update")]
class KnoxumCursorFixPatch
{
    private static GameObject lastHovered;
    static void Postfix(CursorController __instance)
    {
        var ui = KnoxumSelectionUI.Instance;
        if (ui == null || ui.heightUpImg == null) return;
        var results = (List<UnityEngine.EventSystems.RaycastResult>)AccessTools.Field(typeof(CursorController), "results").GetValue(__instance);
        if (results == null || results.Count == 0)
        {
            ResetHover(ui);
            return;
        }
        GameObject currentHit = results[0].gameObject;
        if (currentHit == null)
        {
            ResetHover(ui);
            return;
        }
        if (currentHit != lastHovered)
        {
            ResetHover(ui);
            lastHovered = currentHit;
            if (currentHit == ui.heightUpImg.gameObject && !ui.isHUpL) ui.heightUpImg.sprite = ui.spUpHover;
            else if (currentHit == ui.heightDownImg.gameObject && !ui.isHDownL) ui.heightDownImg.sprite = ui.spDownHover;
            else if (currentHit == ui.yPosUpImg.gameObject && !ui.isYUpL) ui.yPosUpImg.sprite = ui.spUpHover;
            else if (currentHit == ui.yPosDownImg.gameObject && !ui.isYDownL) ui.yPosDownImg.sprite = ui.spDownHover;
        }
    }
}

```

```

    }
    string clickId = (string)AccessTools.Field(typeof(CursorController), "clickId").GetValue(__instance);
    if (Singleton<InputManager>.Instance.GetDigitalInput(clickId, true))
    {
        Button b = currentHit.GetComponent<Button>();
        if (b != null)
        {
            b.onClick.Invoke();
            var audConfirm = (SoundObject)AccessTools.Field(typeof(CursorController), "audConfirm").GetValue(__instance);
            if (audConfirm != null) Singleton<MusicManager>.Instance.PlaySoundEffect(audConfirm);
        }
    }
}
private static void ResetHover(KnoxumSelectionUI ui)
{
    if (lastHovered != null)
    {
        lastHovered = null;
        ui.RefreshSprites();
    }
}
}
[HarmonyPatch(typeof(global::Cell), "LoadTile")]
class Cell_LoadTile_3DFloorLayerPatch
{
    static void Postfix(global::Cell __instance)
    {
        if (__instance == null || __instance.Tile == null) return;
        int roomId = HighWallsGenerator.GetRoomIdForCell(__instance);
        if (Singleton<EditorController>.Instance == null)
        {
            Debug.Log($"[Knoxum][RUNTIME LoadTile] pos=({__instance.position.x},{__instance.position.z}) " +
                $"resolvedRoomId={roomId} " +
                $"height={HighWallsController.GetRoomHeight(roomId)} " +
                $"mapPopulated={HighWallsController.runtimeMapPopulated} " +
                $"mapCount={HighWallsController.runtimeCellRoomIdMap.Count} " +
                $"heightsCount={HighWallsController.roomHeights.Count}");
        }
        HighWallsGenerator.ProcessCell3D(__instance, roomId);
    }
}
[HarmonyPatch(typeof(global::Cell), "SetShape")]
class Cell_SetShape_Dynamic3DScalePatch
{
    static void Postfix(global::Cell __instance)
    {
        if (__instance == null || __instance.Tile == null) return;
        int roomId = HighWallsGenerator.GetRoomIdForCell(__instance);
        HighWallsGenerator.ProcessCell3D(__instance, roomId);
        var editorCtrl = Singleton<EditorController>.Instance;
        if (editorCtrl != null && roomId > 0)
        {
            var rooms = editorCtrl.levelData.rooms;
            int idx = roomId - 1;            if (idx >= 0 && idx < rooms.Count)
            {
                EditorRoom currentRoom = rooms[idx];
                if (currentRoom != null) editorCtrl.UpdateVisualsForRoom(currentRoom);
            }
        }
    }
}
}
[HarmonyPatch(typeof(EditorController), "CompileAndPlay")]
class CompileAndPlay_InjectRoomMapPatch
{
    static void Prefix(EditorController __instance)
    {
        try
        {
            {
                if (__instance == null || __instance.levelData == null)
                {
                    Debug.LogWarning("[Knoxum] CompileAndPlay_InjectRoomMapPatch: levelData == null, карта НЕ заполнена!");
                    return;
                }
                __instance.levelData.UpdateCells(false);
                HighWallsController.runtimeCellRoomIdMap.Clear();
                var levelData = __instance.levelData;
                for (int x = 0; x < levelData.mapSize.x; x++)
                {
                    for (int z = 0; z < levelData.mapSize.z; z++)

```

```

        {
            var cell = levelData.GetCellSafe(x, z);
            if (cell != null && cell.roomId != 0)
            {
                HighWallsController.runtimeCellRoomIdMap[(x, z)] = cell.roomId;
            }
        }
    }
    HighWallsController.runtimeMapPopulated = true;
    Debug.Log($"[Knoxum] CompileAndPlay: карта roomId заполнена для " +
        $"{HighWallsController.runtimeCellRoomIdMap.Count} клеток из " +
        $"{levelData.mapSize.x * levelData.mapSize.z}, комнат с кастомной высотой: " +
        $"{HighWallsController.roomHeights.Count}");
    HighWallsController.DebugDumpState("CompileAndPlay");
}
catch (Exception e)
{
    Debug.LogError($"[Knoxum] CompileAndPlay_InjectRoomMapPatch error: {e}");
}
}
}
[HarmonyPatch(typeof(EditorFileContainer), "Write")]
class SaveHighWallsPatch
{
    static void Postfix(BinaryWriter writer)
    {
        if (writer == null) return;
        try
        {
            writer.Write("KNOXUM_3D");
            writer.Write(HighWallsController.roomHeights.Count);
            foreach (var entry in HighWallsController.roomHeights)
            {
                writer.Write(entry.Key);
                writer.Write(entry.Value);
                writer.Write(HighWallsController.GetRoomYOffset(entry.Key));
            }
        }
        catch (Exception) { }
    }
}
[HarmonyPatch(typeof(EditorFileContainer), "ReadMindful")]
class LoadHighWallsPatch
{
    static void Postfix(EditorFileContainer __result)
    {
        if (__result == null) return;
        try
        {
            if (Singleton<EditorController>.Instance == null) return;
            string currentPath = typeof(EditorController).GetField("currentPath", System.Reflection.BindingFlags.NonPublic |
                System.Reflection.BindingFlags.Instance)?.GetValue(Singleton<EditorController>.Instance) as string
            ?? typeof(EditorController).GetField("path", System.Reflection.BindingFlags.Public |
                System.Reflection.BindingFlags.Instance)?.GetValue(Singleton<EditorController>.Instance) as string;
            if (string.IsNullOrEmpty(currentPath) || !File.Exists(currentPath)) return;
            HighWallsController.ResetAll();
            using (FileStream fs = new FileStream(currentPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite))
            {
                using (BinaryReader reader = new BinaryReader(fs))
                {
                    byte[] fileBytes = File.ReadAllBytes(currentPath);
                    string fileText = System.Text.Encoding.UTF8.GetString(fileBytes);
                    int markerIdx = fileText.LastIndexOf("KNOXUM_3D");
                    if (markerIdx >= 0)
                    {
                        fs.Position = markerIdx + 9;
                        int count = reader.ReadInt32();
                        for (int i = 0; i < count; i++)
                        {
                            int id = reader.ReadInt32();
                            int h = reader.ReadInt32();
                            int y = reader.ReadInt32();
                            HighWallsController.RegisterRoom3D(id, h, y, false);
                        }
                    }
                }
            }
        }
        catch (Exception) { }
    }
}

```

```

    }
}
}
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Rendering;
using PlusLevelStudio.Editor;
namespace KnoxumPLSExtension.Features
{
    public static class HighWallsGenerator
    {
        public static void ProcessCell3D(global::Cell cell, int roomId)
        {
            if (cell == null || cell.Tile == null || roomId <= 0) return;
            int yOffset = HighWallsController.GetRoomYOffset(roomId);
            int targetHeight = HighWallsController.GetRoomHeight(roomId);
            float calculatedY = (float)yOffset * 10f;
            float topY = calculatedY + ((float)(targetHeight - 1) * 10f);
            Vector3 tilePos = cell.Tile.transform.localPosition;
            cell.Tile.transform.localPosition = new Vector3(tilePos.x, topY, tilePos.z);
            RemoveMeshFloor(cell.Tile, targetHeight);
            CreateBottomFloor(cell.Tile, topY, targetHeight);
            FixAndPositionLights(cell.Tile, targetHeight);
            BuildStackedWalls(cell, targetHeight, roomId);
        }

        //
        //
        private static void FixAndPositionLights(Tile tile, int targetHeight)
        {
            if (tile == null) return;
            float lightRange = 20f * Mathf.Max(1, targetHeight);
            LightController[] controllers = tile.GetComponentInChildren<LightController>(true);
            foreach (LightController lc in controllers)
            {
                if (lc == null) continue;
                Vector3 lp = lc.transform.localPosition;
                lc.transform.localPosition = new Vector3(lp.x, 9.5f, lp.z);
                Light l = lc.GetComponent<Light>();
                if (l == null) l = lc.GetComponentInChildren<Light>(true);
                if (l != null)
                {
                    ApplyLightFixSettings(l, targetHeight, lightRange);
                }
                MeshRenderer[] lampRenderers = lc.GetComponentInChildren<MeshRenderer>(true);
                foreach (var mr in lampRenderers)
                {
                    DisableProbeInfluence(mr);
                }
            }

            if (controllers.Length == 0)
            {
                Light[] fallbackLights = tile.GetComponentInChildren<Light>(true);
                foreach (Light l in fallbackLights)
                {
                    if (l == null) continue;
                    Vector3 lp = l.transform.localPosition;
                    l.transform.localPosition = new Vector3(lp.x, 9.5f, lp.z);
                    ApplyLightFixSettings(l, targetHeight, lightRange);
                }
            }
        }

        private static void ApplyLightFixSettings(Light l, int targetHeight, float lightRange)
        {
            l.type = LightType.Point;
            l.range = lightRange;
            l.intensity = Mathf.Max(l.intensity, 3f) * Mathf.Sqrt(targetHeight);
            l.bounceIntensity = 1f;
            l.shadows = LightShadows.None;
            l.renderMode = LightRenderMode.ForcePixel;
        }

        private static void DisableProbeInfluence(Renderer r)
        {
            if (r == null) return;
            r.lightProbeUsage = LightProbeUsage.Off;
            r.reflectionProbeUsage = ReflectionProbeUsage.Off;
        }

        private static void RemoveMeshFloor(Tile tile, int targetHeight)
        {

```



```

MeshFilter mf = tile.GetComponent<MeshFilter>();
if (mf == null || mf.sharedMesh == null) return;
Mesh meshCopy = UnityEngine.Object.Instantiate(mf.sharedMesh);
Vector3[] vertices = meshCopy.vertices;
Vector3[] normals = meshCopy.normals;
int[] triangles = meshCopy.triangles;
List<int> newTriangles = new List<int>();
for (int i = 0; i < triangles.Length; i += 3)
{
    int i1 = triangles[i];
    int i2 = triangles[i + 1];
    int i3 = triangles[i + 2];
    bool isAtFloorHeight = (vertices[i1].y <= 0.5f && vertices[i2].y <= 0.5f && vertices[i3].y <= 0.5f);
    bool isFacingUp = false;
    if (normals != null && normals.Length > i1)
    {
        Vector3 avgNormal = (normals[i1] + normals[i2] + normals[i3]) / 3f;
        if (avgNormal.y > 0.5f) isFacingUp = true;
    }
    if (!(targetHeight > 1 && isAtFloorHeight && isFacingUp))
    {
        newTriangles.Add(i1);
        newTriangles.Add(i2);
        newTriangles.Add(i3);
    }
}
meshCopy.triangles = newTriangles.ToArray();
meshCopy.RecalculateBounds();
mf.mesh = meshCopy;
}

private static void CreateBottomFloor(Tile tile, float topY, int targetHeight)
{
    Transform oldFloor = tile.transform.Find("Knoxum_Floor");
    if (oldFloor != null)
    {
        GameObject.DestroyImmediate(oldFloor.gameObject);
    }
    if (targetHeight <= 1) return;
    MeshRenderer mainRenderer = tile.GetComponent<MeshRenderer>() ?? tile.GetComponentInChildren<MeshRenderer>();
    if (mainRenderer == null || mainRenderer.sharedMaterial == null) return;
    GameObject floorObj = GameObject.CreatePrimitive(PrimitiveType.Quad);
    floorObj.name = "Knoxum_Floor";
    floorObj.transform.SetParent(tile.transform, false);
    floorObj.transform.localRotation = Quaternion.Euler(90f, 0f, 0f);
    floorObj.transform.localPosition = new Vector3(0f, -topY, 0f);
    floorObj.transform.localScale = new Vector3(10f, 10f, 1f);
    MeshFilter floorMf = floorObj.GetComponent<MeshFilter>();
    if (floorMf != null && floorMf.sharedMesh != null)
    {
        Mesh newFloorMesh = UnityEngine.Object.Instantiate(floorMf.sharedMesh);
        newFloorMesh.uv = new Vector2[]
        {
            new Vector2(0.0f, 0.0f),
            new Vector2(0.5f, 0.0f),
            new Vector2(0.0f, 0.5f),
            new Vector2(0.5f, 0.5f)
        };
        floorMf.mesh = newFloorMesh;
    }
    MeshRenderer floorRenderer = floorObj.GetComponent<MeshRenderer>();
    if (floorRenderer != null)
    {
        floorRenderer.sharedMaterial = mainRenderer.sharedMaterial;
        floorRenderer.shadowCastingMode = ShadowCastingMode.On;
        DisableProbeInfluence(floorRenderer);
    }
    var col = floorObj.GetComponent<Collider>();
    if (col != null) GameObject.DestroyImmediate(col);
}

private static void BuildStackedWalls(global::Cell cell, int targetHeight, int currentRoomId)
{
    Tile tileComponent = cell.Tile;
    if (tileComponent == null) return;
    Transform tileTransform = tileComponent.transform;
    List<GameObject> toDestroy = new List<GameObject>();
    foreach (Transform child in tileTransform)
    {
        if (child != null && child.name.StartsWith("Knoxum_Wall_"))
        {

```

```

        toDestroy.Add(child.gameObject);
    }
}
foreach (GameObject obj in toDestroy) { GameObject.DestroyImmediate(obj); }
if (targetHeight <= 1) return;
MeshRenderer mainRenderer = tileComponent.GetComponent<MeshRenderer>() ?? tileComponent.GetComponentInChildren<MeshRenderer>();
if (mainRenderer == null || mainRenderer.sharedMaterial == null) return;
Material wallMat = mainRenderer.sharedMaterial;
    Vector3[] localPositionsXZ = new Vector3[] {
        new Vector3(0f, 0f, 5f),        new Vector3(5f, 0f, 0f),        new Vector3(0f, 0f, -5f),        new Vector3(-5f, 0f, 0f)        };
Quaternion[] outerRotations = new Quaternion[] {
    Quaternion.Euler(0f, 0f, 0f),        Quaternion.Euler(0f, 90f, 0f),        Quaternion.Euler(0f, 180f, 0f),        Quaternion.Euler(0f, 270f, 0f)
};
bool[] hasWall = GetWallDirections(cell, currentRoomId);
for (int dir = 0; dir < 4; dir++)
{
    if (!hasWall[dir]) continue;
    for (int layer = 0; layer < targetHeight - 1; layer++)
    {
        float localY = -((float)(layer + 1) * 10f) + 5f;
        Vector3 basePos = localPositionsXZ[dir] + new Vector3(0f, localY, 0f);
        CreateWallQuad($"Knoxum_Wall_L{layer}_D{dir}", tileTransform, basePos, outerRotations[dir], wallMat);
    }
}
}
//
//
//
private static void CreateWallQuad(string name, Transform parent, Vector3 localPos, Quaternion localRot,
Material mat)
{
    GameObject wallPiece = GameObject.CreatePrimitive(PrimitiveType.Quad);
    wallPiece.name = name;
    wallPiece.isStatic = false;    wallPiece.transform.SetParent(parent, false);
    wallPiece.transform.localScale = new Vector3(10f, 10f, 1f);
    wallPiece.transform.localRotation = localRot;
    wallPiece.transform.localPosition = localPos;
    MeshFilter mf = wallPiece.GetComponent<MeshFilter>();
    if (mf != null && mf.sharedMesh != null)
    {
        Mesh newMesh = UnityEngine.Object.Instantiate(mf.sharedMesh);

        newMesh.uv = new Vector2[]
        {
            new Vector2(0.5f, 0.5f),
            new Vector2(1.0f, 0.5f),
            new Vector2(0.5f, 1.0f),
            new Vector2(1.0f, 1.0f)
        };
        mf.mesh = newMesh;
    }
    MeshRenderer pieceRenderer = wallPiece.GetComponent<MeshRenderer>();
    if (pieceRenderer != null)
    {
        pieceRenderer.sharedMaterial = mat;
        pieceRenderer.shadowCastingMode = ShadowCastingMode.Off;
        pieceRenderer.receiveShadows = true;
        DisableProbeInfluence(pieceRenderer);
    }
    var col = wallPiece.GetComponent<Collider>();
    if (col != null) GameObject.DestroyImmediate(col);
}
private static bool[] GetWallDirections(global::Cell cell, int currentRoomId)
{
    bool[] walls = new bool[] { true, true, true, true };
    int x = cell.position.x;
    int z = cell.position.z;
    var editorLevel = Singleton<EditorController>.Instance?.levelData;
    if (editorLevel != null)
    {
        var n = editorLevel.GetCellSafe(x, z + 1);
        var e = editorLevel.GetCellSafe(x + 1, z);
        var s = editorLevel.GetCellSafe(x, z - 1);
        var w = editorLevel.GetCellSafe(x - 1, z);
        int currentYOffset = HighWallsController.GetRoomYOffset(currentRoomId);
        int currentHeight = HighWallsController.GetRoomHeight(currentRoomId);
        walls[0] = ShouldBuildWall(n, currentRoomId, currentYOffset, currentHeight);
        walls[1] = ShouldBuildWall(e, currentRoomId, currentYOffset, currentHeight);
        walls[2] = ShouldBuildWall(s, currentRoomId, currentYOffset, currentHeight);
    }
}

```

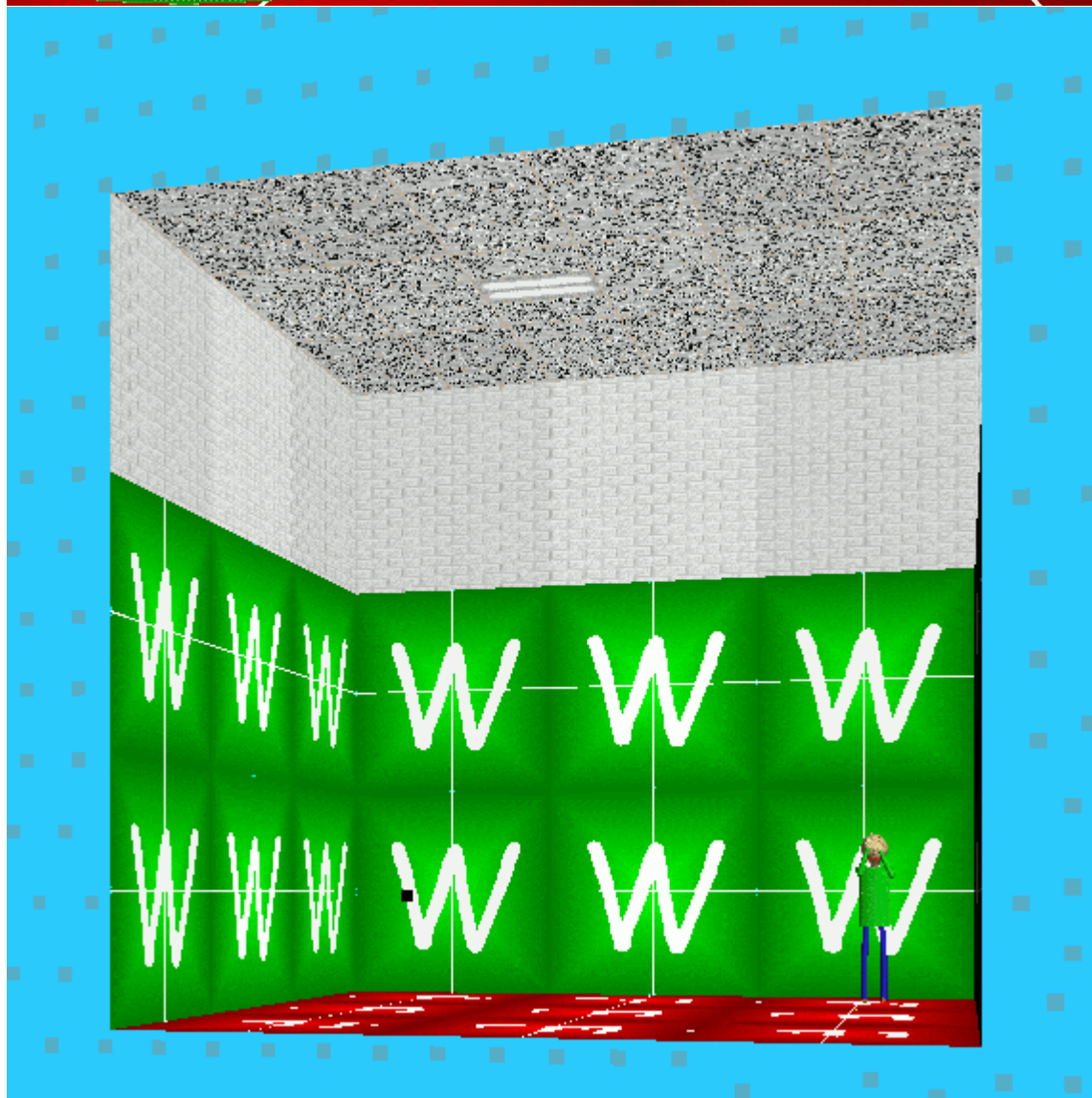
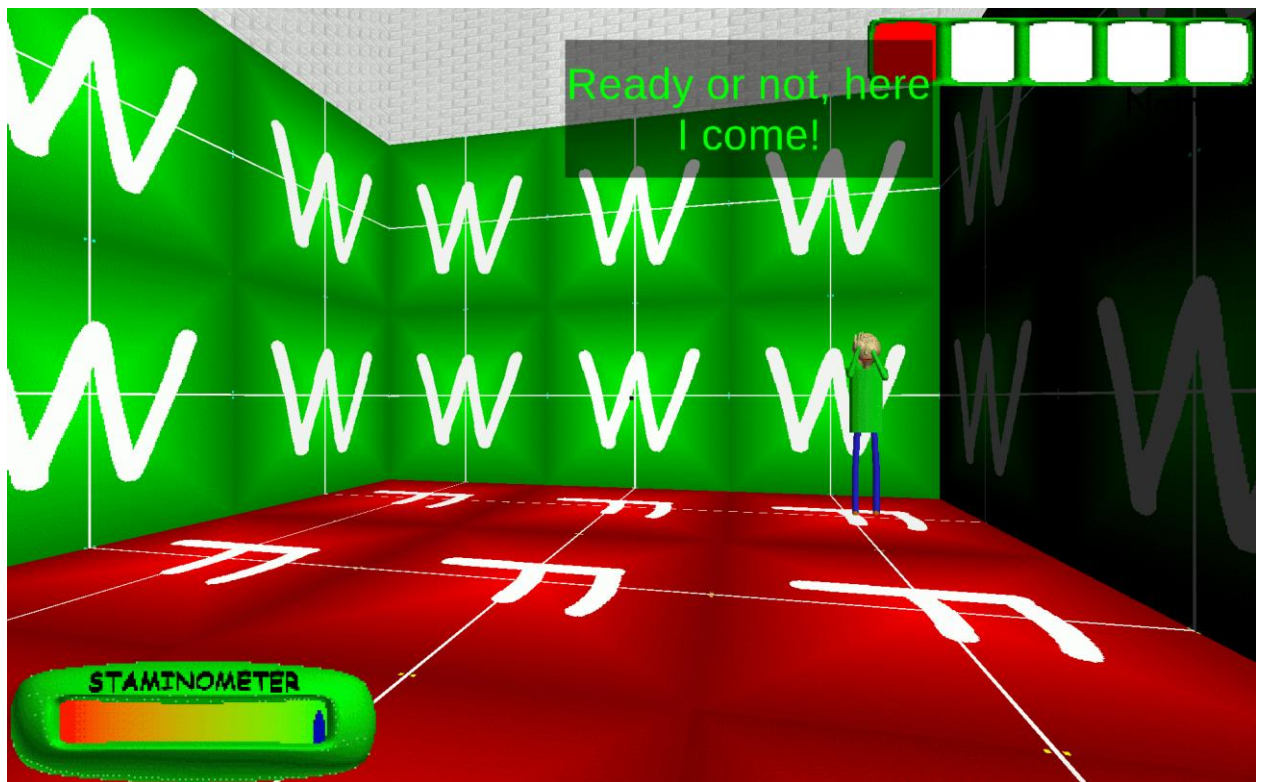
```

        walls[3] = ShouldBuildWall(w, currentRoomId, currentYOffset, currentHeight);
        return walls;
    }
}
var ec = Singleton<BaseGameManager>.Instance?.Ec;
if (ec != null && ec.cells != null)
{
    global::Cell currentCell = GetCellRuntime(ec, x, z);
    var currentRoom = currentCell?.room;
    var n = GetCellRuntime(ec, x, z + 1);
    var e = GetCellRuntime(ec, x + 1, z);
    var s = GetCellRuntime(ec, x, z - 1);
    var w = GetCellRuntime(ec, x - 1, z);
    walls[0] = ShouldBuildWallRuntime(n, currentRoom);
    walls[1] = ShouldBuildWallRuntime(e, currentRoom);
    walls[2] = ShouldBuildWallRuntime(s, currentRoom);
    walls[3] = ShouldBuildWallRuntime(w, currentRoom);
}
return walls;
}
private static bool ShouldBuildWall(PlusStudioLevelFormat.Cell neighbor, int currentRoomId, int currentYOffset, int currentHeight)
{
    if (neighbor == null) return true;
    if (neighbor.roomId != currentRoomId) return true;
    int neighborYOffset = HighWallsController.GetRoomYOffset(neighbor.roomId);
    int neighborHeight = HighWallsController.GetRoomHeight(neighbor.roomId);
    return currentYOffset != neighborYOffset || currentHeight != neighborHeight;
}
private static bool ShouldBuildWallRuntime(global::Cell neighbor, RoomController currentRoom)
{
    if (neighbor == null || neighbor.room == null) return true;
    if (neighbor.room != currentRoom) return true;
    return false;
}
private static global::Cell GetCellRuntime(EnvironmentController ec, int x, int z)
{
    if (ec.cells == null) return null;
    int width = ec.cells.GetLength(0);
    int height = ec.cells.GetLength(1);
    if (x < 0 || z < 0 || x >= width || z >= height) return null;
    return ec.cells[x, z];
}
}
        public static int GetRoomIdForCell(global::Cell cell)
    {
        if (cell == null) return -1;
        var editorCtrl = Singleton<EditorController>.Instance;
        if (editorCtrl != null && editorCtrl.levelData != null)
        {
            var area = editorCtrl.levelData.GetCellSafe(cell.position.x, cell.position.z);
            if (area != null && area.roomId != 0)
                return (int)area.roomId;
            var map = HighWallsController.runtimeCellRoomIdMap;
            if (map.Count > 0)
            {
                var key = (cell.position.x, cell.position.z);
                if (map.TryGetValue(key, out ushort mappedId) && mappedId != 0)
                    return (int)mappedId;
                if (cell.room != null)
                {
                    var ec = cell.room.ec;
                    if (ec != null && ec.rooms != null)
                    {
                        int idx = ec.rooms.IndexOf(cell.room);
                        if (idx >= 0) return idx + 1;
                    }
                }
            }
        }
        return -1;
    }
}
}
}

```

баги:

свет крепится к потолку только в игре! не в редакторе + до сих пор не решена проблема со сломанным светом + надо теперь сделать нормальную генерацию стен и пола в игре! (есть скриншоты)



Много кусков кода:

```
using System;
using System.Collections.Generic;
using UnityEngine;
public class Cell : IHeapItem<Cell>, IComparable<Cell>
{
    public TileShapeMask shape { get; private set; } = TileShapeMask.Open;
    public void Initialize()
    {
        this.initialized = true;
        this.doorDirs = new List<Direction>();
        this.doors = new List<Door>();
        this.doorDirsSpace = new List<Direction>();
        this._openDirs = new List<Direction>();
        this.openTiles = new List<Cell>();
    }
    public void Uninitialize()
    {
        this.initialized = false;
        this.doorDirs.Clear();
        this.doors.Clear();
        this.doorDirsSpace.Clear();
        this._openDirs.Clear();
        this.openTiles.Clear();
        this.hasChunk = false;
        this.chunk = null;
        this.SetShape(0, TileShapeMask.Open);
        this.hardCoverage = CellCoverage.None;
        this.softCoverage = CellCoverage.None;
        if (this.tileLoaded)
        {
            Object.Destroy(this.tile.gameObject);
        }
        this.tileLoaded = false;
    }
    public void LoadTile()
    {
        this.tile = Object.Instantiate<Tile>(this.room.ec.TilePre, this.room.transform);
        this.tile.transform.localPosition = new Vector3((float)(this.position.x * 10 + 5), 0f, (float)(this.position.z * 10 + 5));
        this.tileLoaded = true;
        this.LoadTileMesh();
        this.HideTile(this.tileHidden);
    }
    private void LoadTileMesh()
    {
        if (this.tileLoaded)
        {
            this.tile.MeshFilter.sharedMesh = this.room.ec.TileMesh(this.constBin);
            for (int i = 0; i < 4; i++)
            {
                this.tile.Collider((Direction)i).SetActive(this.constBin.ContainsDirection((Direction)i));
            }
        }
    }
    public void SetShape(int bin, TileShapeMask shape)
    {
        this.constBin = bin;
        this.navBin = bin;
        this.soundBin = bin;
        this.shape = shape;
        if (this.tileLoaded)
        {
            this.LoadTileMesh();
        }
        this.room.ec.RecalculateNavigation();
        GameCamera.RecalculateDijkstraMap();
    }
    public void HideTile(bool value)
    {
        if (this.tileLoaded)
        {
            this.tile.MeshRenderer.enabled = !value;
        }
        this.tileHidden = value;
    }
    public void SetBase(Material mat)
    {
        if (this.tileLoaded)
        {
            this.tile.BaseMaterial = mat;
        }
    }
}
```

```

this.tile.MeshRenderer.sharedMaterial = mat;
}
}
public void PrepareForPoster()
{
if (this.tile.MeshRenderer.sharedMaterial.shader.name != "Shader Graphs/TileStandardWPoster")
{
MaterialPropertyBlock materialPropertyBlock = new MaterialPropertyBlock();
this.tile.MeshRenderer.sharedMaterial = this.room.posterMat;
this.tile.MeshRenderer.GetPropertyBlock(materialPropertyBlock);
materialPropertyBlock.SetTexture("_PosterN", this.room.defaultPoster);
materialPropertyBlock.SetTexture("_PosterE", this.room.defaultPoster);
materialPropertyBlock.SetTexture("_PosterS", this.room.defaultPoster);
materialPropertyBlock.SetTexture("_PosterW", this.room.defaultPoster);
this.tile.MeshRenderer.SetPropertyBlock(materialPropertyBlock);
}
}
public void AddPoster(Direction dir, Texture overlay)
{
this.PrepareForPoster();
MaterialPropertyBlock materialPropertyBlock = new MaterialPropertyBlock();
this.tile.MeshRenderer.GetPropertyBlock(materialPropertyBlock);
switch (dir)
{
case Direction.North:
materialPropertyBlock.SetTexture("_PosterN", overlay);
break;
case Direction.East:
materialPropertyBlock.SetTexture("_PosterE", overlay);
break;
case Direction.South:
materialPropertyBlock.SetTexture("_PosterS", overlay);
break;
case Direction.West:
materialPropertyBlock.SetTexture("_PosterW", overlay);
break;
}
this.tile.MeshRenderer.SetPropertyBlock(materialPropertyBlock);
this.SoftCoverWall(dir);
}
public void AddRenderer(Renderer renderer)
{
this.renderers.Add(renderer);
}
public void AssignDoor(Door door, Direction direction)
{
this.room.doors.Add(door);
this.doorDirsSpace.Add(direction);
this.doorHere = true;
if (!this.doorDirs.Contains(direction))
{
this.doorDirs.Add(direction);
}
this.doors.Add(door);
this.HardCoverWall(direction, true);
}
public bool DoorInDirection(Direction direction, out Door door)
{
foreach (Door door2 in this.doors)
{
if (door2.aTile == this)
{
if (door2.direction == direction)
{
door = door2;
return true;
}
}
else if (door2.direction.GetOpposite() == direction)
{
door = door2;
return true;
}
}
door = null;
return false;
}
public bool TileMatches(RoomController checkRoom)
{

```

```

return this.room == checkRoom;
}
private Direction RandomOccupiedDirection(int bin, Random rng)
{
    int num = 0;
    for (int i = 0; i < 4; i++)
    {
        if ((bin & 1 << i) > 0)
        {
            Cell._potentialDirs[num] = i;
            num++;
        }
    }
    if (num > 0)
    {
        return (Direction)Cell._potentialDirs[rng.Next(0, num)];
    }
    return Direction.Null;
}
private Direction RandomUnoccupiedDirection(int bin, Random rng)
{
    int num = 0;
    for (int i = 0; i < 4; i++)
    {
        if ((bin & 1 << i) == 0)
        {
            Cell._potentialDirs[num] = i;
            num++;
        }
    }
    if (num > 0)
    {
        return (Direction)Cell._potentialDirs[rng.Next(0, num)];
    }
    return Direction.Null;
}
public Direction RandomConstDirection(Random rng)
{
    return this.RandomOccupiedDirection(this.ConstBin, rng);
}
public Direction RandomUncoveredDirection(Random rng)
{
    return this.RandomUnoccupiedDirection(this.SoftWallAvailabilityBin, rng);
}
public List<Direction> AllWallDirections
{
    get
    {
        List<Direction> list = new List<Direction>();
        for (int i = 0; i < 4; i++)
        {
            if ((this.ConstBin & 1 << i) > 0)
            {
                list.Add((Direction)i);
            }
        }
        return list;
    }
}
public List<Direction> AllOpenNavDirections
{
    get
    {
        Directions.FillOpenDirectionsFromBin(this._openDirs, this.navBin);
        return this._openDirs;
    }
}
public int ConstBin
{
    get
    {
        return this.constBin;
    }
}
public int NavBin
{
    get
    {
        return this.navBin;
    }
}

```

```

    }
    set
    {
        this.navBin = value;
    }
    public int SoundBin
    {
        get
        {
            return this.soundBin;
        }
    }
    public int HardCoverageBin
    {
        get
        {
            return (int)this.hardCoverage;
        }
    }
    public int HardWallAvailabilityBin
    {
        get
        {
            return ~(this.ConstBin ^ (int)this.hardCoverage) | (int)this.hardCoverage;
        }
    }
    public bool HardCoverageFits(CellCoverage coverage)
    {
        return (coverage & (CellCoverage)this.HardCoverageBin) == CellCoverage.None;
    }
    public bool HardCoverageFitsInAnyDirection(CellCoverage coverage)
    {
        for (int i = 0; i < 4; i++)
        {
            if (this.HardCoverageFits(coverage.Rotated((Direction)i)))
            {
                return true;
            }
        }
        return false;
    }
    public bool HardCoverageFitsInDirection(CellCoverage coverage, Direction direction)
    {
        return this.HardCoverageFits(coverage.Rotated(direction));
    }
    public bool HasAnyHardCoverage
    {
        get
        {
            return this.hardCoverage > CellCoverage.None;
        }
    }
    public bool WallHardCovered(Direction dir)
    {
        return (this.hardCoverage & (CellCoverage)(1 << (int)dir)) > CellCoverage.None;
    }
    public void FilterDirectionsThroughHardCoverage(List<Direction> list, bool debug)
    {
        for (int i = 0; i < list.Count; i++)
        {
            if (debug)
            {
                Debug.Log(string.Format("wall covered {0} = {1}", list[i], this.WallHardCovered(list[i])));
            }
            if (this.WallHardCovered(list[i]))
            {
                list.RemoveAt(i);
                i--;
            }
        }
    }
    public int SoftCoverageBin
    {
        get
        {
            return (int)this.softCoverage;
        }
    }

```



```

    }
    public int SoftWallAvailabilityBin
    {
        get
        {
            return ~(this.ConstBin ^ (int)this.softCoverage) | (int)this.softCoverage;
        }
    }
    public bool SoftCoverageFits(CellCoverage coverage)
    {
        return (coverage & (CellCoverage)this.SoftCoverageBin) == CellCoverage.None;
    }
    public bool SoftCoverageFitsInAnyDirection(CellCoverage coverage)
    {
        for (int i = 0; i < 4; i++)
        {
            if (this.SoftCoverageFits(coverage.Rotated((Direction)i)))
            {
                return true;
            }
        }
        return false;
    }
    public bool SoftCoverageFitsInDirection(CellCoverage coverage, Direction direction)
    {
        return this.SoftCoverageFits(coverage.Rotated(direction));
    }
    public bool WallSoftCovered(Direction dir)
    {
        return (this.softCoverage & (CellCoverage)(1 << (int)dir)) > CellCoverage.None;
    }
    public void FilterDirectionsThroughSoftCoverage(List<Direction> list)
    {
        for (int i = 0; i < list.Count; i++)
        {
            if (this.WallSoftCovered(list[i]))
            {
                list.RemoveAt(i);
                i--;
            }
        }
    }
    public bool AllCoverageFits(CellCoverage coverage)
    {
        return this.HardCoverageFits(coverage) && this.SoftCoverageFits(coverage);
    }
    public bool AllCoverageFitsInAnyDirection(CellCoverage coverage)
    {
        for (int i = 0; i < 4; i++)
        {
            if (this.HardCoverageFits(coverage.Rotated((Direction)i)) && this.SoftCoverageFits(coverage.Rotated((Direction)i)))
            {
                return true;
            }
        }
        return false;
    }
    public bool AllCoverageFitsInDirection(CellCoverage coverage, Direction direction)
    {
        return this.AllCoverageFits(coverage.Rotated(direction));
    }
    public bool HasAnyAllCoverage
    {
        get
        {
            return this.hardCoverage > CellCoverage.None || this.softCoverage > CellCoverage.None;
        }
    }
    public bool WallAnyCovered(Direction dir)
    {
        return this.WallHardCovered(dir) || this.WallSoftCovered(dir);
    }
    public int fCost
    {
        get
        {
            return this.gCost + this.hCost;
        }
    }

```

```

}
public int HeapIndex
{
    get
    {
        return this.heapIndex;
    }
    set
    {
        this.heapIndex = value;
    }
}
public int CompareTo(Cell tileToCompare)
{
    int num = this.fCost.CompareTo(tileToCompare.fCost);
    if (num == 0)
    {
        num = this.hCost.CompareTo(tileToCompare.hCost);
    }
    return -num;
}
public bool Navigable(Direction dir, PathType pathType)
{
    switch (pathType)
    {
        case PathType.Const:
            return this.ConstNavigable(dir);
        case PathType.Nav:
            return this.NavNavigable(dir);
        case PathType.Sound:
            return this.SoundNavigable(dir);
        default:
            return false;
    }
}
public bool ConstNavigable(Direction dir)
{
    return (this.ConstBin & 1 << (int)dir) == 0;
}
public bool NavNavigable(Direction dir)
{
    return (this.NavBin & 1 << (int)dir) == 0;
}
public bool SoundNavigable(Direction dir)
{
    return (this.SoundBin & 1 << (int)dir) == 0;
}
public int WallCount
{
    get
    {
        int num = 0;
        for (int i = 0; i < 4; i++)
        {
            if ((this.ConstBin & 1 << i) > 0)
            {
                num++;
            }
        }
        return num;
    }
}
public bool HasWallInDirection(Direction dir)
{
    return (this.ConstBin & 1 << (int)dir) > 0;
}
public void HardCover(CellCoverage coverage)
{
    this.hardCoverage |= coverage;
    this.SoftCover(coverage);
}
public void HardCoverInDirection(CellCoverage coverage, Direction direction)
{
    this.HardCover(coverage.Rotated(direction));
}
public void HardCoverEntirely()
{
    this.HardCover(CellCoverage.North | CellCoverage.East | CellCoverage.South | CellCoverage.West | CellCoverage.Up | CellCoverage.Down | CellCoverage.Center);
}

```

```

}
public void HardCoverWall(Direction dir, bool covered)
{
if ((this.hardCoverage & (CellCoverage)(1 << (int)dir)) == CellCoverage.None)
{
this.hardCoverage |= (CellCoverage)(1 << (int)dir);
}
}
this.SoftCoverWall(dir);
}
public void SoftCoverWall(Direction dir)
{
if ((this.softCoverage & (CellCoverage)(1 << (int)dir)) == CellCoverage.None)
{
this.softCoverage |= (CellCoverage)(1 << (int)dir);
}
}
}
public void SoftCover(CellCoverage coverage)
{
this.softCoverage |= coverage;
}
}
public void SoftCoverInDirection(CellCoverage coverage, Direction direction)
{
this.SoftCover(coverage.Rotated(direction));
}
}
public void Block(Direction dir, bool block)
{
if (block)
{
this.blocks[(int)dir]++;
if ((this.navBin & 1 << (int)dir) == 0)
{
this.navBin += 1 << (int)dir;
this.room.ec.RecalculateNavigation();
return;
}
}
else
{
this.blocks[(int)dir]--;
if (this.blocks[(int)dir] < 0)
{
this.blocks[(int)dir] = 0;
}
if ((this.navBin & 1 << (int)dir) > 0 && this.blocks[(int)dir] <= 0 && (this.constBin & 1 << (int)dir) == 0)
{
this.navBin -= 1 << (int)dir;
this.room.ec.RecalculateNavigation();
}
}
}
}
public void Mute(Direction dir, bool block)
{
if (block)
{
this.mutes[(int)dir]++;
if ((this.soundBin & 1 << (int)dir) == 0)
{
this.soundBin += 1 << (int)dir;
GameCamera.RecalculateDijkstraMap();
return;
}
}
else
{
this.mutes[(int)dir]--;
if (this.mutes[(int)dir] < 0)
{
this.mutes[(int)dir] = 0;
}
if ((this.soundBin & 1 << (int)dir) > 0 && this.mutes[(int)dir] <= 0)
{
this.soundBin -= 1 << (int)dir;
GameCamera.RecalculateDijkstraMap();
}
}
}
}
public void SilentBlock(Direction dir, bool block)
{
this.room.ec FreezeNavigationUpdates(true);
}

```

```

this.Block(dir, block);
this.room.ec.FreezeNavigationUpdates(false);
}
public void RefreshNavBin()
{
for (int i = 0; i < 4; i++)
{
if (this.blocks[i] > 0)
{
this.navBin |= 1 << i;
}
}
}
public bool HasSoftFreeWall
{
get
{
for (int i = 0; i < 4; i++)
{
if ((this.SoftWallAvailabilityBin & 1 << i) == 0)
{
return true;
}
}
return false;
}
}
public bool HasHardFreeWall
{
get
{
for (int i = 0; i < 4; i++)
{
if ((this.HardWallAvailabilityBin & 1 << i) == 0)
{
return true;
}
}
return false;
}
}
public bool HasAllFreeWall
{
get
{
return this.HasHardFreeWall && this.HasSoftFreeWall;
}
}
public void SetSilence(bool value)
{
if (value)
{
this.silences++;
this.silent = true;
return;
}
this.silences--;
if (this.silences <= 0)
{
this.silent = false;
}
}
public bool Silent
{
get
{
return this.silent;
}
}
public void SetLight(bool on)
{
if (this.hasLight)
{
this.lightOn = on;
this.room.ec.UpdateLightingAtCell(this);
LightController lightController = this.lightController;
if (lightController == null)
{
return;
}
}
}

```

```

    }
    lightController.Set(on);
    }
    }
    public void AssignLightController(Transform controllerToAssign)
    {
        RendererContainer component = controllerToAssign.GetComponent<RendererContainer>();
        if (component != null)
        {
            this.renderers.AddRange(component.renderers);
        }
        this.lightController = controllerToAssign.GetComponent<LightController>();
        LightController lightController = this.lightController;
        if (lightController == null)
        {
            return;
        }
        lightController.Set(this.lightOn);
    }
    public void SetPower(bool power)
    {
        if (!power)
        {
            this.lightPowerRemovals++;
        }
        else
        {
            this.lightPowerRemovals--;
            if (this.lightPowerRemovals <= 0)
            {
                this.lightPowerRemovals = 0;
            }
        }
        this.room.ec.UpdateLightingAtCell(this);
    }
    public void Flicker()
    {
        this.SetPower(this.lightPowerFlickered);
        this.lightPowerFlickered = !this.lightPowerFlickered;
    }
    public void FixFlicker()
    {
        if (this.lightPowerFlickered)
        {
            this.SetPower(true);
        }
    }
    public Cell RandomEntitySafeCellFromRoomIfAnyNoGarbage()
    {
        if (this.room.entitySafeCells.Count > 0)
        {
            return this.room.ec.CellFromPosition(this.room.entitySafeCells[Random.Range(0, this.room.entitySafeCells.Count)]);
        }
        return this;
    }
    public Color CurrentColor
    {
        get
        {
            if (this.lightOn && this.lightPowerRemovals <= 0)
            {
                return this.lightColor;
            }
            return Color.black;
        }
    }
    public bool HasObjectBase
    {
        get
        {
            return this.objectBase != null;
        }
    }
    public Vector3 FloorWorldPosition
    {
        get
        {
            return new Vector3((float)this.position.x * 10f + 5f, 0f, (float)this.position.z * 10f + 5f);
        }
    }

```

```

    }
    public Vector3 CenterWorldPosition
    {
        get
        {
            return new Vector3((float)this.position.x * 10f + 5f, 5f, (float)this.position.z * 10f + 5f);
        }
    }
    public Vector3 RandomCenterWorldPosition
    {
        get
        {
            return this.CenterWorldPosition + new Vector3(Random.Range(5f, -5f), 0f, Random.Range(5f, -5f));
        }
    }
    public Transform TileTransform
    {
        get
        {
            {
                if (this.tile == null)
                {
                    return null;
                }
                return this.tile.transform;
            }
        }
    }
    public Tile Tile
    {
        get
        {
            return this.tile;
        }
        set
        {
            {
                this.tile = value;
            }
        }
    }
    public int Mutes(Direction direction)
    {
        {
            return this.mutes[(int)direction];
        }
    }
    public bool Null
    {
        get
        {
            {
                return !this.initialized;
            }
        }
    }
    public bool Hidden
    {
        get
        {
            return this.tileHidden;
        }
    }
    public Transform ObjectBase
    {
        get
        {
            {
                if (this.objectBase == null)
                {
                    this.objectBase = new GameObject().transform;
                    this.objectBase.name = "ObjectBase";
                    this.objectBase.transform.SetParent(this.TileTransform, false);
                }
                return this.objectBase;
            }
        }
        set
        {
            {
                this.objectBase = value;
            }
        }
    }
    public bool TileLoaded
    {
        get
        {
            {
                return this.tileLoaded;
            }
        }
    }

```

```

}
public void SetChunk(Chunk chunk)
{
    this.chunk = chunk;
    this.hasChunk = true;
}
public void RemoveChunk()
{
    this.hasChunk = false;
}
public bool HasChunk
{
    get
    {
        return this.hasChunk;
    }
}
public Chunk Chunk
{
    get
    {
        return this.chunk;
    }
}
private const CellCoverage fullCoverage = CellCoverage.North | CellCoverage.East | CellCoverage.South | CellCoverage.West | CellCoverage.Up |
CellCoverage.Down | CellCoverage.Center;
public RoomController room;
public IntVector2 position;
public Color lightColor;
public int label;
public int lightStrength;
private int lightPowerRemovals;
private int silences;
public bool labeled;
public bool open;
public bool locked;
public bool blockHallRemoval;
public bool hasLight;
public bool lightOn;
public bool lightHasPower = true;
public bool permanentLight;
public bool excludeFromOpen;
public bool offLimits;
public bool hideFromMap;
public bool excludeFromRoomGroup;
private bool initialized;
private bool tileLoaded;
private bool tileHidden;
private bool silent;
private bool lightPowerFlickered;
private CellCoverage hardCoverage;
private CellCoverage softCoverage;
private Tile tile;
private Transform objectBase;
public List<Renderer> renderers = new List<Renderer>();
private LightController lightController;
public bool doorHere;
public List<Direction> doorDirs;
public List<Door> doors;
public List<Direction> doorDirsSpace;
private int constBin;
private int navBin;
private int soundBin;
private int heapIndex;
private int _ogNavBin;
private int[] blocks = new int[4];
private int[] mutes = new int[4];
private static int[] _potentialDirs = new int[4];
private Chunk chunk;
private bool hasChunk;
public Cell parent;
private List<Direction> _openDirs = new List<Direction>();
public List<Cell> openTiles = new List<Cell>();
public List<Cell> lightAffectingCells = new List<Cell>();
public OpenTileGroup openTileGroup;
public int gCost;
public int hCost;
}
using System;

```

```

using UnityEngine;
public class Tile : MonoBehaviour
{
    public MeshFilter MeshFilter
    {
        get
        {
            return this.meshFilter;
        }
    }
    public MeshRenderer MeshRenderer
    {
        get
        {
            return this.meshRenderer;
        }
    }
    public GameObject Collider(Direction dir)
    {
        return this.collider[(int)dir];
    }
    [SerializeField]
    private MeshFilter meshFilter;
    [SerializeField]
    private MeshRenderer meshRenderer;
    [SerializeField]
    private GameObject[] collider = new GameObject[4];
}

using System;
using System.Collections;
using System.Collections.Generic;
using System.Diagnostics;
using UnityEngine;
public class LevelGenerator : LevelBuilder
{
    public override void StartGenerate()
    {
        base.StartGenerate();
        base.StartCoroutine(this.Generate());
    }
    public IEnumerator Generate()
    {
        {
            this.levelCreated = false;
            this.levelInProgress = true;
            Stopwatch sw = new Stopwatch();
            sw.Start();
            this.controlledRNG = new Random(Singleton<CoreGameManager>.Instance.Seed());
            Debug.Log(string.Concat(new string[]
            {
                "Generating level ",
                this.ec.name,
                this.seedOffset.ToString(),
                " with seed:",
                Singleton<CoreGameManager>.Instance.Seed().ToString()
            }));
            this.ec.SetTileInstantiation(false);
            this.generateAtlases = false;
            yield return null;
            yield return null;
            this.ec.npcsToSpawn = new List<NPC>();
            base.AddNpcsFromPreviousLevels();
            foreach (NPC item in this.ld.forcedNpcs)
            {
                this.ec.npcsToSpawn.Add(item);
            }
            if (this.scene.baldiPrefab != null)
            {
                this.ec.npcsToSpawn.Add(this.scene.baldiPrefab);
            }
            this.controlledRNG = new Random(Singleton<CoreGameManager>.Instance.Seed() + this.seedOffset);
            this.levelSize = new IntVector2(this.controlledRNG.Next(this.ld.minSize.x, this.ld.maxSize.x + 1) + this.ld.outerEdgeBuffer * 2,
            this.controlledRNG.Next(this.ld.minSize.z, this.ld.maxSize.z + 1) + this.ld.outerEdgeBuffer * 2);
            if (this.levelSize.x > 256 || this.levelSize.z > 256)
            {
                Debug.LogWarning(string.Format("Level size in at least one dimension was greater than 256 at {0},{1}! Cancelling level generation", this.levelSize.x,
                this.levelSize.z));
                yield break;
            }
            if (this.controlledRNG.Next(0, 2) == 1)

```



```

{
    int x2 = this.levelSize.x;
    this.levelSize.x = this.levelSize.z;
    this.levelSize.z = x2;
}
this.ec.InitializeCells(this.levelSize);
if (base.FrameShouldEnd())
{
    yield return null;
}
this.ec.levelSize = this.levelSize;
this.ec.InitializeHeap();
if (base.FrameShouldEnd())
{
    yield return null;
}
this.ec.standardDarkLevel = this.ld.standardDarkLevel;
this.ec.InitializeLighting();
if (base.FrameShouldEnd())
{
    yield return null;
}
base.CreateMap();
if (base.FrameShouldEnd())
{
    yield return null;
}
this.plots.Clear();
this.halls.Clear();
this.doorPlacements.Clear();
this.pickups.Clear();
int plotCount = this.controlledRNG.Next(this.ld.minPlots, this.ld.maxPlots + 1);
int hallsToRemove = this.controlledRNG.Next(this.ld.minHallsToRemove, this.ld.maxHallsToRemove + 1);
this.controlledRNG.Next(this.ld.minSideHallsToRemove, this.ld.maxSideHallsToRemove + 1);
int hallsToAdd = this.controlledRNG.Next(this.ld.minReplacementHalls, this.ld.maxReplacementHalls + 1);
List<ItemObject> lootTable = new List<ItemObject>();
lootTable.AddRange(this.ld.forcedItems);
Random itemRng = new Random();
WeightedItemObject[] array = new WeightedItemObject[this.ld.potentialItems.Length];
for (int k = 0; k < this.ld.potentialItems.Length; k++)
{
    array[k] = new WeightedItemObject();
    array[k].selection = this.ld.potentialItems[k].selection;
    array[k].weight = this.ld.potentialItems[k].weight;
}
int l = 0;
while (l < this.ld.maxItemValue)
{
    WeightedSelection<ItemObject>[] items = array;
    int num = WeightedSelection<ItemObject>.ControlledRandomIndex(items, itemRng);
    lootTable.Add(array[num].selection);
    l += lootTable[lootTable.Count - 1].value;
    array[num].weight = Mathf.Max(array[num].weight - Mathf.RoundToInt((float)array[num].weight * this.ld.duplicateItemWeightReduction), 1);
}
int eventCount = this.controlledRNG.Next(this.ld.minEvents, this.ld.maxEvents);
List<WeightedRandomEvent> potentialEvents = new List<WeightedRandomEvent>(this.ld.randomEvents);
List<RandomEvent> eventsToLaunch = new List<RandomEvent>();
List<RandomEvent> events = new List<RandomEvent>();
RandomEvent timeOutEvent = this.ld.timeOutEvent;
int i = 0;
int j;
while (i < eventCount && potentialEvents.Count > 0)
{
    RandomEvent randomEvent = WeightedSelection<RandomEvent>.ControlledRandomSelectionList(WeightedRandomEvent.Convert(potentialEvents),
    this.controlledRNG);
    eventsToLaunch.Add(randomEvent);
    for (int m = 0; m < potentialEvents.Count; m++)
    {
        if (potentialEvents[m].selection == randomEvent)
        {
            potentialEvents.RemoveAt(m);
            m--;
        }
    }
}
if (base.FrameShouldEnd())
{
    yield return null;
}
j = i;

```

```

i = j + 1;
}
foreach (RandomEvent original in eventsToLaunch)
{
    RandomEvent randomEvent2 = Object.Instantiate<RandomEvent>(original, this.ec.transform);
    randomEvent2.Initialize(this.ec, this.controlledRNG);
    randomEvent2.SetEventTime(new Random(Random.Range(0, int.MaxValue)));
    events.Add(randomEvent2);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<RandomEvent>.Enumerator enumerator = default(List<RandomEvent>.Enumerator);
if (timeOutEvent != null)
{
    timeOutEvent = Object.Instantiate<RandomEvent>(timeOutEvent, this.ec.transform);
    timeOutEvent.Initialize(this.ec, this.controlledRNG);
}
if (base.FrameShouldEnd())
{
    yield return null;
}
using (List<RandomEvent>.Enumerator enumerator2 = events.GetEnumerator())
{
    while (enumerator2.MoveNext())
    {
        if (enumerator2.Current.requiresSpawnerRooms)
        {
            this.ld.roomGroup.Insert(0, this.eventSpawnerRoomGroup);
            break;
        }
    }
}
if (base.FrameShouldEnd())
{
    yield return null;
}
List<StructureWithParameters> structuresToGenerate = new List<StructureWithParameters>();
List<StructureBuilder> initializedStructures = new List<StructureBuilder>();
new List<StructureBuilderData>();
List<WeightedStructureWithParameters> list = new List<WeightedStructureWithParameters>(this.ld.potentialStructures);
int num2 = this.controlledRNG.Next(this.ld.minSpecialBuilders, this.ld.maxSpecialBuilders);
int num3 = 0;
while (num3 < num2 && list.Count > 0)
{
    int index = WeightedSelection<StructureWithParameters>.ControlledRandomIndexList(WeightedStructureWithParameters.Convert(list),
    this.controlledRNG);
    structuresToGenerate.Add(list[index].selection);
    list.RemoveAt(index);
    num3++;
}
structuresToGenerate.AddRange(this.ld.forcedStructures);
if (base.FrameShouldEnd())
{
    yield return null;
}
if (base.FrameShouldEnd())
{
    yield return null;
}
WeightedSelection<Texture2D>[] items2 = this.ld.hallWallTexs;
Texture2D hallWallTex = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
items2 = this.ld.hallFloorTexs;
Texture2D hallFloorTex = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
items2 = this.ld.hallCeilingTexs;
Texture2D hallCeilingTex = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
WeightedSelection<Transform>[] items3 = this.ld.hallLights;
Transform hallLight = WeightedSelection<Transform>.ControlledRandomSelection(items3, this.controlledRNG);
Texture2D[] groupWallTexture = new Texture2D[this.ld.roomGroup.Count];
Texture2D[] groupFloorTexture = new Texture2D[this.ld.roomGroup.Count];
Texture2D[] groupCeilingTexture = new Texture2D[this.ld.roomGroup.Count];
Transform[] groupLight = new Transform[this.ld.roomGroup.Count];
for (int n = 0; n < this.ld.roomGroup.Count; n++)
{
    Texture2D[] array2 = groupWallTexture;
    int num4 = n;
    items2 = this.ld.roomGroup[n].wallTexture;
    array2[num4] = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
}

```

```

Texture2D[] array3 = groupFloorTexture;
int num5 = n;
items2 = this.ld.roomGroup[n].floorTexture;
array3[num5] = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
Texture2D[] array4 = groupCeilingTexture;
int num6 = n;
items2 = this.ld.roomGroup[n].ceilingTexture;
array4[num6] = WeightedSelection<Texture2D>.ControlledRandomSelection(items2, this.controlledRNG);
Transform[] array5 = groupLight;
int num7 = n;
items3 = this.ld.roomGroup[n].light;
array5[num7] = WeightedSelection<Transform>.ControlledRandomSelection(items3, this.controlledRNG);
}
int specialRoomsCount = this.controlledRNG.Next(this.ld.minSpecialRooms, this.ld.maxSpecialRooms + 1);
if (base.FrameShouldEnd())
{
yield return null;
}
RoomController bufferTiles = Object.Instantiate<RoomController>(this.roomControllerPre, this.ec.transform);
bufferTiles.transform.name = "Buffer Tiles";
bufferTiles.ec = this.ec;
if (this.ld.includeBuffers)
{
bufferTiles.type = RoomType.Room;
}
else
{
bufferTiles.type = RoomType.Null;
}
if (base.FrameShouldEnd())
{
yield return null;
}
int northEdgeBuffer = this.ld.outerEdgeBuffer;
int eastEdgeBuffer = this.ld.outerEdgeBuffer;
int southEdgeBuffer = this.ld.outerEdgeBuffer;
int westEdgeBuffer = this.ld.outerEdgeBuffer;
if (base.FrameShouldEnd())
{
yield return null;
}
for (i = 0; i < this.levelSize.x; i = j + 1)
{
for (int z = 0; z < this.levelSize.z; z = j + 1)
{
if (i < westEdgeBuffer || this.levelSize.x - i <= eastEdgeBuffer || z < southEdgeBuffer || this.levelSize.z - z <= northEdgeBuffer)
{
this.ec.CreateCell(this.placeholderTileVal, bufferTiles.transform, new IntVector2(i, z), bufferTiles);
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = z;
}
j = i;
}
foreach (StructureWithParameters structureWithParameters in structuresToGenerate)
{
initializedStructures.Add(Object.Instantiate<StructureBuilder>(structureWithParameters.prefab));
initializedStructures[initializedStructures.Count - 1].Initialize(this.ec, structureWithParameters.parameters);
if (base.FrameShouldEnd())
{
yield return null;
}
}
List<StructureWithParameters>.Enumerator enumerator3 = default(List<StructureWithParameters>.Enumerator);
this.halls.Add(null);
base.CreateArea(this.halls, RoomType.Hall, RoomCategory.Hall, 0, new IntVector2(0, 0), new IntVector2(this.levelSize.x, this.levelSize.z), false);
base.TextureArea(this.halls[0], hallWallTex, hallFloorTex, hallCeilingTex);
this.halls[0].lightPre = hallLight;
this.halls[0].acceptsExits = true;
this.halls[0].doorMats = this.ld.standardDoorMat;
this.ec.mainHall = this.halls[0];
foreach (StructureBuilder structureBuilder in initializedStructures)
{
structureBuilder.PreHallPrep(this, this.controlledRNG);
if (base.FrameShouldEnd())
{

```

```

yield return null;
}
}
List<StructureBuilder>.Enumerator enumerator4 = default(List<StructureBuilder>.Enumerator);
this.roomSpawnWeightDijkstraMap = new DijkstraMap(this.ec, PathType.Const, int.MaxValue, Array.Empty<Transform>());
this.specialRooms = new List<RoomController>();
for (i = 0; i < specialRoomsCount; i = j + 1)
{
    base.UpdatePotentialSpawnsForPlots(this.ld.specialRoomsStickToEdge);
    WeightedSelection<RoomAsset>[] items4 = this.ld.potentialSpecialRooms;
    RoomController item2;
    this.RandomlyPlaceRoom(WeightedSelection<RoomAsset>.ControlledRandomSelection(items4, this.controlledRNG), false, out item2);
    this.specialRooms.Add(item2);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
    j = i;
}
if (base.FrameShouldEnd())
{
    yield return null;
}
foreach (StructureBuilder structureBuilder2 in initializedStructures)
{
    structureBuilder2.PostSpecialRoomPrep(this, this.controlledRNG);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator4 = default(List<StructureBuilder>.Enumerator);
base.UpdatePotentialSpawnsForPlots(false);
if (base.FrameShouldEnd())
{
    yield return null;
}
i = 0;
while (i < plotCount && this.potentialRoomSpawns.Count > 0)
{
    this.plots.Add(null);
    base.CreateArea(this.plots, RoomType.Room, RoomCategory.Null, i,
        WeightedSelection<RoomSpawn>.ControlledRandomSelectionList(WeightedRoomSpawn.Convert(this.potentialRoomSpawns),
            this.controlledRNG).position, new IntVector2(this.levelSize.x, this.levelSize.z), true);
    base.UpdatePotentialSpawnsForPlots(false);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
    j = i;
    i = j + 1;
}
List<RoomController> plotsToExpand = new List<RoomController>(this.plots);
while (plotsToExpand.Count > 0)
{
    for (i = 0; i < plotsToExpand.Count; i = j + 1)
    {
        List<Direction> possibleDirections = base.GetPossibleDirections(plotsToExpand[i], 1);
        if (possibleDirections.Count > 0)
        {
            base.ExpandArea(plotsToExpand[i], possibleDirections[this.controlledRNG.Next(possibleDirections.Count)]);
        }
        else
        {
            plotsToExpand[i].expandable = false;
            plotsToExpand.RemoveAt(i);
            i--;
        }
    }
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
    j = i;
}
List<HallJunction> junctions = new List<HallJunction>();
bool[, ] foundJunctionPositions = new bool[this.levelSize.x, this.levelSize.z];
foreach (RoomController room2 in this.plots)
{

```

```

foreach (IntVector2 intVector in base.GetPlotCorners(room2))
{
    if (!foundJunctionPositions[intVector.x, intVector.z])
    {
        junctions.Add(new HallJunction(intVector));
        foundJunctionPositions[intVector.x, intVector.z] = true;
    }
}
yield return null;
if (base.FrameShouldEnd())
{
    yield return null;
}
for (i = 0; i < this.levelSize.x; i = i + 1)
{
    for (int z = 0; z < this.levelSize.z; z = j + 1)
    {
        IntVector2 intVector2 = new IntVector2(i, z);
        if (this.ec.cells[intVector2.x, intVector2.z].Null && (base.ProximityCheck(intVector2, RoomType.Room, 2) || this.ld.fillEmptySpace))
        {
            bool blockHallRemoval = this.ec.cells[intVector2.x, intVector2.z].blockHallRemoval;
            this.ec.CreateCell(this.placeholderTileVal, this.halls[0].transform, intVector2, this.halls[0]);
            this.ec.cells[intVector2.x, intVector2.z].blockHallRemoval = blockHallRemoval;
        }
        if (base.FrameShouldEnd())
        {
            yield return null;
        }
        j = z;
    }
    j = i;
}
base.DestroyRoom(bufferTiles);
if (base.FrameShouldEnd())
{
    yield return null;
}
while (this.plots.Count > 0)
{
    base.DestroyRoom(this.plots[0]);
    this.plots.RemoveAt(0);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<Hallway> connectedHallways = new List<Hallway>();
foreach (HallJunction junction in junctions)
{
    base.FindJunctionConnections(junction, new List<HallJunction>(junctions), connectedHallways, foundJunctionPositions);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<HallJunction>.Enumerator enumerator7 = default(List<HallJunction>.Enumerator);
List<Hallway> potentialHallwaysToRemove = new List<Hallway>(connectedHallways);
yield return null;
i = 0;
while (i < hallsToRemove && potentialHallwaysToRemove.Count > 0)
{
    int index2 = this.controlledRNG.Next(0, potentialHallwaysToRemove.Count);
    List<Hallway> list2 = new List<Hallway>(connectedHallways);
    list2.Remove(potentialHallwaysToRemove[index2]);
    if (base.RemoveHallwayIfPossible(potentialHallwaysToRemove[index2], list2))
    {
        connectedHallways.Remove(potentialHallwaysToRemove[index2]);
    }
    else
    {
        i--;
    }
    potentialHallwaysToRemove.RemoveAt(index2);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
    j = i;
}

```

```

i = j + 1;
}
this.previousRoomSpawnPositions.Clear();
if (this.Id.potentialPostPlotSpecialHalls.Length != 0)
{
i = 0;
while (i < this.Id.maxPostPlotSpecialHalls && (i < this.Id.minPostPlotSpecialHalls || this.controlledRNG.NextDouble() <
(double)this.Id.postPlotSpecialHallChance))
{
base.UpdatePotentialSpawnsForSpecialHalls();
WeightedSelection<RoomAsset>[] items4 = this.Id.potentialPostPlotSpecialHalls;
this.RandomlyPlaceRoom(WeightedSelection<RoomAsset>.ControlledRandomSelection(items4, this.controlledRNG), this.halls[0]);
if (base.FrameShouldEnd())
{
yield return null;
}
j = i;
i = j + 1;
}
}
yield return null;
if (base.FrameShouldEnd())
{
yield return null;
}
yield return null;
RoomController cornerBuffers = null;
List<Cell> potentialStartingPoints = new List<Cell>();
for (i = 0; i < hallsToAdd; i = j + 1)
{
if (cornerBuffers != null)
{
base.DestroyRoom(cornerBuffers);
}
cornerBuffers = Object.Instantiate<RoomController>(this.roomControllerPre, this.ec.transform);
cornerBuffers.transform.name = "CornerBuffers";
cornerBuffers.category = RoomCategory.Buffer;
cornerBuffers.ec = this.ec;
int z = 4;
for (int x = 0; x < this.levelSize.x; x = j + 1)
{
for (int z2 = 0; z2 < this.levelSize.z; z2 = j + 1)
{
if (this.ec.cells[x, z2].Null)
{
IntVector2 intVector3 = new IntVector2(x, z2);
if ((this.ec.BufferTileInDirectionCheck(intVector3, Direction.North, z) || this.ec.BufferTileInDirectionCheck(intVector3, Direction.South, z)) &&
(this.ec.BufferTileInDirectionCheck(intVector3, Direction.East, z) || this.ec.BufferTileInDirectionCheck(intVector3, Direction.West, z)))
{
this.ec.CreateCell(15, cornerBuffers.transform, intVector3, cornerBuffers);
}
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = z2;
}
j = x;
}
potentialStartingPoints = new List<Cell>(this.halls[0].GetNewTileList());
for (int x = 0; x < potentialStartingPoints.Count; x = j + 1)
{
if (this.ec.GetCellNeighbors(potentialStartingPoints[x].position).Count >= 4)
{
potentialStartingPoints.RemoveAt(x);
x--;
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = x;
}
int val = this.controlledRNG.Next(0, potentialStartingPoints.Count);
while (potentialStartingPoints.Count > 0 && base.PotentialPathDirections(potentialStartingPoints[val].position).Count == 0)
{
potentialStartingPoints.RemoveAt(val);
val = this.controlledRNG.Next(0, potentialStartingPoints.Count);
}

```

```

if (base.FrameShouldEnd())
{
yield return null;
}
}
if (potentialStartingPoints.Count > 0)
{
bool success = false;
int x = 0;
Queue<IntVector2> curPath = new Queue<IntVector2>();
Cell tileToConnect = null;
Direction connectDir = Direction.Null;
while (!success && x < this.Id.maxHallAttempts)
{
curPath = base.GetRandomPath(potentialStartingPoints[val].position, this.Id.additionTurnChance, out success, out tileToConnect, out connectDir);
x++;
if (base.FrameShouldEnd())
{
yield return null;
}
}
if (success)
{
foreach (IntVector2 position in curPath)
{
this.ec.CreateCell(this.placeholderTileVal, this.halls[0].transform, position, this.halls[0]);
if (base.FrameShouldEnd())
{
yield return null;
}
}
Queue<IntVector2>.Enumerator enumerator8 = default(Queue<IntVector2>.Enumerator);
if (tileToConnect != null && tileToConnect.room != this.halls[0] && tileToConnect.room.ContainsDoorPosition(tileToConnect.position))
{
RoomController roomController;
base.BuildDoorInDirection(tileToConnect.room, tileToConnect.position, connectDir.GetOpposite(), false, out roomController);
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
curPath = null;
tileToConnect = null;
}
j = i;
}
if (cornerBuffers != null)
{
base.DestroyRoom(cornerBuffers);
}
if (base.FrameShouldEnd())
{
yield return null;
}
foreach (StructureBuilder structureBuilder3 in initializedStructures)
{
structureBuilder3.PostHallPrep(this, this.controlledRNG);
}
yield return null;
List<Direction> potentialExitDirections = Directions.All();
int exitCount = this.Id.exitCount;
for (i = 0; i < exitCount; i = j + 1)
{
int index3 = this.controlledRNG.Next(0, potentialExitDirections.Count);
Direction connectDir = potentialExitDirections[index3];
potentialExitDirections.RemoveAt(index3);
bool success = false;
List<IntVector2> potentialSpawns = new List<IntVector2>();
IntVector2 intVector4 = connectDir.ToIntVector2();
int num8 = Mathf.Max(0, (this.levelSize.x - 1) * intVector4.x);
int val = Mathf.Max(0, (this.levelSize.x - 1) * (intVector4.x + Mathf.Abs(intVector4.z)));
int z = Mathf.Max(0, (this.levelSize.z - 1) * intVector4.z);
int x = Mathf.Max(0, (this.levelSize.z - 1) * (intVector4.z + Mathf.Abs(intVector4.x)));
for (int z2 = num8; z2 <= val; z2 = j + 1)
{
for (int z3 = z; z3 <= x; z3 = j + 1)
{
IntVector2 travDirInt = connectDir.GetOpposite().ToIntVector2();

```

```

IntVector2 offset = default(IntVector2);
while (this.ec.ContainsCoordinates(new IntVector2(z2 + offset.x, z3 + offset.z)) && this.ec.cells[z2 + offset.x, z3 + offset.z].Null)
{
    offset += travDirInt;
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
if (this.ec.ContainsCoordinates(new IntVector2(z2 + offset.x, z3 + offset.z)))
{
    IntVector2 intVector5 = new IntVector2(z2 + offset.x, z3 + offset.z);
    if (!this.ec.cells[intVector5.x, intVector5.z].Null)
    {
        Cell cell = this.ec.cells[intVector5.x, intVector5.z];
        if (cell.room.acceptsExits)
        {
            List<Direction> list3 = connectDir.PerpendicularList();
            bool flag = true;
            foreach (Direction direction in list3)
            {
                if (!this.ec.ContainsCoordinates(intVector5 + direction.ToIntVector2()))
                {
                    flag = false;
                }
                else if (this.ec.cells[intVector5.x + direction.ToIntVector2().x, intVector5.z + direction.ToIntVector2().z].Null)
                {
                    flag = false;
                }
                else if (this.ec.cells[intVector5.x + direction.ToIntVector2().x, intVector5.z + direction.ToIntVector2().z].room != cell.room)
                {
                    flag = false;
                }
                else if (this.ec.cells[intVector5.x + direction.ToIntVector2().x, intVector5.z + direction.ToIntVector2().z].locked)
                {
                    flag = false;
                }
                else if (this.ec.cells[intVector5.x + direction.ToIntVector2().x, intVector5.z + direction.ToIntVector2().z].HasAnyHardCoverage)
                {
                    flag = false;
                }
            }
            if (flag)
            {
                if (base.RoomFits(this.Id.elevatorRoom, null, intVector5 + connectDir.ToIntVector2(), this.Id.elevatorRoom.potentialDoorPositions[0], connectDir.GetOpposite()))
                {
                    potentialSpawns.Add(intVector5 + connectDir.ToIntVector2());
                }
                if (cell.room.type != RoomType.Hall)
                {
                    success = true;
                }
            }
        }
        if (base.FrameShouldEnd())
        {
            yield return null;
        }
        j = z3;
    }
    j = z2;
}
if (potentialSpawns.Count > 0)
{
    if (success)
    {
        for (int num9 = 0; num9 < potentialSpawns.Count; num9++)
        {
            if (this.ec.CellFromPosition(potentialSpawns[num9] + connectDir.GetOpposite().ToIntVector2()).room.type == RoomType.Hall)
            {
                potentialSpawns.RemoveAt(num9);
                num9--;
            }
        }
    }
    int index4 = this.controlledRNG.Next(0, potentialSpawns.Count);

```



```

base.CreateElevator(potentialSpawns[index4], connectDir.GetOpposite(), this.Id.elevatorPre, this.Id.elevatorRoom, this.ec.Elevators.Count == 0);
if (this.ec.CellFromPosition(potentialSpawns[index4]).room.type != RoomType.Hall)
{
    this.ec.CellFromPosition(potentialSpawns[index4]).room.acceptsExits = false;
}
}
else
{
    Debug.LogWarning("No position for the exit on the " + connectDir.ToString() + " side was found!\nReducing total number of exits.");
    i--;
    exitCount = Mathf.Min(exitCount, potentailExitDirections.Count);
}
if (base.FrameShouldEnd())
{
    yield return null;
}
potentialSpawns = null;
j = i;
}
if (base.FrameShouldEnd())
{
    yield return null;
}
for (i = 0; i < this.levelSize.x; i = i + 1)
{
    for (int x = 0; x < this.levelSize.z; x = j + 1)
    {
        IntVector2 intVector6 = new IntVector2(i, x);
        if (!this.ec.cells[intVector6.x, intVector6.z].Null)
        {
            this.ec.UpdateCell(intVector6);
        }
    }
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
    j = x;
}
j = i;
}
foreach (DoorPlacement doorPlacement in this.doorPlacements)
{
    this.ec.ConnectCells(doorPlacement.position, doorPlacement.dir);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<DoorPlacement>.Enumerator enumerator11 = default(List<DoorPlacement>.Enumerator);
this.roomSpawnWeightDijkstraMap = new DijkstraMap(this.ec, PathType.Const, int.MaxValue, Array.Empty<Transform>());
int[] roomCount = new int[this.Id.roomGroup.Count];
for (int num10 = 0; num10 < roomCount.Length; num10++)
{
    roomCount[num10] = this.controlledRNG.Next(this.Id.roomGroup[num10].minRooms, this.Id.roomGroup[num10].maxRooms + 1);
}
if (base.FrameShouldEnd())
{
    yield return null;
}
base.UpdatePotentialSpawnsForRooms(true);
if (base.FrameShouldEnd())
{
    yield return null;
}
this.ec.npcSpawnTile = new Cell[this.ec.npcsToSpawn.Count];
for (i = 0; i < this.ec.npcsToSpawn.Count; i = j + 1)
{
    if (this.ec.npcsToSpawn[i].potentialRoomAssets.Length != 0)
    {
        RoomController roomController2 = null;
        WeightedSelection<RoomAsset>[] items4 = this.ec.npcsToSpawn[i].potentialRoomAssets;
        if (this.RandomlyPlaceRoom(WeightedSelection<RoomAsset>.ControlledRandomSelection(items4, this.controlledRNG), true, out roomController2))
        {
            this.standardRooms.Add(roomController2);
            this.ec.npcSpawnTile[i] = roomController2.RandomEntitySafeCellNoGarbage();
        }
    }
}
if (base.FrameShouldEnd())
{
    yield return null;
}
}

```

```

yield return null;
}
base.UpdatePotentialSpawnsForRooms(true);
if (base.FrameShouldEnd())
{
yield return null;
}
j = i;
}
foreach (RandomEvent randomEvent3 in events)
{
if (randomEvent3.PotentialRoomAssets.Length != 0)
{
RoomController roomController3 = null;
WeightedSelection<RoomAsset>[] items4 = randomEvent3.PotentialRoomAssets;
if (this.RandomlyPlaceRoom(WeightedSelection<RoomAsset>.ControlledRandomSelection(items4, this.controlledRNG), true, out roomController3))
{
this.standardRooms.Add(roomController3);
randomEvent3.AssignRoom(roomController3);
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
base.UpdatePotentialSpawnsForRooms(true);
if (base.FrameShouldEnd())
{
yield return null;
}
}
enumerator = default(List<RandomEvent>.Enumerator);
for (i = 0; i < this.Id.roomGroup.Count; i = j + 1)
{
RoomGroup group = this.Id.roomGroup[i];
List<WeightedRoomAsset> potentialRoomsList = new List<WeightedRoomAsset>(group.potentialRooms);
this.previousRoomSpawnPositions.Clear();
if (group.overridePositionWeighting)
{
this.usingPositionWeightingOverrides = true;
this.overrideCenterWeightMultiplier = group.centerWeightMultiplier;
this.overrideDijkstraWeightValueMultiplier = group.dijkstraWeightValueMultiplier;
this.overrideDijkstraWeightPower = group.dijkstraWeightPower;
}
else
{
this.usingPositionWeightingOverrides = false;
}
bool success = this.controlledRNG.NextDouble() < (double)group.stickToHallChance;
base.UpdatePotentialSpawnsForRooms(success);
if (base.FrameShouldEnd())
{
yield return null;
}
int x = 0;
while (x < roomCount[i] && potentialRoomsList.Count > 0)
{
WeightedSelection<RoomAsset>[] items4 = potentialRoomsList.ToArray();
int index5 = WeightedSelection<RoomAsset>.ControlledRandomIndex(items4, this.controlledRNG);
RoomController roomController4;
bool flag2 = this.RandomlyPlaceRoom(potentialRoomsList[index5].selection, true, out roomController4);
if (!flag2)
{
if (!success)
{
success = true;
base.UpdatePotentialSpawnsForRooms(success);
flag2 = this.RandomlyPlaceRoom(potentialRoomsList[index5].selection, true, out roomController4);
}
if (!flag2)
{
potentialRoomsList.RemoveAt(index5);
x--;
}
}
if (flag2)
{
this.standardRooms.Add(roomController4);
success = (this.controlledRNG.NextDouble() < (double)group.stickToHallChance);
}
}
}

```

```

base.UpdatePotentialSpawnsForRooms(success);
if (!potentialRoomsList[index5].selection.keepTextures)
{
    roomController4.florTex = groupFloorTexture[i];
    roomController4.wallTex = groupWallTexture[i];
    roomController4.ceilTex = groupCeilingTexture[i];
    if (groupLight[i] != null)
    {
        roomController4.lightPre = groupLight[i];
    }
}
}
if (base.FrameShouldEnd())
{
    yield return null;
}
j = x;
x = j + 1;
}
group = null;
potentialRoomsList = null;
j = i;
}
this.usingPositionWeightingOverrides = false;
foreach (RoomController roomController5 in this.standardRooms)
{
    roomController5.functions.AfterAllRoomsPlaced(this, this.controlledRNG);
}
using (List<RoomController>.Enumerator enumerator12 = this.standardRooms.GetEnumerator())
{
    IL_31C0:
    while (enumerator12.MoveNext())
    {
        RoomController room = enumerator12.Current;
        for (i = 0; i < room.forcedDoorPositions.Count; i = j + 1)
        {
            RoomController roomController6;
            IntVector2 position2;
            if (base.BuildDoorIfPossible(room.forcedDoorPositions[i], room, false, true, out roomController6, out position2))
            {
                room.forcedDoorPositions.RemoveAt(i);
                room.ConnectRooms(roomController6);
                roomController6.RemoveFromDoorPositions(position2);
                i--;
            }
            if (base.FrameShouldEnd())
            {
                yield return null;
            }
            j = i;
        }
        while (room.potentialDoorPositions.Count > 0)
        {
            bool success;
            if (this.controlledRNG.NextDouble() >= (double)this.ld.extraDoorChance)
            {
                IL_31A3:
                while (room.potentialDoorPositions.Count > 0 && this.controlledRNG.NextDouble() < (double)room.windowChance)
                {
                    success = false;
                    while (room.potentialDoorPositions.Count > 0 && !success)
                    {
                        int index6 = this.controlledRNG.Next(0, room.potentialDoorPositions.Count);
                        RoomController roomController7;
                        IntVector2 position3;
                        if (base.BuildWindowIfPossible(room.potentialDoorPositions[index6], room, out roomController7, out position3))
                        {
                            success = true;
                            roomController7.RemoveFromDoorPositions(position3);
                        }
                        room.potentialDoorPositions.RemoveAt(index6);
                    }
                    if (base.FrameShouldEnd())
                    {
                        yield return null;
                    }
                }
            }
            room = null;
            goto IL_31C0;

```

```

    }
    success = false;
    while (room.potentialDoorPositions.Count > 0 && !success)
    {
        int index7 = this.controlledRNG.Next(0, room.potentialDoorPositions.Count);
        RoomController roomController8;
        IntVector2 position4;
        if (base.BuildDoorIfPossible(room.potentialDoorPositions[index7], room, false, false, out roomController8, out position4))
        {
            success = true;
            room.ConnectRooms(roomController8);
            roomController8.RemoveFromDoorPositions(position4);
        }
        room.potentialDoorPositions.RemoveAt(index7);
        if (base.FrameShouldEnd())
        {
            yield return null;
        }
    }
    goto IL_31A3;
}

List<RoomController>.Enumerator enumerator12 = default(List<RoomController>.Enumerator);
this.ec.SetTileInstantiation(true);
this.generateAtlases = true;
if (base.FrameShouldEnd())
{
    yield return null;
}
this.ec.mainHall.GenerateTextureAtlas();
if (base.FrameShouldEnd())
{
    yield return null;
}
foreach (RoomController roomController9 in this.ec.rooms)
{
    roomController9.GenerateTextureAtlas();
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator12 = default(List<RoomController>.Enumerator);
for (i = 0; i < this.levelSize.x; i = j + 1)
{
    for (int x = 0; x < this.levelSize.z; x = j + 1)
    {
        IntVector2 intVector7 = new IntVector2(i, x);
        if (!this.ec.cells[intVector7.x, intVector7.z].Null)
        {
            this.ec.RefreshCell(intVector7);
        }
        if (base.FrameShouldEnd())
        {
            yield return null;
        }
        j = x;
    }
    j = i;
}
foreach (PosterData posterData in this.postersToBuild)
{
    this.ec.BuildPoster(posterData.poster, this.ec.CellFromPosition(posterData.position), posterData.direction);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<PosterData>.Enumerator enumerator13 = default(List<PosterData>.Enumerator);
foreach (RoomController roomController10 in this.ec.rooms)
{
    roomController10.functions.Build(this, this.controlledRNG);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator12 = default(List<RoomController>.Enumerator);

```

```

foreach (DoorPlacement placement in this.doorPlacements)
{
    base.BuildDoorPlacement(placement);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator11 = default(List<DoorPlacement>.Enumerator);
this.doorPlacements.Clear();
if (base.FrameShouldEnd())
{
    yield return null;
}
int structureIndex = 0;
foreach (StructureBuilder structureBuilder4 in initializedStructures)
{
    structureBuilder4.Generate(this, this.controlledRNG);
    base.FrameShouldEnd();
    structureIndex++;
    yield return null;
}
enumerator4 = default(List<StructureBuilder>.Enumerator);
this.ec.lightMode = this.ld.lightMode;
Cell[] openCheck = new Cell[4];
List<Direction>[] openCheckDirs = new List<Direction>[]
{
    new List<Direction>(),
    new List<Direction>(),
    new List<Direction>(),
    new List<Direction>()
};
openCheckDirs[0].Add(Direction.North);
openCheckDirs[0].Add(Direction.East);
openCheckDirs[1].Add(Direction.North);
openCheckDirs[1].Add(Direction.West);
openCheckDirs[2].Add(Direction.South);
openCheckDirs[2].Add(Direction.East);
openCheckDirs[3].Add(Direction.South);
openCheckDirs[3].Add(Direction.West);
List<Direction> currentOpenCheckDirs = new List<Direction>();
for (i = 0; i < this.levelSize.x - 1; i = j + 1)
{
    for (int x = 0; x < this.levelSize.z - 1; x = j + 1)
    {
        bool flag3 = true;
        bool @null = this.ec.cells[i, x].Null;
        openCheck[0] = this.ec.cells[i, x];
        openCheck[1] = this.ec.cells[i + 1, x];
        openCheck[2] = this.ec.cells[i, x + 1];
        openCheck[3] = this.ec.cells[i + 1, x + 1];
        for (int num11 = 0; num11 < 4; num11++)
        {
            if (openCheck[num11].excludeFromOpen || openCheck[num11].Null != @null)
            {
                flag3 = false;
                break;
            }
        }
        Directions.FillOpenDirectionsFromBin(currentOpenCheckDirs, openCheck[num11].NavBin);
        if (!currentOpenCheckDirs.Contains(openCheckDirs[num11][0]) || !currentOpenCheckDirs.Contains(openCheckDirs[num11][1]))
        {
            flag3 = false;
            break;
        }
    }
    if (flag3)
    {
        foreach (Cell cell2 in openCheck)
        {
            cell2.open = true;
            for (int num12 = 0; num12 < openCheck.Length; num12++)
            {
                if (!cell2.openTiles.Contains(openCheck[num12]) && cell2 != openCheck[num12])
                {
                    cell2.openTiles.Add(openCheck[num12]);
                }
            }
        }
    }
}

```

```

Object.Instantiate<Transform>(this.navQuadPre, this.ec.navMeshTransform).transform.localPosition = new Vector3((float)i * 10f + 10f, 0f, (float)x * 10f + 10f);
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = x;
}
j = i;
}
List<OpenTileGroup> openTileGroups = new List<OpenTileGroup>();
Queue<Cell> openTilesToCheck = new Queue<Cell>();
for (i = 0; i < this.levelSize.x; i = j + 1)
{
for (int x = 0; x < this.levelSize.z; x = j + 1)
{
Cell cell3 = this.ec.cells[i, x];
if (cell3.open)
{
bool flag4 = false;
foreach (OpenTileGroup openTileGroup in openTileGroups)
{
if (cell3.openTileGroup == openTileGroup)
{
flag4 = true;
}
}
if (!flag4)
{
OpenTileGroup currentGroup = new OpenTileGroup();
openTileGroups.Add(currentGroup);
openTilesToCheck.Clear();
openTilesToCheck.Enqueue(cell3);
while (openTilesToCheck.Count > 0)
{
cell3 = openTilesToCheck.Dequeue();
cell3.openTileGroup = currentGroup;
currentGroup.cells.Add(cell3);
foreach (Cell cell4 in cell3.openTiles)
{
if (cell4.openTileGroup != currentGroup && !openTilesToCheck.Contains(cell4))
{
openTilesToCheck.Enqueue(cell4);
}
}
if (cell3.room.type == RoomType.Room && !cell3.excludeFromRoomGroup)
{
foreach (Cell cell5 in cell3.room.cells)
{
if (cell5.openTileGroup != currentGroup && !cell5.excludeFromRoomGroup && !openTilesToCheck.Contains(cell5))
{
openTilesToCheck.Enqueue(cell5);
}
}
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
foreach (Cell tileToConnect in currentGroup.cells)
{
foreach (Direction direction2 in Directions.OpenDirectionsFromBin(tileToConnect.ConstBin))
{
if (this.ec.ContainsCoordinates(tileToConnect.position + direction2.ToIntVector2()) && !this.ec.CellFromPosition(tileToConnect.position + direction2.ToIntVector2()).Null && !currentGroup.cells.Contains(this.ec.CellFromPosition(tileToConnect.position + direction2.ToIntVector2())))
{
currentGroup.exits.Add(new OpenGroupExit(currentGroup, tileToConnect, direction2));
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
}

```

```

List<Direction>.Enumerator enumerator17 = default(List<Direction>.Enumerator);
tileToConnect = null;
}
List<Cell>.Enumerator enumerator16 = default(List<Cell>.Enumerator);
if (base.FrameShouldEnd())
{
yield return null;
}
currentGroup = null;
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = x;
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = i;
}
base.BlockTilesToBlock();
if (base.FrameShouldEnd())
{
yield return null;
}
yield return null;
foreach (RandomEvent randomEvent4 in events)
{
randomEvent4.AfterUpdateSetup(this.controlledRNG);
base.FrameShouldEnd();
yield return null;
}
enumerator = default(List<RandomEvent>.Enumerator);
foreach (StructureBuilder structureBuilder5 in initializedStructures)
{
structureBuilder5.PostOpenCalcGenerate(this, this.controlledRNG);
base.FrameShouldEnd();
yield return null;
}
enumerator4 = default(List<StructureBuilder>.Enumerator);
if (timeOutEvent != null)
{
timeOutEvent.AfterUpdateSetup(this.controlledRNG);
}
yield return null;
for (i = 0; i < 0; i = j + 1)
{
for (int x = 0; x < this.levelSize.z; x = j + 1)
{
if (!this.ec.cells[i, x].Null)
{
Cell cell6 = this.ec.cells[i, x];
if (cell6.room.type == RoomType.Hall)
{
RandomSelection<HallBuilder>[] standardHallBuilders = this.ld.standardHallBuilders;
List<HallBuilder> list4 = RandomSelection<HallBuilder>.ControlledRandomSelection(standardHallBuilders, this.controlledRNG);
if (list4.Count > 0)
{
HallBuilder hallBuilder = Object.Instantiate<HallBuilder>(list4[this.controlledRNG.Next(0, list4.Count)], cell6.room.transform);
hallBuilder.Build(cell6, this.ec, new Random(this.controlledRNG.Next()));
Object.Destroy(hallBuilder.gameObject);
yield return null;
}
}
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = x;
}
j = i;
}
CoreGameManager.lightMapPaused = true;
if (base.FrameShouldEnd())
{

```

```

yield return null;
}
foreach (RoomController room in this.ec.rooms)
{
RoomController room;
if (room.potentialPosters.Count > 0)
{
while (room.HasFreeWall && this.controlledRNG.NextDouble() < (double)room.posterChance)
{
this.ec.BuildPosterInRoom(room,
WeightedSelection<PosterObject>.ControlledRandomSelectionList(WeightedPosterObject.Convert(room.potentialPosters), this.controlledRNG),
this.controlledRNG);
if (base.FrameShouldEnd())
{
yield return null;
}
}
}
room = null;
}
enumerator12 = default(List<RoomController>.Enumerator);
List<Cell> hallLights = new List<Cell>();
for (i = 0; i < this.levelSize.x; i = j + 1)
{
for (int x = 0; x < this.levelSize.z; x = j + 1)
{
IntVector2 intVector8 = new IntVector2(i, x);
if (!this.ec.cells[intVector8.x, intVector8.z].Null)
{
Cell tileToConnect = this.ec.cells[intVector8.x, intVector8.z];
if (tileToConnect.HasSoftFreeWall && tileToConnect.room.acceptsPosters && (double)this.ld.posterChance > this.controlledRNG.NextDouble() * 100.0)
{
Direction dir = tileToConnect.RandomUncoveredDirection(this.controlledRNG);
WeightedSelection<PosterObject>[] posters = this.ld.posters;
PosterObject poster = WeightedSelection<PosterObject>.ControlledRandomSelection(posters, this.controlledRNG);
this.ec.BuildPoster(poster, tileToConnect, dir, this.controlledRNG);
}
if (tileToConnect.room.type == RoomType.Hall && tileToConnect.HardCoverageFits(CellCoverage.Up))
{
bool success = false;
foreach (Cell cell7 in hallLights)
{
if (!cell7.permanentLight && Mathf.Abs(tileToConnect.position.x - cell7.position.x) + Mathf.Abs(tileToConnect.position.z - cell7.position.z) <
this.ld.maxLightDistance && this.ec.NavigableDistance(cell7, tileToConnect, PathType.Const) < this.ld.maxLightDistance)
{
success = true;
break;
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
List<Cell>.Enumerator enumerator16 = default(List<Cell>.Enumerator);
if (!success)
{
tileToConnect.room.ec.GenerateLight(tileToConnect, this.ld.standardLightColor, this.ld.standardLightStrength);
hallLights.Add(tileToConnect);
if (tileToConnect.room.lightPre != null)
{
Transform controllerToAssign = Object.Instantiate<Transform>(tileToConnect.room.lightPre, tileToConnect.TileTransform);
tileToConnect.AssignLightController(controllerToAssign);
}
else
{
Transform controllerToAssign2 = Object.Instantiate<Transform>(hallLight, tileToConnect.TileTransform);
tileToConnect.AssignLightController(controllerToAssign2);
}
}
tileToConnect = null;
}
if (base.FrameShouldEnd())
{
yield return null;
}
j = x;
}
j = i;

```



```

}
this.ec.hallLights = hallLights;
foreach (RoomController room in this.ec.rooms)
{
RoomController room;
foreach (IntVector2 position5 in room.standardLightCells)
{
this.ec.GenerateLight(this.ec.CellFromPosition(position5), this.ld.standardLightColor, this.ld.standardLightStrength);
if (room.lightPre != null)
{
Transform controllerToAssign3 = Object.Instantiate<Transform>(room.lightPre, this.ec.CellFromPosition(position5).TileTransform);
this.ec.CellFromPosition(position5).AssignLightController(controllerToAssign3);
}
else
{
Transform controllerToAssign4 = Object.Instantiate<Transform>(hallLight, this.ec.CellFromPosition(position5).TileTransform);
this.ec.CellFromPosition(position5).AssignLightController(controllerToAssign4);
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
List<IntVector2>.Enumerator enumerator18 = default(List<IntVector2>.Enumerator);
room = null;
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (LightSourceData lightSourceData in this.lightsToGenerate)
{
this.ec.GenerateLight(this.ec.CellFromPosition(lightSourceData.position), lightSourceData.color, lightSourceData.strength);
if (lightSourceData.prefab != null)
{
Transform controllerToAssign5 = Object.Instantiate<Transform>(lightSourceData.prefab, this.ec.CellFromPosition(lightSourceData.position).TileTransform);
this.ec.CellFromPosition(lightSourceData.position).AssignLightController(controllerToAssign5);
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
List<LightSourceData>.Enumerator enumerator19 = default(List<LightSourceData>.Enumerator);
CoreGameManager.lightMapPaused = false;
if (base.FrameShouldEnd())
{
yield return null;
}
this.ec.BuildNavMesh();
if (base.FrameShouldEnd())
{
yield return null;
}
foreach (RoomController roomController11 in this.ec.rooms)
{
if (roomController11.type != RoomType.Hall && roomController11.spawnItems && roomController11.category != RoomCategory.Class)
{
int num13 = roomController11.maxItemValue;
if (roomController11.doors.Count <= 1)
{
num13 += this.ld.singleEntranceItemVal;
}
if (roomController11.roomsFromHall > 0)
{
num13 += 10 * (int)Mathf.Pow(2f, (float)(roomController11.roomsFromHall + 1));
}
roomController11.maxItemValue = num13;
}
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (RoomController roomController12 in this.ec.rooms)
{
roomController12.functions.AfterRoomValuesCalculated(this, this.controlledRNG);
if (base.FrameShouldEnd())
{
yield return null;
}
}
}

```

```

}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (Activity activity in this.ec.activities)
{
    activity.Initialize();
}
foreach (RoomController roomController13 in this.ec.rooms)
{
    foreach (ItemObject item3 in roomController13.itemList)
    {
        base.PlaceItemInRoom(item3, roomController13, new Random());
    }
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator12 = default(List<RoomController>.Enumerator);
lootTable.SortByValue();
string str = "High End Loot:\n";
List<ItemObject> highEndLoot = new List<ItemObject>();
List<ItemObject> midEndLoot = new List<ItemObject>();
List<ItemObject> lowEndLoot = new List<ItemObject>();
int num14 = 0;
while (lootTable.Count > 0 && num14 < this.Id.highEndCutoff)
{
    str = str + lootTable[0].nameKey + "\n";
    highEndLoot.Add(lootTable[0]);
    num14 += lootTable[0].value;
    lootTable.RemoveAt(0);
}
List<ItemObject> emergencyHighEndLoot = new List<ItemObject>(highEndLoot);
str += "Mid End Loot:\n";
while (lootTable.Count > 0 && lootTable.Value() > this.Id.lowEndCutoff)
{
    str = str + lootTable[0].nameKey + "\n";
    midEndLoot.Add(lootTable[0]);
    lootTable.RemoveAt(0);
}
lowEndLoot.AddRange(lootTable);
str += "Low End Loot:\n";
foreach (ItemObject itemObject in lootTable)
{
    str = str + itemObject.nameKey + "\n";
}
if (base.FrameShouldEnd())
{
    yield return null;
}
foreach (ItemObject item4 in highEndLoot)
{
    base.PlaceItemInRandomRoom(item4, true, false, itemRng);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<ItemObject>.Enumerator enumerator22 = default(List<ItemObject>.Enumerator);
foreach (ItemObject item5 in midEndLoot)
{
    base.PlaceItemInRandomRoom(item5, false, false, itemRng);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator22 = default(List<ItemObject>.Enumerator);
foreach (ItemObject item6 in lowEndLoot)
{
    base.PlaceItemInRandomRoom(item6, false, true, itemRng);
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator22 = default(List<ItemObject>.Enumerator);
foreach (RoomController roomController14 in this.ec.rooms)
{

```

```

while (roomController14.maxItemValue - roomController14.currentItemValue > this.Id.maxAllowedUnusedValue &&
roomController14.itemSpawnPoints.Count > 0)
{
ItemObject item7 = emergencyHighEndLoot[itemRng.Next(0, emergencyHighEndLoot.Count)];
base.PlaceItemInRoom(item7, roomController14, itemRng);
}
if (base.FrameShouldEnd())
{
yield return null;
}
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (RoomController room3 in this.ec.rooms)
{
base.FillRoomWithPooledPickups(room3, false);
if (base.FrameShouldEnd())
{
yield return null;
}
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (RoomController room4 in this.ec.rooms)
{
base.UpdateRoomObjectPosition(room4);
if (base.FrameShouldEnd())
{
yield return null;
}
}
enumerator12 = default(List<RoomController>.Enumerator);
if (base.FrameShouldEnd())
{
yield return null;
}
this.ec.map.Updatelcons();
this.ec.SetSpawn(this.ec.Elevators[0].transform.position + -this.ec.Elevators[0].transform.forward * 10f, this.ec.Elevators[0].transform.rotation);
this.ec.standardDoors = this.standardDoors;
if (base.FrameShouldEnd())
{
yield return null;
}
float currentTime = this.Id.initialEventGap;
List<RandomEvent> eventsToQueue = new List<RandomEvent>();
eventsToQueue.AddRange(events);
if (!Singleton<CoreGameManager>.Instance.timeLimitChallenge)
{
while (eventsToQueue.Count > 0)
{
RandomEvent randomEvent5 = eventsToQueue[Random.Range(0, eventsToQueue.Count)];
currentTime += Random.value * (this.Id.maxEventGap - this.Id.minEventGap) + this.Id.minEventGap;
this.ec.AddEvent(randomEvent5, currentTime);
currentTime += randomEvent5.EventTime;
eventsToQueue.Remove(randomEvent5);
if (base.FrameShouldEnd())
{
yield return null;
}
}
}
if (timeOutEvent != null && Singleton<CoreGameManager>.Instance.currentMode == Mode.Main)
{
if (Singleton<CoreGameManager>.Instance.timeLimitChallenge)
{
this.ec.AddEvent(timeOutEvent, 60f);
}
else
{
this.ec.AddEvent(timeOutEvent, this.Id.timeLimit);
}
}
List<Cell> allTiles = new List<Cell>(this.ec.AllTilesNoGarbage(false, false));
i = 0;
while (i < this.ec.npcsToSpawn.Count && allTiles.Count > 0)
{
if (this.ec.npcSpawnTile[i] == null)
{
List<Cell> potentialNPCSpawns = new List<Cell>();
foreach (RoomController roomController15 in this.ec.rooms)
{

```

```

if (this.ec.npcsToSpawn[i].spawnableRooms.Contains(roomController15.category))
{
    potentialNPCSpawns.AddRange(roomController15.AllEventSafeCellsNoGarbage());
}
if (base.FrameShouldEnd())
{
    yield return null;
}
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (RoomController roomController16 in this.halls)
{
    if (this.ec.npcsToSpawn[i].spawnableRooms.Contains(roomController16.category))
    {
        potentialNPCSpawns.AddRange(roomController16.AllTilesNoGarbage(false, true));
    }
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator12 = default(List<RoomController>.Enumerator);
if (potentialNPCSpawns.Count > 0)
{
    Cell cell8 = potentialNPCSpawns[this.controlledRNG.Next(0, potentialNPCSpawns.Count)];
    this.ec.npcSpawnTile[i] = cell8;
}
else
{
    Cell cell8 = allTiles[this.controlledRNG.Next(0, allTiles.Count)];
    this.ec.npcSpawnTile[i] = cell8;
}
if (base.FrameShouldEnd())
{
    yield return null;
}
potentialNPCSpawns = null;
}
j = i;
i = j + 1;
}
foreach (RoomController roomController17 in this.ec.rooms)
{
    roomController17.functions.OnGenerationFinished();
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
enumerator12 = default(List<RoomController>.Enumerator);
foreach (StructureBuilder structureBuilder6 in initializedStructures)
{
    structureBuilder6.OnGenerationFinished(this);
    base.FrameShouldEnd();
    yield return null;
}
enumerator4 = default(List<StructureBuilder>.Enumerator);
foreach (EnvironmentObject environmentObject in this.environmentObjects)
{
    environmentObject.LoadingFinished();
    if (base.FrameShouldEnd())
    {
        yield return null;
    }
}
List<EnvironmentObject>.Enumerator enumerator23 = default(List<EnvironmentObject>.Enumerator);
if (base.FrameShouldEnd())
{
    yield return null;
}
this.ec.CullingManager.PrepareOcclusionCalculations();
if (base.FrameShouldEnd())
{
    yield return null;
}
for (i = 0; i < this.ec.CullingManager.TotalChunks; i = j + 1)
{
    this.ec.CullingManager.CalculateOcclusionCullingForChunk(i);
    if (base.FrameShouldEnd())

```

```

{
yield return null;
}
j = i;
}
this.ec.CullingManager.SetActive(true);
if (base.FrameShouldEnd())
{
yield return null;
}
yield return null;
yield return null;
this.levelInProgress = false;
this.levelCreated = true;
if (Singleton<CoreGameManager>.Instance.GetCamera(0) != null)
{
Singleton<CoreGameManager>.Instance.GetCamera(0).StopRendering(false);
}
sw.Stop();
Debug.Log("Level generator took " + sw.ElapsedMilliseconds.ToString() + " milliseconds to complete.");
yield break;
yield break;
}
private bool RandomlyPlaceRoom(RoomAsset room, RoomController existingRoom)
{
RoomController roomController = null;
return this.RandomlyPlaceRoom(room, existingRoom, false, out roomController);
}
private bool RandomlyPlaceRoom(RoomAsset room, bool addDoor, out RoomController roomController)
{
return this.RandomlyPlaceRoom(room, null, addDoor, out roomController);
}
public bool RandomlyPlaceRoom(RoomAsset room, RoomController existingRoom, bool addDoor, out RoomController roomController)
{
bool flag = false;
int index = 0;
List<IntVector2> list = new List<IntVector2>();
List<IntVector2> list2 = new List<IntVector2>(room.potentialDoorPositions);
List<WeightedRoomSpawn> list3 = new List<WeightedRoomSpawn>(this.potentialRoomSpawns);
if (room.requiredDoorPositions.Count > 0)
{
list2 = new List<IntVector2>(room.requiredDoorPositions);
}
else if (room.forcedDoorPositions.Count > 0)
{
list2 = new List<IntVector2>(room.forcedDoorPositions);
}
if (list2.Count > 0)
{
while (list2.Count > 0)
{
int index2 = this.controlledRNG.Next(0, list2.Count);
list.Add(list2[index2]);
list2.RemoveAt(index2);
}
}
else
{
list.Add(default(IntVector2));
}
List<Direction> list4 = new List<Direction>();
List<Direction> list5 = new List<Direction>(Directions.All());
while (list5.Count > 0)
{
int index3 = this.controlledRNG.Next(0, list5.Count);
list4.Add(list5[index3]);
list5.RemoveAt(index3);
}
Direction direction = list4[0];
IntVector2 intVector = list[0];
while (!flag && list3.Count > 0)
{
index = WeightedSelection<RoomSpawn>.ControlledRandomIndexList(WeightedRoomSpawn.Convert(list3), this.controlledRNG);
IntVector2 position = list3[index].selection.position;
RoomSpawn selection = list3[index].selection;
int num = 0;
while (num < list.Count && !flag)
{
intVector = list[num];

```

```

int num2 = 0;
while (num2 < list4.Count && !flag)
{
    direction = list4[num2];
    bool flag2 = true;
    if (!base.RoomFits(room, existingRoom, position, intVector, direction))
    {
        flag2 = false;
    }
    if (flag2)
    {
        flag = true;
        break;
    }
    num2++;
}
num++;
}
if (!flag)
{
    list3.RemoveAt(index);
}
}
if (flag)
{
    if (existingRoom == null)
    {
        roomController = base.LoadRoom(room, list3[index].selection.position, intVector, direction, true);
        RoomController roomController2;
        if (addDoor && base.BuildDoorInDirection(roomController, list3[index].selection.position, list3[index].selection.direction, false, out roomController2))
        {
            roomController.RemoveFromDoorPositions(list3[index].selection.position);
            roomController.ConnectRooms(roomController2);
            roomController2.RemoveFromDoorPositions(list3[index].selection.position + list3[index].selection.direction.ToIntVector2());
        }
    }
    else
    {
        roomController = existingRoom;
        base.LoadIntoExistingRoom(existingRoom, room, list3[index].selection.position, intVector, direction, false);
    }
    return true;
}
Debug.Log(string.Format("Could not find valid position for room {0} after several attempts.", room.name));
roomController = null;
return false;
}
[SerializeField]
private RoomGroup eventSpawnerRoomGroup = new RoomGroup();
private List<RoomController> standardRooms = new List<RoomController>();
}
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Experimental.Rendering;
public class RoomController : MonoBehaviour
{
    private void Awake()
    {
        this.baseMat = new Material(this.defaultMat);
        this.posterMat = new Material(this.defaultPosterMat);
    }
    public void ConnectRooms(RoomController newRoom)
    {
        if (newRoom.type != RoomType.Hall && !this.connectedRooms.Contains(newRoom))
        {
            this.connectedRooms.Add(newRoom);
        }
        if (this.type != RoomType.Hall && !newRoom.connectedRooms.Contains(this))
        {
            newRoom.connectedRooms.Add(this);
        }
        if (newRoom.type == RoomType.Hall)
        {
            this.roomsFromHall = 0;
        }
        else if (this.roomsFromHall > newRoom.roomsFromHall + 1 && newRoom.roomsFromHall != 2147483647)
        {
            this.roomsFromHall = newRoom.roomsFromHall + 1;
        }
    }
}

```

```

    }
    foreach (RoomController roomController in this.connectedRooms)
    {
        roomController.UpdateHallDistance(this.roomsFromHall);
    }
    public void UpdateHallDistance(int distance)
    {
        if (this.roomsFromHall > distance + 1 && distance != 2147483647)
        {
            this.roomsFromHall = distance + 1;
            foreach (RoomController roomController in this.connectedRooms)
            {
                roomController.UpdateHallDistance(this.roomsFromHall);
            }
        }
    }
    public void AddTile(Cell tile)
    {
        this.cells.Add(tile);
    }
    public void RemoveTile(Cell tile)
    {
        this.cells.Remove(tile);
    }
    public void GenerateTextureAtlas()
    {
        this.GenerateTextureAtlas(256);
    }
    public void GenerateTextureAtlas(int size)
    {
        bool flag;
        Texture2D matchingTextureAtlas = this.ec.GetMatchingTextureAtlas(this.florTex, this.wallTex, this.ceilTex, out flag);
        if (flag)
        {
            this.textureAtlas = matchingTextureAtlas;
        }
        else
        {
            this.textureAtlas = new Texture2D(size * 2, size * 2, TextureFormat.RGBA32, false);
            this.textureAtlas.filterMode = FilterMode.Point;
            Color[] colors = new Color[0];
            colors = MaterialModifier.GetColorsForTileTexture(this.florTex, size);
            this.textureAtlas.SetPixels(0, 0, size, size, colors);
            colors = MaterialModifier.GetColorsForTileTexture(this.wallTex, size);
            this.textureAtlas.SetPixels(size, size, size, size, colors);
            colors = MaterialModifier.GetColorsForTileTexture(this.ceilTex, size);
            this.textureAtlas.SetPixels(0, size, size, size, colors);
            this.textureAtlas.Apply();
            this.ec.generatedTextureAtlases.Add(new RoomTextureAtlas(this.florTex, this.wallTex, this.ceilTex, this.textureAtlas));
        }
        if (GraphicsFormatUtility.HasAlphaChannel(this.florTex.graphicsFormat) || GraphicsFormatUtility.HasAlphaChannel(this.wallTex.graphicsFormat) ||
            GraphicsFormatUtility.HasAlphaChannel(this.ceilTex.graphicsFormat))
        {
            this.baseMat = new Material(this.defaultAlphaMat);
            this.posterMat = new Material(this.defaultAlphaPosterMap);
        }
        this.baseMat.SetTexture("_MainTex", this.textureAtlas);
        this.posterMat.SetTexture("_MainTex", this.textureAtlas);
    }
    public void UpdatePosition()
    {
        {
            IntVector2 intVector = new IntVector2(this.ec.levelSize.x, this.ec.levelSize.z);
            IntVector2 intVector2 = default(IntVector2);
            for (int i = 0; i < this.ec.levelSize.x; i++)
            {
                for (int j = 0; j < this.ec.levelSize.z; j++)
                {
                    if (this.ec.cells[i, j].room == this)
                    {
                        {
                            if (i < intVector.x)
                            {
                                intVector.x = i;
                            }
                            if (i > intVector2.x)
                            {
                                intVector2.x = i;
                            }
                            if (j < intVector.z)

```

```

{
    intVector.z = j;
}
if (j > intVector2.z)
{
    intVector2.z = j;
}
}
}
}
this.position = intVector;
this.size.x = intVector2.x - intVector.x + 1;
this.size.z = intVector2.z - intVector.z + 1;
}
public void RemoveFromDoorPositions(IntVector2 position)
{
    this.potentialDoorPositions.Remove(position);
    this.forcedDoorPositions.Remove(position);
}
public bool HasDoorPositions
{
    get
    {
        return this.potentialDoorPositions.Count > 0 || this.forcedDoorPositions.Count > 0;
    }
}
public bool ContainsDoorPosition(IntVector2 position)
{
    return this.potentialDoorPositions.Contains(position) || this.forcedDoorPositions.Contains(position);
}
public void ApplyActivity(Activity activity)
{
    this.hasActivity = true;
    this.activity = activity;
}
public bool HasIncompleteActivity
{
    get
    {
        return this.hasActivity && !this.activity.IsCompleted;
    }
}
public bool notebookCollected
{
    get
    {
        return this.hasActivity && this.activity.NotebookCollected;
    }
}
public bool GetRandomConnectionPoint(Random rng, out IntVector2 position, out Direction direction)
{
    return this.GetRandomConnectionPoint(rng, CellCoverage.None, out position, out direction);
}
public bool GetRandomConnectionPoint(Random rng, CellCoverage requiredCoverage, out IntVector2 position, out Direction direction)
{
    this._potentialConnectionPoints.Clear();
    this._potentialConnectionDirections.Clear();
    for (int i = 0; i < this.connectionPoints.Count; i++)
    {
        Cell cell = this.ec.CellFromPosition(this.connectionPoints[i]);
        if (cell.HasAllFreeWall)
        {
            for (int j = 0; j < 4; j++)
            {
                if (cell.HasWallInDirection((Direction)j) && !cell.WallAnyCovered((Direction)j) && this.ec.ContainsCoordinates(cell.position + ((Direction)j).ToIntVector2())
                    && this.ec.CellFromPosition(cell.position + ((Direction)j).ToIntVector2()).Null && cell.HardCoverageFits(requiredCoverage.Rotated((Direction)j)))
                {
                    this._potentialConnectionPoints.Add(cell.position);
                    this._potentialConnectionDirections.Add((Direction)j);
                }
            }
        }
    }
    if (this._potentialConnectionPoints.Count > 0)
    {
        int index = rng.Next(0, this._potentialConnectionPoints.Count);
        position = this._potentialConnectionPoints[index];
        direction = this._potentialConnectionDirections[index];
        return true;
    }
}

```



```

    }
    position = default(IntVector2);
    direction = Direction.Null;
    return false;
}
public void SetPower(bool val)
{
    if (val != this.powered)
    {
        this.powered = val;
        foreach (Cell cell in this.lights)
        {
            cell.SetLight(this.powered);
        }
        if (this.hasActivity)
        {
            this.activity.SetPower(val);
        }
    }
}
public bool Powered
{
    get
    {
        return this.powered;
    }
}
public Texture2D TextureAtlas
{
    get
    {
        return this.textureAtlas;
    }
}
public List<Cell> GetNewTileList()
{
    return new List<Cell>(this.cells);
}
public List<Cell> AllTilesNoGarbage(bool includeOffLimits, bool includeWithHardCoverage)
{
    this._allTiles.Clear();
    for (int i = 0; i < this.cells.Count; i++)
    {
        if ((includeOffLimits || !this.cells[i].offLimits) && (includeWithHardCoverage || !this.cells[i].HasAnyHardCoverage))
        {
            this._allTiles.Add(this.cells[i]);
        }
    }
    return this._allTiles;
}
public List<Cell> AllEntitySafeCellsNoGarbage()
{
    this._allTiles.Clear();
    for (int i = 0; i < this.entitySafeCells.Count; i++)
    {
        this._allTiles.Add(this.ec.CellFromPosition(this.entitySafeCells[i]));
    }
    return this._allTiles;
}
public Cell RandomEntitySafeCellNoGarbage()
{
    if (this.entitySafeCells.Count > 0)
    {
        return this.ec.CellFromPosition(this.entitySafeCells[Random.Range(0, this.entitySafeCells.Count)]);
    }
    return this.cells[0];
}
public Cell ControlledRandomEntitySafeCellNoGarbage(Random rng)
{
    if (this.entitySafeCells.Count > 0)
    {
        return this.ec.CellFromPosition(this.entitySafeCells[rng.Next(0, this.entitySafeCells.Count)]);
    }
    return this.cells[0];
}
public List<Cell> AllEventSafeCellsNoGarbage()
{
    this._allTiles.Clear();
    for (int i = 0; i < this.eventSafeCells.Count; i++)

```

```

{
    this._allTiles.Add(this.ec.CellFromPosition(this.eventSafeCells[i]));
}
return this._allTiles;
}
public Cell RandomEventSafeCellNoGarbage()
{
    if (this.eventSafeCells.Count > 0)
    {
        return this.ec.CellFromPosition(this.eventSafeCells[Random.Range(0, this.eventSafeCells.Count)]);
    }
    return this.cells[0];
}
public Cell ControlledRandomEventSafeCellNoGarbage(Random rng)
{
    if (this.eventSafeCells.Count > 0)
    {
        return this.ec.CellFromPosition(this.eventSafeCells[rng.Next(0, this.eventSafeCells.Count)]);
    }
    return this.cells[0];
}
public List<Cell> GetTilesOfShape(TileShapeMask shapes, CellCoverage coverageToFit, bool includeOpen)
{
    List<Cell> newTileList = this.GetNewTileList();
    for (int i = 0; i < newTileList.Count; i++)
    {
        if ((shapes & newTileList[i].shape) == TileShapeMask.None || newTileList[i].HasAnyHardCoverage || (newTileList[i].open && !includeOpen) ||
            !newTileList[i].HardCoverageFitsInAnyDirection(coverageToFit))
        {
            newTileList.RemoveAt(i);
            i--;
        }
    }
    return newTileList;
}
public List<Cell> GetTilesOfShape(TileShapeMask shapes, bool includeOpen)
{
    return this.GetTilesOfShape(shapes, CellCoverage.None, includeOpen);
}
public List<Cell> GetPotentialDoorCells()
{
    List<Cell> list = new List<Cell>();
    foreach (IntVector2 intVector in this.potentialDoorPositions)
    {
        list.Add(this.ec.CellFromPosition(intVector));
    }
    foreach (IntVector2 intVector2 in this.forcedDoorPositions)
    {
        list.Add(this.ec.CellFromPosition(intVector2));
    }
    return list;
}
public Cell TileAtIndex(int index)
{
    return this.cells[index];
}
public List<Cell> GetSurroundingCells(bool includeNull)
{
    List<Cell> list = new List<Cell>();
    foreach (Cell cell in this.cells)
    {
        foreach (Cell cell2 in this.ec.GetCellNeighbors(cell.position, includeNull))
        {
            if (cell2.room != this && !list.Contains(cell2))
            {
                list.Add(cell2);
            }
        }
    }
    return list;
}
public int TileCount
{
    get
    {
        return this.cells.Count;
    }
}
public int AvailableItemRespawnPoints

```

```

{
    get
    {
        int num = 0;
        for (int i = 0; i < this.pickups.Count; i++)
        {
            if (!this.pickups[i].gameObject.activeSelf)
            {
                num++;
            }
        }
        return num;
    }
}

public bool ContainsCoordinates(IntVector2 pos)
{
    return pos.x >= this.position.x && pos.x < this.position.x + this.size.x && pos.z >= this.position.z && pos.z < this.position.z + this.size.z;
}

public bool containsPosition(Vector3 position)
{
    return position.x <= this.ec.RealRoomMax(this).x && position.x >= this.ec.RealRoomMin(this).x && position.z <= this.ec.RealRoomMax(this).z && position.z
    >= this.ec.RealRoomMin(this).z;
}

public bool HasFreeWall
{
    get
    {
        using (List<Cell>.Enumerator enumerator = this.cells.GetEnumerator())
        {
            while (enumerator.MoveNext())
            {
                if (enumerator.Current.HasSoftFreeWall)
                {
                    return true;
                }
            }
        }
        return false;
    }
}

public EnvironmentController ec;
public RoomCategory category;
public RoomType type;
public Direction dir;
public Color color = Color.white;
public IntVector2 loadedPosition;
public IntVector2 loadedPivot;
public Material mapMaterial;
public Texture2D florTex;
public Texture2D wallTex;
public Texture2D ceilTex;
public StandardDoorMats doorMats;
public WindowObject windowObject;
public IntVector2 position;
public IntVector2 size;
public IntVector2 maxSize;
public int spawnWeight = 100;
public int nearbyRooms;
public int roomsFromHall = int.MaxValue;
public List<ItemObject> itemList = new List<ItemObject>();
public int minItemValue;
public int maxItemValue = 100;
public int currentItemValue;
public bool expandable;
public bool acceptsExits;
public bool acceptsPosters = true;
public List<Cell> cells = new List<Cell>();
public List<Cell> lights = new List<Cell>();
private List<Cell> _allTiles = new List<Cell>();
public List<IntVector2> builtDoorPositions = new List<IntVector2>();
public List<IntVector2> potentialDoorPositions = new List<IntVector2>();
public List<IntVector2> forcedDoorPositions = new List<IntVector2>();
public List<IntVector2> connectionPoints = new List<IntVector2>();
public List<IntVector2> entitySafeCells = new List<IntVector2>();
public List<IntVector2> eventSafeCells = new List<IntVector2>();
public List<IntVector2> standardLightCells = new List<IntVector2>();
public List<IntVector2> secretCells = new List<IntVector2>();
public List<IntVector2> cellsExcludedFromRoomGroup = new List<IntVector2>();
public List<RoomController> connectedRooms;

```

```

public RoomFunctionContainer functions;
public GameObject functionObject;
public GameObject objectObject;
private Activity activity;
public Door doorPre;
public Transform lightPre;
public List<Door> doors;
public List<Direction> doorDirs;
public List<ItemSpawnPoint> itemSpawnPoints = new List<ItemSpawnPoint>();
public List<Pickup> pickups = new List<Pickup>();
public WeightedTransform[] decorations = new WeightedTransform[0];
public List<WeightedPosterObject> potentialPosters;
public float windowChance;
public float posterChance;
public bool spawnItems = true;
public bool offLimits;
private bool powered = true;
private bool hasActivity;
public Material defaultMat;
public Material defaultAlphaMat;
public Material defaultPosterMat;
public Material defaultAlphaPosterMap;
public Material baseMat;
public Material posterMat;
public Material wallMat;
public Material floorMat;
public Material ceilingMat;
public Texture2D textureAtlas;
public Texture2D defaultPoster;
private List<IntVector2> _potentialConnectionPoints = new List<IntVector2>();
private List<Direction> _potentialConnectionDirections = new List<Direction>();
}
public Cell[,] cells;
using System;
namespace PlusStudioLevelFormat
{
    public class RoomCellInfo
    {
    public RoomCellInfo()
    {
    }
    public RoomCellInfo(Cell cell)
    {
    this.position = cell.position;
    this.walls = cell.walls;
    }
    public ByteVector2 position;
    public Nybble walls = new Nybble(0);
    public PlusCellCoverage coverage;
    }
    }
    public void RefreshCells(bool refreshLights = true, bool updateDataCells = true)
    {
    if (updateDataCells)
    {
    {
    this.levelData.UpdateCells(true);
    }
    for (int i = 0; i < this.workerEc.cells.GetLength(0); i++)
    {
    for (int j = 0; j < this.workerEc.cells.GetLength(1); j++)
    {
    if (this.levelData.cells[i, j].type == 16)
    {
    this.workerEc.cells[i, j].Tile.gameObject.SetActive(false);
    if (!this.workerEc.cells[i, j].Null)
    {
    EditorController._initialized.SetValue(this.workerEc.cells[i, j], false);
    }
    }
    }
    else
    {
    EditorRoom editorRoom = this.levelData.RoomFromId(this.levelData.GetCellSafe(i, j).roomId);
    this.workerEc.cells[i, j].Tile.gameObject.SetActive(true);
    this.workerEc.cells[i, j].Tile.MeshRenderer.material.SetMainTexture(this.GenerateTextureAtlas(editorRoom.floorTex, editorRoom.wallTex,
    editorRoom.ceilTex));
    this.workerEc.cells[i, j].SetShape(this.levelData.cells[i, j].type, TileShapeMask.None);
    if (this.workerEc.cells[i, j].Null)
    {
    this.workerEc.cells[i, j].Initialize();

```

```

    }
    }
    }
    }
    for (int k = 0; k < this.levelData.premadeRooms.Count; k++)
    {
        this.levelData.premadeRooms[k].ModifyCellDisplay(this);
    }
    this.UnhighlightAllCells();
    this.UpdateStructuresWithReason(PotentialStructureUpdateReason.CellChange);
    if (!refreshLights)
    {
        return;
    }
    this.RefreshLights();
    }
    public virtual void PlayLevel()
    {
        if (this.currentFileName != null && File.Exists(Path.Combine(LevelStudioPlugin.levelFilePath, this.currentFileName + ".ebpl")))
        {
            EditorController.lastPlayedLevel = this.currentFileName;
        }
        else
        {
            EditorController.lastPlayedLevel = null;
        }
        this.CompileAndPlay();
    }
    public void CompileAndPlay()
    {
        BaldiLevel baldiLevel = this.Compile();
        AccessTools.Field(typeof(Singleton<CoreGameManager>), "m_Instance").SetValue(null, null);
        AccessTools.Field(typeof(Singleton<StickerManager>), "m_Instance").SetValue(null, null);
        PlayableEditorLevel playableEditorLevel = new PlayableEditorLevel();
        playableEditorLevel.uniqueId = 0UL;
        playableEditorLevel.data = baldiLevel;
        playableEditorLevel.meta = this.levelData.meta.CompileContent();
        if (!this.currentMode.vanillaComplaint)
        {
            this.HandleNPCProperties(baldiLevel, playableEditorLevel.meta.contentPackage);
        }
        EditorPlayModeManager.LoadLevel(playableEditorLevel, 0, true, EditorController.lastPlayedLevel, this.currentMode.id);
        this.DestroySelf();
    }
    using System;
    using System.IO;
    namespace PlusLevelStudio.Editor
    {
        public class EditorFileContainer
        {
            public void Write(BinaryWriter writer)
            {
                this.meta.Write(writer);
                this.data.Write(writer);
            }
            public static EditorFileContainer Read(BinaryReader reader)
            {
                return new EditorFileContainer
                {
                    meta = EditorFileMeta.Read(reader, null),
                    data = EditorLevelData.ReadFrom(reader)
                };
            }
            public static EditorFileContainer ReadMindful(BinaryReader reader)
            {
                EditorFileContainer editorFileContainer = new EditorFileContainer();
                long position = reader.BaseStream.Position;
                if (reader.ReadByte() < 2)
                {
                    reader.BaseStream.Position = position;
                    editorFileContainer.data = EditorLevelData.ReadFrom(reader);
                    return editorFileContainer;
                }
                reader.BaseStream.Position = position;
                editorFileContainer.meta = EditorFileMeta.Read(reader, null);
                editorFileContainer.data = EditorLevelData.ReadFrom(reader);
                return editorFileContainer;
            }
        }
    }

```

```

    }
    public EditorFileMeta meta;
    public EditorLevelData data;
    }
    }
    public void UpdateVisualsForRoom(EditorRoom room)
    {
        if (room.myVisual == null)
        {
            return;
        }
        room.myVisual.RoomUpdated();
    }
    public BaldiLevel Compile()
    {
        this.selector.DisableSelection();
        BaldiLevel baldiLevel = this.levelData.Compile();
        for (int i = 0; i < this.levelData.objects.Count; i++)
        {
            this.GetVisual(this.levelData.objects[i]).GetComponent<EditorBasicObject>().SetMode(false);
        }
        baldiLevel.entitySafeCells = this.CompileSafeCells(this.levelData, 1f);
        baldiLevel.eventSafeCells = this.CompileSafeCells(this.levelData, 2f);
        baldiLevel.coverage = this.CompileBlockedCells(this.levelData, 1.5f, 3.4f);
        this.levelData.FinalizeCompile(baldiLevel);
        for (int j = 0; j < this.levelData.markers.Count; j++)
        {
            this.levelData.markers[j].Compile(this.levelData, baldiLevel);
        }
        for (int k = 0; k < this.levelData.objects.Count; k++)
        {
            this.GetVisual(this.levelData.objects[k]).GetComponent<EditorBasicObject>().SetMode(true);
        }
        return baldiLevel;
    }
    using System;
    using System.Collections.Generic;
    using System.IO;
    using System.Linq;
    namespace PlusStudioLevelFormat
    {
        public class BaldiLevel
        {
            public UnityVector3 UnitySpawnPoint
            {
                get
                {
                    {
                        if ((from x in this.exits
                             where x.isSpawn
                             select x).Count<ExitInfo>() == 0)
                        {
                            return this.spawnPoint;
                        }
                        ExitInfo exitInfo = this.exits.Last((ExitInfo x) => x.isSpawn);
                        return new UnityVector3((float)exitInfo.position.x * 10f + 5f, 5f, (float)exitInfo.position.y * 10f + 5f);
                    }
                }
            }
            public BaldiLevel(ByteVector2 size)
            {
                this.Initialize(size);
            }
            public BaldiLevel()
            {
            }
            public void Initialize(ByteVector2 size)
            {
                this.levelSize = size;
                this.cells = new Cell[(int)size.x, (int)size.y];
                this.entitySafeCells = new bool[(int)size.x, (int)size.y];
                this.eventSafeCells = new bool[(int)size.x, (int)size.y];
                this.secretCells = new bool[(int)size.x, (int)size.y];
                this.excludedFromRoomGroupCells = new bool[(int)size.x, (int)size.y];
                this.coverage = new PlusCellCoverage[(int)size.x, (int)size.y];
                for (int i = 0; i < (int)this.levelSize.x; i++)
                {
                    {
                        for (int j = 0; j < (int)this.levelSize.y; j++)
                        {
                            this.cells[i, j] = new Cell(new ByteVector2(i, j));
                            this.entitySafeCells[i, j] = false;
                        }
                    }
                }
            }
        }
    }

```

```

this.eventSafeCells[i, j] = false;
this.secretCells[i, j] = false;
this.excludedFromRoomGroupCells[i, j] = false;
this.coverage[i, j] = PlusCellCoverage.None;
}
}
}
public static BaldiLevel Read(BinaryReader reader)
{
byte b = reader.ReadByte();
StringCompressor stringCompressor = StringCompressor.ReadStringDatabase(reader);
StringCompressor stringCompressor2 = StringCompressor.ReadStringDatabase(reader);
BaldiLevel baldiLevel = new BaldiLevel();
baldiLevel.levelTitle = reader.ReadString();
baldiLevel.timeLimit = reader.ReadSingle();
if (b >= 3)
{
baldiLevel.seed = reader.ReadInt32();
}
if (b >= 5)
{
baldiLevel.usesMap = reader.ReadBoolean();
}
baldiLevel.skybox = reader.ReadString();
baldiLevel.minLightColor = reader.ReadUnityColor();
baldiLevel.lightMode = (PlusLightMode)reader.ReadByte();
baldiLevel.initialRandomEventGap = reader.ReadSingle();
baldiLevel.minRandomEventGap = reader.ReadSingle();
baldiLevel.maxRandomEventGap = reader.ReadSingle();
int num = reader.ReadInt32();
for (int i = 0; i < num; i++)
{
baldiLevel.randomEvents.Add(reader.ReadString());
}
baldiLevel.spawnPoint = reader.ReadUnityVector3();
baldiLevel.spawnDirection = (PlusDirection)reader.ReadByte();
baldiLevel.Initialize(reader.ReadByteVector2());
Nybble[] array = reader.ReadNybbles();
int num2 = 0;
for (int j = 0; j < (int)baldiLevel.levelSize.x; j++)
{
for (int k = 0; k < (int)baldiLevel.levelSize.y; k++)
{
baldiLevel.cells[j, k].walls = array[num2];
num2++;
}
}
for (int l = 0; l < (int)baldiLevel.levelSize.x; l++)
{
for (int m = 0; m < (int)baldiLevel.levelSize.y; m++)
{
baldiLevel.cells[l, m].roomId = reader.ReadUInt16();
}
}
bool[] array2 = reader.ReadBoolArray();
for (int n = 0; n < (int)baldiLevel.levelSize.x; n++)
{
for (int num3 = 0; num3 < (int)baldiLevel.levelSize.y; num3++)
{
baldiLevel.entitySafeCells[n, num3] = array2[n * (int)baldiLevel.levelSize.y + num3];
}
}
bool[] array3 = reader.ReadBoolArray();
for (int num4 = 0; num4 < (int)baldiLevel.levelSize.x; num4++)
{
for (int num5 = 0; num5 < (int)baldiLevel.levelSize.y; num5++)
{
baldiLevel.eventSafeCells[num4, num5] = array3[num4 * (int)baldiLevel.levelSize.y + num5];
}
}
bool[] array4 = reader.ReadBoolArray();
for (int num6 = 0; num6 < (int)baldiLevel.levelSize.x; num6++)
{
for (int num7 = 0; num7 < (int)baldiLevel.levelSize.y; num7++)
{
baldiLevel.secretCells[num6, num7] = array4[num6 * (int)baldiLevel.levelSize.y + num7];
}
}
if (b >= 9)

```

```

{
    bool[] array5 = reader.ReadBoolArray();
    for (int num8 = 0; num8 < (int)baldiLevel.levelSize.x; num8++)
    {
        for (int num9 = 0; num9 < (int)baldiLevel.levelSize.y; num9++)
        {
            baldiLevel.excludedFromRoomGroupCells[num8, num9] = array5[num8 * (int)baldiLevel.levelSize.y + num9];
        }
    }
    for (int num10 = 0; num10 < (int)baldiLevel.levelSize.x; num10++)
    {
        for (int num11 = 0; num11 < (int)baldiLevel.levelSize.y; num11++)
        {
            baldiLevel.coverage[num10, num11] = (PlusCellCoverage)reader.ReadByte();
        }
    }
    int num12 = reader.ReadInt32();
    for (int num13 = 0; num13 < num12; num13++)
    {
        RoomInfo roomInfo;
        if (b < 6)
        {
            roomInfo = new RoomInfo(stringCompressor.ReadStoredString(reader), new TextureContainer(stringCompressor.ReadStoredString(reader),
            stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader)));
        }
        else
        {
            roomInfo = new RoomInfo(stringCompressor.ReadStoredString(reader), reader.ReadString(), new
            TextureContainer(stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader)));
        }
        int num14 = reader.ReadInt32();
        for (int num15 = 0; num15 < num14; num15++)
        {
            roomInfo.items.Add(new ItemInfo
            {
                item = stringCompressor2.ReadStoredString(reader),
                position = reader.ReadUnityVector2()
            });
        }
        if (b >= 1)
        {
            int num16 = reader.ReadInt32();
            for (int num17 = 0; num17 < num16; num17++)
            {
                roomInfo.itemSpawns.Add(new ItemSpawnInfo
                {
                    weight = reader.ReadInt32(),
                    position = reader.ReadUnityVector2()
                });
            }
            int num18 = reader.ReadInt32();
            for (int num19 = 0; num19 < num18; num19++)
            {
                roomInfo.basicObjects.Add(new BasicObjectInfo
                {
                    prefab = stringCompressor2.ReadStoredString(reader),
                    position = reader.ReadUnityVector3(),
                    rotation = reader.ReadUnityQuaternion()
                });
            }
            string text = stringCompressor2.ReadStoredString(reader);
            if (text != "null")
            {
                ActivityInfo activity = new ActivityInfo
                {
                    type = text,
                    position = reader.ReadUnityVector3(),
                    direction = (PlusDirection)reader.ReadByte()
                };
                roomInfo.activity = activity;
            }
            if (b >= 10)
            {
                roomInfo.minItemValue = reader.ReadInt32();
                roomInfo.maxItemValue = reader.ReadInt32();
            }
            baldiLevel.rooms.Add(roomInfo);
        }
    }
}

```



```

}
int num20 = reader.ReadInt32();
for (int num21 = 0; num21 < num20; num21++)
{
    baldiLevel.lights.Add(new LightInfo
    {
        prefab = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        color = reader.ReadUnityColor(),
        strength = reader.ReadByte()
    });
}
int num22 = reader.ReadInt32();
for (int num23 = 0; num23 < num22; num23++)
{
    baldiLevel.doors.Add(new DoorInfo
    {
        prefab = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte(),
        roomId = reader.ReadUInt16()
    });
}
int num24 = reader.ReadInt32();
for (int num25 = 0; num25 < num24; num25++)
{
    baldiLevel.windows.Add(new WindowInfo
    {
        prefab = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte()
    });
}
int num26 = reader.ReadInt32();
for (int num27 = 0; num27 < num26; num27++)
{
    baldiLevel.tileObjects.Add(new TileObjectInfo
    {
        prefab = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte()
    });
}
int num28 = reader.ReadInt32();
for (int num29 = 0; num29 < num28; num29++)
{
    baldiLevel.exits.Add(new ExitInfo
    {
        type = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte(),
        isSpawn = reader.ReadBoolean()
    });
}
int num30 = reader.ReadInt32();
for (int num31 = 0; num31 < num30; num31++)
{
    StructureInfo structureInfo = new StructureInfo();
    structureInfo.type = stringCompressor2.ReadStoredString(reader);
    int num32 = reader.ReadInt32();
    for (int num33 = 0; num33 < num32; num33++)
    {
        if (b >= 2)
        {
            structureInfo.data.Add(new StructureDataInfo
            {
                prefab = stringCompressor2.ReadStoredString(reader),
                position = reader.ReadMystIntVector2(),
                direction = (PlusDirection)reader.ReadByte(),
                data = reader.ReadInt32()
            });
        }
        else
        {
            structureInfo.data.Add(new StructureDataInfo
            {
                prefab = stringCompressor2.ReadStoredString(reader),
                position = new MystIntVector2(reader.ReadByte(), reader.ReadByte()),
                direction = (PlusDirection)reader.ReadByte(),
            }

```

```

data = reader.ReadInt32()
});
}
}
baldiLevel.structures.Add(structureInfo);
}
int num34 = reader.ReadInt32();
for (int num35 = 0; num35 < num34; num35++)
{
    baldiLevel.npcs.Add(new NPCInfo
    {
        npc = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2()
    });
}
int num36 = reader.ReadInt32();
for (int num37 = 0; num37 < num36; num37++)
{
    baldiLevel.posters.Add(new PosterInfo
    {
        poster = stringCompressor2.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte()
    });
}
if (b <= 2)
{
    return baldiLevel;
}
int num38 = reader.ReadInt32();
for (int num39 = 0; num39 < num38; num39++)
{
    RandomStructureInfo randomStructureInfo = new RandomStructureInfo();
    randomStructureInfo.type = stringCompressor2.ReadStoredString(reader);
    int num40 = reader.ReadInt32();
    for (int num41 = 0; num41 < num40; num41++)
    {
        randomStructureInfo.info.chance.Add(reader.ReadSingle());
    }
    int num42 = reader.ReadInt32();
    for (int num43 = 0; num43 < num42; num43++)
    {
        randomStructureInfo.info.minMax.Add(reader.ReadMystIntVector2());
    }
    int num44 = reader.ReadInt32();
    for (int num45 = 0; num45 < num44; num45++)
    {
        randomStructureInfo.info.prefab.Add(new WeightedPrefab
        {
            prefab = stringCompressor2.ReadStoredString(reader),
            weight = reader.ReadInt32()
        });
    }
    baldiLevel.randomStructures.Add(randomStructureInfo);
}
if (b <= 3)
{
    return baldiLevel;
}
int num46 = reader.ReadInt32();
for (int num47 = 0; num47 < num46; num47++)
{
    baldiLevel.potentialStickers.Add(new WeightedID
    {
        id = reader.ReadString(),
        weight = reader.ReadInt32()
    });
}
if (b >= 8)
{
    baldiLevel.storeItemCount = reader.ReadInt32();
    int num48 = reader.ReadInt32();
    for (int num49 = 0; num49 < num48; num49++)
    {
        baldiLevel.potentialStoreItems.Add(new WeightedID
        {
            id = reader.ReadString(),
            weight = reader.ReadInt32()
        });
    }
}

```

```

    }
}
if (b <= 7)
{
    return baldiLevel;
}
int num50 = reader.ReadInt32();
for (int num51 = 0; num51 < num50; num51++)
{
    RoomPlacementInfo roomPlacementInfo = new RoomPlacementInfo
    {
        room = stringCompressor.ReadStoredString(reader),
        position = reader.ReadByteVector2(),
        direction = (PlusDirection)reader.ReadByte(),
        doorSpawnId = reader.ReadInt32()
    };
    if (reader.ReadBoolean())
    {
        roomPlacementInfo.textureOverride = new TextureContainer(stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader),
        stringCompressor.ReadStoredString(reader));
    }
    baldiLevel.premadeRoomPlacements.Add(roomPlacementInfo);
}
return baldiLevel;
}
public void Write(BinaryWriter writer)
{
    StringCompressor stringCompressor = new StringCompressor();
    stringCompressor.AddStrings(from x in this.rooms
    select x.type);
    stringCompressor.AddStrings(from x in this.rooms
    select x.textureContainer.floor);
    stringCompressor.AddStrings(from x in this.rooms
    select x.textureContainer.wall);
    stringCompressor.AddStrings(from x in this.rooms
    select x.textureContainer.ceiling);
    stringCompressor.AddStrings(from x in this.premadeRoomPlacements
    select x.room);
    for (int i = 0; i < this.premadeRoomPlacements.Count; i++)
    {
        if (this.premadeRoomPlacements[i].textureOverride != null)
        {
            stringCompressor.AddString(this.premadeRoomPlacements[i].textureOverride.floor);
            stringCompressor.AddString(this.premadeRoomPlacements[i].textureOverride.wall);
            stringCompressor.AddString(this.premadeRoomPlacements[i].textureOverride.ceiling);
        }
    }
    StringCompressor stringCompressor2 = new StringCompressor();
    foreach (RoomInfo roomInfo in this.rooms)
    {
        stringCompressor2.AddStrings(from x in roomInfo.items
        select x.item);
        stringCompressor2.AddStrings(from x in roomInfo.basicObjects
        select x.prefab);
        if (roomInfo.activity == null)
        {
            stringCompressor2.AddString("null");
        }
        else
        {
            stringCompressor2.AddString(roomInfo.activity.type);
        }
    }
    stringCompressor2.AddStrings(from x in this.lights
    select x.prefab);
    stringCompressor2.AddStrings(from x in this.doors
    select x.prefab);
    stringCompressor2.AddStrings(from x in this.windows
    select x.prefab);
    stringCompressor2.AddStrings(from x in this.tileObjects
    select x.prefab);
    stringCompressor2.AddStrings(from x in this.exits
    select x.type);
    stringCompressor2.AddStrings(from x in this.npcs
    select x.npc);
    stringCompressor2.AddStrings(from x in this.posters
    select x.poster);
    stringCompressor2.AddStrings(from x in this.structures
    select x.type);
}

```

```

stringCompressor2.AddStrings(from x in this.randomStructures
select x.type);
stringCompressor2.AddStrings(this.randomStructures.SelectMany((RandomStructureInfo x) => from z in x.info.prefab
select z.prefab));
foreach (StructureInfo structureInfo in this.structures)
{
stringCompressor2.AddStrings(from x in structureInfo.data
where x.prefab != null
select x.prefab);
}
stringCompressor2.FinalizeDatabase();
stringCompressor.FinalizeDatabase();
writer.Write(BaldiLevel.version);
stringCompressor.WriteStringDatabase(writer);
stringCompressor2.WriteStringDatabase(writer);
writer.Write(this.levelTitle);
writer.Write(this.timeLimit);
writer.Write(this.seed);
writer.Write(this.usesMap);
writer.Write(this.skybox);
writer.Write(this.minLightColor);
writer.Write((byte)this.lightMode);
writer.Write(this.initialRandomEventGap);
writer.Write(this.minRandomEventGap);
writer.Write(this.maxRandomEventGap);
writer.Write(this.randomEvents.Count);
for (int j = 0; j < this.randomEvents.Count; j++)
{
writer.Write(this.randomEvents[j]);
}
writer.Write(this.spawnPoint);
writer.Write((byte)this.spawnDirection);
writer.Write(this.levelSize);
List<Nybble> list = new List<Nybble>();
for (int k = 0; k < (int)this.levelSize.x; k++)
{
for (int l = 0; l < (int)this.levelSize.y; l++)
{
list.Add(this.cells[k, l].walls);
}
}
writer.Write(list.ToArray());
for (int m = 0; m < (int)this.levelSize.x; m++)
{
for (int n = 0; n < (int)this.levelSize.y; n++)
{
writer.Write(this.cells[m, n].roomId);
}
}
List<bool> list2 = new List<bool>();
for (int num = 0; num < (int)this.levelSize.x; num++)
{
for (int num2 = 0; num2 < (int)this.levelSize.y; num2++)
{
list2.Add(this.entitySafeCells[num, num2]);
}
}
writer.Write(list2.ToArray());
list2.Clear();
for (int num3 = 0; num3 < (int)this.levelSize.x; num3++)
{
for (int num4 = 0; num4 < (int)this.levelSize.y; num4++)
{
list2.Add(this.eventSafeCells[num3, num4]);
}
}
writer.Write(list2.ToArray());
list2.Clear();
for (int num5 = 0; num5 < (int)this.levelSize.x; num5++)
{
for (int num6 = 0; num6 < (int)this.levelSize.y; num6++)
{
list2.Add(this.secretCells[num5, num6]);
}
}
writer.Write(list2.ToArray());
list2.Clear();
for (int num7 = 0; num7 < (int)this.levelSize.x; num7++)
{

```

```

for (int num8 = 0; num8 < (int)this.levelSize.y; num8++)
{
list2.Add(this.excludedFromRoomGroupCells[num7, num8]);
}
}
writer.Write(list2.ToArray());
for (int num9 = 0; num9 < (int)this.levelSize.x; num9++)
{
for (int num10 = 0; num10 < (int)this.levelSize.y; num10++)
{
writer.Write((byte)this.coverage[num9, num10]);
}
}
writer.Write(this.rooms.Count);
for (int num11 = 0; num11 < this.rooms.Count; num11++)
{
stringCompressor.WriteStoredString(writer, this.rooms[num11].type);
writer.Write(this.rooms[num11].name);
stringCompressor.WriteStoredString(writer, this.rooms[num11].textureContainer.floor);
stringCompressor.WriteStoredString(writer, this.rooms[num11].textureContainer.wall);
stringCompressor.WriteStoredString(writer, this.rooms[num11].textureContainer.ceiling);
writer.Write(this.rooms[num11].items.Count);
for (int num12 = 0; num12 < this.rooms[num11].items.Count; num12++)
{
stringCompressor2.WriteStoredString(writer, this.rooms[num11].items[num12].item);
writer.Write(this.rooms[num11].items[num12].position);
}
writer.Write(this.rooms[num11].itemSpawns.Count);
for (int num13 = 0; num13 < this.rooms[num11].itemSpawns.Count; num13++)
{
writer.Write(this.rooms[num11].itemSpawns[num13].weight);
writer.Write(this.rooms[num11].itemSpawns[num13].position);
}
writer.Write(this.rooms[num11].basicObjects.Count);
for (int num14 = 0; num14 < this.rooms[num11].basicObjects.Count; num14++)
{
stringCompressor2.WriteStoredString(writer, this.rooms[num11].basicObjects[num14].prefab);
writer.Write(this.rooms[num11].basicObjects[num14].position);
writer.Write(this.rooms[num11].basicObjects[num14].rotation);
}
if (this.rooms[num11].activity == null)
{
stringCompressor2.WriteStoredString(writer, "null");
}
else
{
stringCompressor2.WriteStoredString(writer, this.rooms[num11].activity.type);
writer.Write(this.rooms[num11].activity.position);
writer.Write((byte)this.rooms[num11].activity.direction);
}
writer.Write(this.rooms[num11].minItemValue);
writer.Write(this.rooms[num11].maxItemValue);
}
writer.Write(this.lights.Count);
for (int num15 = 0; num15 < this.lights.Count; num15++)
{
stringCompressor2.WriteStoredString(writer, this.lights[num15].prefab);
writer.Write(this.lights[num15].position);
writer.Write(this.lights[num15].color);
writer.Write(this.lights[num15].strength);
}
writer.Write(this.doors.Count);
for (int num16 = 0; num16 < this.doors.Count; num16++)
{
stringCompressor2.WriteStoredString(writer, this.doors[num16].prefab);
writer.Write(this.doors[num16].position);
writer.Write((byte)this.doors[num16].direction);
writer.Write(this.doors[num16].roomId);
}
writer.Write(this.windows.Count);
for (int num17 = 0; num17 < this.windows.Count; num17++)
{
stringCompressor2.WriteStoredString(writer, this.windows[num17].prefab);
writer.Write(this.windows[num17].position);
writer.Write((byte)this.windows[num17].direction);
}
writer.Write(this.tileObjects.Count);
for (int num18 = 0; num18 < this.tileObjects.Count; num18++)
{

```

```

stringCompressor2.WriteStoredString(writer, this.tileObjects[num18].prefab);
writer.Write(this.tileObjects[num18].position);
writer.Write((byte)this.tileObjects[num18].direction);
}
writer.Write(this.exits.Count);
for (int num19 = 0; num19 < this.exits.Count; num19++)
{
stringCompressor2.WriteStoredString(writer, this.exits[num19].type);
writer.Write(this.exits[num19].position);
writer.Write((byte)this.exits[num19].direction);
writer.Write(this.exits[num19].isSpawn);
}
writer.Write(this.structures.Count);
for (int num20 = 0; num20 < this.structures.Count; num20++)
{
stringCompressor2.WriteStoredString(writer, this.structures[num20].type);
writer.Write(this.structures[num20].data.Count);
for (int num21 = 0; num21 < this.structures[num20].data.Count; num21++)
{
StructureDataInfo structureDataInfo = this.structures[num20].data[num21];
stringCompressor2.WriteStoredString(writer, structureDataInfo.prefab);
writer.Write(structureDataInfo.position);
writer.Write((byte)structureDataInfo.direction);
writer.Write(structureDataInfo.data);
}
}
writer.Write(this.npcs.Count);
for (int num22 = 0; num22 < this.npcs.Count; num22++)
{
stringCompressor2.WriteStoredString(writer, this.npcs[num22].npc);
writer.Write(this.npcs[num22].position);
}
writer.Write(this.posters.Count);
for (int num23 = 0; num23 < this.posters.Count; num23++)
{
stringCompressor2.WriteStoredString(writer, this.posters[num23].poster);
writer.Write(this.posters[num23].position);
writer.Write((byte)this.posters[num23].direction);
}
writer.Write(this.randomStructures.Count);
for (int num24 = 0; num24 < this.randomStructures.Count; num24++)
{
stringCompressor2.WriteStoredString(writer, this.randomStructures[num24].type);
writer.Write(this.randomStructures[num24].info.chance.Count);
for (int num25 = 0; num25 < this.randomStructures[num24].info.chance.Count; num25++)
{
writer.Write(this.randomStructures[num24].info.chance[num25]);
}
writer.Write(this.randomStructures[num24].info.minMax.Count);
for (int num26 = 0; num26 < this.randomStructures[num24].info.minMax.Count; num26++)
{
writer.Write(this.randomStructures[num24].info.minMax[num26]);
}
writer.Write(this.randomStructures[num24].info.prefab.Count);
for (int num27 = 0; num27 < this.randomStructures[num24].info.prefab.Count; num27++)
{
stringCompressor2.WriteStoredString(writer, this.randomStructures[num24].info.prefab[num27].prefab);
writer.Write(this.randomStructures[num24].info.prefab[num27].weight);
}
}
writer.Write(this.potentialStickers.Count);
for (int num28 = 0; num28 < this.potentialStickers.Count; num28++)
{
writer.Write(this.potentialStickers[num28].id);
writer.Write(this.potentialStickers[num28].weight);
}
writer.Write(this.storeItem.Count);
writer.Write(this.potentialStoreItems.Count);
for (int num29 = 0; num29 < this.potentialStoreItems.Count; num29++)
{
writer.Write(this.potentialStoreItems[num29].id);
writer.Write(this.potentialStoreItems[num29].weight);
}
writer.Write(this.premadeRoomPlacements.Count);
for (int num30 = 0; num30 < this.premadeRoomPlacements.Count; num30++)
{
stringCompressor2.WriteStoredString(writer, this.premadeRoomPlacements[num30].room);
writer.Write(this.premadeRoomPlacements[num30].position);
writer.Write((byte)this.premadeRoomPlacements[num30].direction);
}
}

```

```

writer.Write(this.premadeRoomPlacements[num30].doorSpawnId);
writer.Write(this.premadeRoomPlacements[num30].textureOverride != null);
if (this.premadeRoomPlacements[num30].textureOverride != null)
{
    stringCompressor.WriteStoredString(writer, this.premadeRoomPlacements[num30].textureOverride.floor);
    stringCompressor.WriteStoredString(writer, this.premadeRoomPlacements[num30].textureOverride.wall);
    stringCompressor.WriteStoredString(writer, this.premadeRoomPlacements[num30].textureOverride.ceiling);
}
}
}

public ByteVector2 levelSize;
public Cell[,] cells;
public bool[,] entitySafeCells;
public bool[,] eventSafeCells;
public bool[,] secretCells;
public bool[,] excludedFromRoomGroupCells;
public PlusCellCoverage[,] coverage;
public List<RoomInfo> rooms = new List<RoomInfo>();
public List<LightInfo> lights = new List<LightInfo>();
public List<TileObjectInfo> tileObjects = new List<TileObjectInfo>();
public List<DoorInfo> doors = new List<DoorInfo>();
public List<WindowInfo> windows = new List<WindowInfo>();
public List<ExitInfo> exits = new List<ExitInfo>();
public List<StructureInfo> structures = new List<StructureInfo>();
public List<RandomStructureInfo> randomStructures = new List<RandomStructureInfo>();
public List<NPCInfo> npcs = new List<NPCInfo>();
public List<PosterInfo> posters = new List<PosterInfo>();
public List<RoomPlacementInfo> premadeRoomPlacements = new List<RoomPlacementInfo>();
public UnityVector3 spawnPoint = new UnityVector3(5f, 5f, 5f);
public PlusDirection spawnDirection;
public static readonly byte version = 10;
public string levelTitle = "WIP";
public float timeLimit;
public string skybox = "default";
public UnityColor minLightColor = new UnityColor(0f, 0f, 0f);
public PlusLightMode lightMode = PlusLightMode.Cumulative;
public float initialRandomEventGap = 30f;
public float minRandomEventGap = 45f;
public float maxRandomEventGap = 180f;
public List<string> randomEvents = new List<string>();
public int seed;
public List<WeightedID> potentialStickers = new List<WeightedID>();
public List<WeightedID> potentialStoreItems = new List<WeightedID>();
public int storeItemCount;
public bool usesMap = true;
}
}

using System;
using System.Collections.Generic;
using System.IO;
using MTM101BaldAPI;
using PlusStudioLevelLoader;
using TMPro;
using UnityEngine;
namespace PlusLevelStudio
{
    public class EditorPlayModeManager : Singleton<EditorPlayModeManager>
    {
        public void OnExit()
        {
            if (this.customContent != null)
            {
                this.customContent.CleanupContent();
            }
            for (int i = 0; i < this.sceneObjectsToCleanUp.Count; i++)
            {
                Object.Destroy(this.sceneObjectsToCleanUp[i].extraAsset);
                Object.Destroy(this.sceneObjectsToCleanUp[i].levelAsset);
                Object.Destroy(this.sceneObjectsToCleanUp[i]);
            }
            if (!this.returnToEditor)
            {
                Object.Destroy(base.gameObject);
                return;
            }
            this.GoToEditor();
        }
        public void Win(string text = "Congratulation! You won!")
        {

```

```

Singleton<MusicManager>.Instance.StopMidi();
Singleton<MusicManager>.Instance.StopFile();
AudioListener.pause = true;
Time.timeScale = 0f;
Singleton<CoreGameManager>.Instance.disablePause = true;
this.winText.text = text;
this.winScreen.gameObject.SetActive(true);
}
public void GoToEditor()
{
Singleton<MusicManager>.Instance.StopMidi();
LevelStudioPlugin.Instance.StartCoroutine(LevelStudioPlugin.Instance.LoadEditorScene(this.editorModeToLoad, (this.editorLevelToLoad == null) ? null :
Path.Combine(LevelStudioPlugin.levelFilePath, this.editorLevelToLoad + ".ebpl"), this.editorLevelToLoad));
Object.Destroy(base.gameObject);
}
private void Update()
{
if (this.waitForCreation && Singleton<BaseGameManager>.Instance != null)
{
this.waitForCreation = false;
if (Singleton<BaseGameManager>.Instance is IStudioLegacyKnowledgable)
{
((IStudioLegacyKnowledgable)Singleton<BaseGameManager>.Instance).legacyFlags |= this.customContent.legacyFlags;
}
}
}
public static void LoadLevel(PlayableEditorLevel level, int lives, bool returnToEditor, string levelToLoad = null, string modeToLoad = "full")
{
if (Singleton<HighScoreManager>.Instance != null)
{
Singleton<HighScoreManager>.Instance.currentLevelId = string.Empty;
}
EditorPlayModeManager editorPlayModeManager =
Object.Instantiate<EditorPlayModeManager>(LevelStudioPlugin.Instance.assetMan.Get<EditorPlayModeManager>("playModeManager"));
editorPlayModeManager.waitForCreation = true;
editorPlayModeManager.customContent = new EditorCustomContent(level.meta.contentPackage);
SceneObject sceneObject = LevelImporter.CreateSceneObject(level.data);
sceneObject.manager = LevelStudioPlugin.Instance.gameModeAliases[level.meta.gameMode].prefab;
GameLoader gameLoader = Object.Instantiate<GameLoader>(LevelStudioPlugin.Instance.assetMan.Get<GameLoader>("gameLoaderPrefab"));
ElevatorScreen elevatorScreen = Object.Instantiate<ElevatorScreen>(LevelStudioPlugin.Instance.assetMan.Get<ElevatorScreen>("elevatorScreenPrefab"));
editorPlayModeManager.returnToEditor = returnToEditor;
if (level.meta.modeSettings != null)
{
BaseGameManager baseGameManager =
Object.Instantiate<BaseGameManager>(LevelStudioPlugin.Instance.gameModeAliases[level.meta.gameMode].prefab,
MTM101BaldiDevAPI.prefabTransform);
baseGameManager.name = baseGameManager.name.Replace("(Clone)", "_Customized");
level.meta.modeSettings.ApplySettingsToManager(baseGameManager);
sceneObject.manager = baseGameManager;
editorPlayModeManager.customContent.gameManagerPre = baseGameManager;
}
editorPlayModeManager.sceneObjectsToCleanUp.Add(sceneObject);
editorPlayModeManager.editorLevelToLoad = levelToLoad;
editorPlayModeManager.editorModeToLoad = modeToLoad;
gameLoader.AssignElevatorScreen(elevatorScreen);
gameLoader.Initialize(lives);
gameLoader.SetMode(0);
gameLoader.LoadLevel(sceneObject);
elevatorScreen.Initialize();
gameLoader.SetSave(false);
}
public EditorCustomContent customContent;
public ChallengeWin winScreen;
public TextMeshProUGUI winText;
public List<SceneObject> sceneObjectsToCleanUp = new List<SceneObject>();
public bool returnToEditor;
public string editorLevelToLoad;
public string editorModeToLoad;
public bool waitForCreation;
}
}
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using PlusStudioLevelFormat;
using PlusStudioLevelLoader;
using UnityEngine;
namespace PlusLevelStudio.Editor

```



```

{
public class EditorLevelData : IStudioLegacyKnowledgable
{
public EditorRoom hall
{
get
{
return this.rooms[0];
}
}
public Vector3 PracticalSpawnPoint
{
get
{
if ((from x in this.exits
where x.isSpawn
select x).Count<ExitLocation>() == 0)
{
return this.spawnPoint;
}
ExitLocation exitLocation = this.exits.Last((ExitLocation x) => x.isSpawn);
return new Vector3((float)exitLocation.position.x * 10f + 5f, 5f, (float)exitLocation.position.z * 10f + 5f);
}
}
public Direction PracticalSpawnDirection
{
get
{
if ((from x in this.exits
where x.isSpawn
select x).Count<ExitLocation>() == 0)
{
return this.spawnDirection;
}
return this.exits.Last((ExitLocation x) => x.isSpawn).direction;
}
}
public StudioLevelLegacyFlags legacyFlags { get; set; }
public void RedefineDefaultTextures()
{
this.defaultTextures.Clear();
foreach (KeyValuePair<string, TextureContainer> keyValuePair in LevelStudioPlugin.Instance.defaultRoomTextures)
{
this.defaultTextures.Add(keyValuePair.Key, new TextureContainer(keyValuePair.Value));
}
}
public IntVector2[] GetCellsOwnedByRoom(EditorRoom room)
{
CellArea[] array = (from x in this.areas
where x.roomId == this.IdFromRoom(room)
select x).ToArray<CellArea>();
List<IntVector2> list = new List<IntVector2>();
foreach (CellArea cellArea in array)
{
list.AddRange(cellArea.CalculateOwnedCells());
}
return list.ToArray();
}
public EditorRoom CreateRoomWithDefaultSettings(string type)
{
return new EditorRoom(type, this.defaultTextures[type]);
}
public void RemoveUnusedRoom(ushort idToCheck)
{
if (idToCheck == 1)
{
return;
}
for (int i = 0; i < this.areas.Count; i++)
{
if (this.areas[i].roomId == idToCheck)
{
return;
}
}
this.RemoveRoom(this.rooms[(int)(idToCheck - 1)]);
}
public EditorRoom RoomFromId(ushort roomId)
{

```

```

if (roomId == 0)
{
return null;
}
return this.rooms[(int)(roomId - 1)];
}
public ushort IdFromRoom(EditorRoom room)
{
if (room == null)
{
return 0;
}
return (ushort)(this.rooms.IndexOf(room) + 1);
}
public void RemoveRoom(EditorRoom toRemove)
{
Singleton<EditorController>.Instance.CleanupVisualsForRoom(toRemove);
Dictionary<EditorRoom, ushort> dictionary = new Dictionary<EditorRoom, ushort>();
Dictionary<EditorRoom, ushort> dictionary2 = new Dictionary<EditorRoom, ushort>();
foreach (EditorRoom editorRoom in this.rooms)
{
dictionary.Add(editorRoom, (ushort)(this.rooms.IndexOf(editorRoom) + 1));
}
this.rooms.Remove(toRemove);
foreach (EditorRoom editorRoom2 in this.rooms)
{
dictionary2.Add(editorRoom2, (ushort)(this.rooms.IndexOf(editorRoom2) + 1));
}
Dictionary<ushort, ushort> dictionary3 = new Dictionary<ushort, ushort>();
foreach (KeyValuePair<EditorRoom, ushort> keyValuePair in dictionary2)
{
dictionary3.Add(dictionary[keyValuePair.Key], keyValuePair.Value);
}
for (int i = 0; i < this.areas.Count; i++)
{
this.areas[i].roomId = dictionary3[this.areas[i].roomId];
}
this.UpdateCells(true);
}
public bool ValidatePlacementsFor<T>(List<T> verifyables, bool updateVisuals) where T : IEditorPositionVerifiable
{
bool result = false;
for (int i = verifyables.Count - 1; i >= 0; i--)
{
T t = verifyables[i];
if (!t.ValidatePosition(this))
{
if (updateVisuals)
{
Singleton<EditorController>.Instance.RemoveVisual(((IEditorVisualizable)(object)verifyables[i]));
}
verifyables.RemoveAt(i);
result = true;
}
}
return result;
}
public bool ValidatePlacements(bool updateVisuals)
{
return false | this.ValidatePlacementsFor<ExitLocation>(this.exits, updateVisuals) | this.ValidatePlacementsFor<LightPlacement>(this.lights, updateVisuals)
| this.ValidatePlacementsFor<DoorLocation>(this.doors, updateVisuals) | this.ValidatePlacementsFor<WindowLocation>(this.windows, updateVisuals) |
this.ValidatePlacementsFor<StructureLocation>(this.structures, updateVisuals) | this.ValidatePlacementsFor<PosterPlacement>(this.posters,
updateVisuals) | this.ValidatePlacementsFor<WallLocation>(this.walls, updateVisuals) | this.ValidatePlacementsFor<BasicObjectLocation>(this.objects,
updateVisuals) | this.ValidatePlacementsFor<ItemPlacement>(this.items, updateVisuals) |
this.ValidatePlacementsFor<ItemSpawnPlacement>(this.itemSpawns, updateVisuals) | this.ValidatePlacementsFor<MarkerLocation>(this.markers,
updateVisuals) | this.ValidatePlacementsFor<TileBasedObjectPlacement>(this.tileBasedObjects, updateVisuals) |
this.ValidatePlacementsFor<NPCPlacement>(this.npcs, updateVisuals) | this.ValidatePlacementsFor<PremadeRoomLocation>(this.premadeRooms, false);
}
public void ValidateActivityInRoom(EditorRoom room)
{
if (room.activity == null)
{
return;
}
room.activity.SetupDeleteIfInvalid();
}
public void RemoveObjectsInArea(CellArea area)
{
List<IntVector2> list = area.CalculateOwnedCells().ToList<IntVector2>();

```

```

for (int i = this.objects.Count - 1; i >= 0; i--)
{
    IntVector2 item = new IntVector2(Mathf.RoundToInt((this.objects[i].position.x - 5f) / 10f), Mathf.RoundToInt((this.objects[i].position.z - 5f) / 10f));
    if (list.Contains(item))
    {
        this.objects[i].OnDelete(this);
    }
}
for (int j = this.items.Count - 1; j >= 0; j--)
{
    IntVector2 item2 = new IntVector2(Mathf.RoundToInt((this.items[j].position.x - 5f) / 10f), Mathf.RoundToInt((this.items[j].position.y - 5f) / 10f));
    if (list.Contains(item2))
    {
        this.items[j].OnDelete(this);
    }
}
for (int k = this.itemSpawns.Count - 1; k >= 0; k--)
{
    IntVector2 item3 = new IntVector2(Mathf.RoundToInt((this.itemSpawns[k].position.x - 5f) / 10f), Mathf.RoundToInt((this.itemSpawns[k].position.y - 5f) / 10f));
    if (list.Contains(item3))
    {
        this.itemSpawns[k].OnDelete(this);
    }
}
for (int l = this.npcs.Count - 1; l >= 0; l--)
{
    if (list.Contains(this.npcs[l].position))
    {
        this.npcs[l].OnDelete(this);
    }
}
protected void ApplyCellModifiers(IEnumerable<IEditorCellModifier> modifiers, bool forEditor)
{
    foreach (IEditorCellModifier editorCellModifier in modifiers)
    {
        editorCellModifier.ModifyCells(this, forEditor);
    }
}
public void UpdateCells(bool forEditor)
{
    List<Direction> list = Directions.All();
    for (int i = 0; i < this.mapSize.x; i++)
    {
        for (int j = 0; j < this.mapSize.z; j++)
        {
            this.cells[i, j].roomId = this.RoomIdFromPos(this.cells[i, j].position.ToInt(), forEditor);
            this.cells[i, j].walls = new Nybble(0);
        }
    }
    for (int k = 0; k < this.mapSize.x; k++)
    {
        for (int l = 0; l < this.mapSize.z; l++)
        {
            if (this.cells[k, l].roomId != 0)
            {
                for (int m = 0; m < list.Count; m++)
                {
                    Direction direction = list[m];
                    IntVector2 intVector = direction.ToIntVector2();
                    PlusStudioLevelFormat.Cell cellSafe = this.GetCellSafe((int)this.cells[k, l].position.x + intVector.x, (int)this.cells[k, l].position.y + intVector.z);
                    if (cellSafe == null || cellSafe.roomId != this.cells[k, l].roomId)
                    {
                        this.cells[k, l].walls |= (Nybble)((byte)(1 << direction.BitPosition()));
                    }
                }
            }
        }
    }
    this.ApplyCellModifiers(this.doors, forEditor);
    this.ApplyCellModifiers(this.windows, forEditor);
    this.ApplyCellModifiers(this.exits, forEditor);
    this.ApplyCellModifiers(this.walls, forEditor);
    this.ApplyCellModifiers(this.structures, forEditor);
    this.ApplyCellModifiers(this.premadeRooms, forEditor);
    this.ValidatePlacements(forEditor);
}
public bool ResizeLevel(IntVector2 posDif, IntVector2 sizeDif, EditorController toAddUndo = null)
{

```

```

IntVector2 intVector = this.mapSize + sizeDif;
for (int i = 0; i < this.areas.Count; i++)
{
    IntVector2[] array = this.areas[i].CalculateOwnedCells();
    for (int j = 0; j < array.Length; j++)
    {
        IntVector2 intVector2 = array[j] - posDif;
        if (intVector2.x < 0 || intVector2.z < 0 || intVector2.x >= intVector.x || intVector2.z >= intVector.z)
        {
            return false;
        }
    }
}
if (toAddUndo != null)
{
    toAddUndo.AddUndo();
}
for (int k = 0; k < this.areas.Count; k++)
{
    this.areas[k].origin = this.areas[k].origin - posDif;
}
IntVector2 intVector3 = this.mapSize;
PlusStudioLevelFormat.Cell[,] array2 = this.cells;
this.mapSize = intVector;
this.cells = new PlusStudioLevelFormat.Cell[this.mapSize.x, this.mapSize.z];
for (int l = 0; l < this.mapSize.x; l++)
{
    for (int m = 0; m < this.mapSize.z; m++)
    {
        this.cells[l, m] = new PlusStudioLevelFormat.Cell(new ByteVector2(l, m));
        if (l + posDif.x >= 0 && l + posDif.x < intVector3.x && m + posDif.z >= 0 && m + posDif.z < intVector3.z)
        {
            PlusStudioLevelFormat.Cell cell = array2[l + posDif.x, m + posDif.z];
            this.cells[l, m].walls = cell.walls;
            this.cells[l, m].roomId = cell.roomId;
        }
    }
}
for (int n = 0; n < this.lights.Count; n++)
{
    this.lights[n].position -= posDif;
}
for (int num = 0; num < this.doors.Count; num++)
{
    this.doors[num].position -= posDif;
}
for (int num2 = 0; num2 < this.windows.Count; num2++)
{
    this.windows[num2].position -= posDif;
}
for (int num3 = 0; num3 < this.exits.Count; num3++)
{
    this.exits[num3].position -= posDif;
}
for (int num4 = 0; num4 < this.items.Count; num4++)
{
    this.items[num4].position -= new Vector2((float)posDif.x * 10f, (float)posDif.z * 10f);
}
for (int num5 = 0; num5 < this.itemSpawns.Count; num5++)
{
    this.itemSpawns[num5].position -= new Vector2((float)posDif.x * 10f, (float)posDif.z * 10f);
}
for (int num6 = 0; num6 < this.objects.Count; num6++)
{
    this.objects[num6].position -= new Vector3((float)posDif.x * 10f, 0f, (float)posDif.z * 10f);
}
for (int num7 = 0; num7 < this.rooms.Count; num7++)
{
    if (this.rooms[num7].activity != null)
    {
        this.rooms[num7].activity.position -= new Vector3((float)posDif.x * 10f, 0f, (float)posDif.z * 10f);
    }
}
for (int num8 = 0; num8 < this.structures.Count; num8++)
{
    this.structures[num8].ShiftBy(new Vector3((float)posDif.x * 10f, 0f, (float)posDif.z * 10f), posDif, sizeDif);
}
for (int num9 = 0; num9 < this.npcs.Count; num9++)
{

```

```

this.npcs[num9].position -= posDif;
}
for (int num10 = 0; num10 < this.posters.Count; num10++)
{
this.posters[num10].position -= posDif;
}
for (int num11 = 0; num11 < this.walls.Count; num11++)
{
this.walls[num11].position -= posDif;
}
for (int num12 = 0; num12 < this.markers.Count; num12++)
{
this.markers[num12].ShiftBy(new Vector3((float)posDif.x * 10f, 0f, (float)posDif.z * 10f), posDif, sizeDif);
}
for (int num13 = 0; num13 < this.premadeRooms.Count; num13++)
{
this.premadeRooms[num13].position -= posDif;
}
this.spawnPoint = new Vector3((float)posDif.x * 10f, 0f, (float)posDif.z * 10f);
this.ValidatePlacements(true);
return true;
}
public EditorLevelData(IntVector2 mapSize)
{
this.legacyFlags = StudioLevelLegacyFlags.None;
this.mapSize = mapSize;
this.cells = new PlusStudioLevelFormat.Cell[mapSize.x, mapSize.z];
for (int i = 0; i < mapSize.x; i++)
{
for (int j = 0; j < mapSize.z; j++)
{
this.cells[i, j] = new PlusStudioLevelFormat.Cell(new ByteVector2(i, j));
}
}
this.RedefineDefaultTextures();
this.rooms.Add(this.CreateRoomWithDefaultSettings("hall"));
for (int k = 0; k < PlayableEditorLevel.defaultStickers.Length; k++)
{
this.potentialStickers.Add(new WeightedID
{
id = PlayableEditorLevel.defaultStickers[k],
weight = 100
});
}
}
public bool AreaValid(CellArea area)
{
for (int i = 0; i < this.areas.Count; i++)
{
if (this.areas[i] != area && this.areas[i].CollidesWith(area))
{
return false;
}
}
IntVector2[] array = area.CalculateOwnedCells();
for (int j = 0; j < array.Length; j++)
{
if (this.GetCellSafe(array[j].x, array[j].z) == null)
{
return false;
}
}
return true;
}
public PlusStudioLevelFormat.Cell GetCellSafe(int x, int y)
{
if (x < 0)
{
return null;
}
if (x >= this.mapSize.x)
{
return null;
}
if (y < 0)
{
return null;
}
if (y >= this.mapSize.z)

```

```

{
    return null;
}
return this.cells[x, y];
}
public bool GetSmartDoorPosition(IntVector2 pos, Direction dir, out IntVector2 outPos, out Direction outDir)
{
    outPos = pos;
    outDir = dir;
    PlusStudioLevelFormat.Cell cellSafe = this.GetCellSafe(pos);
    if (cellSafe == null)
    {
        return false;
    }
    EditorRoom editorRoom = this.RoomFromId(cellSafe.roomId);
    if (editorRoom == null)
    {
        return false;
    }
    if (editorRoom.roomType != "hall")
    {
        return true;
    }
    PlusStudioLevelFormat.Cell cellSafe2 = this.GetCellSafe(pos + dir.ToIntVector2());
    if (cellSafe2 != null)
    {
        EditorRoom editorRoom2 = this.RoomFromId(cellSafe2.roomId);
        if (editorRoom2 != null && editorRoom2.roomType != "hall")
        {
            outPos = pos + dir.ToIntVector2();
            outDir = dir.GetOpposite();
            return true;
        }
    }
    return false;
}
public PlusStudioLevelFormat.Cell GetCellSafe(IntVector2 vec2)
{
    return this.GetCellSafe(vec2.x, vec2.z);
}
public ushort RoomIdFromPos(IntVector2 vector, bool forEditor)
{
    CellArea cellArea = this.AreaFromPos(vector, forEditor);
    if (cellArea == null)
    {
        return 0;
    }
    return cellArea.roomId;
}
public ushort RoomIdFromPosIncludeExisting(IntVector2 vector)
{
    CellArea cellArea = this.AreaFromPos(vector, true);
    if (cellArea != null)
    {
        return cellArea.roomId;
    }
    PlusStudioLevelFormat.Cell cellSafe = this.GetCellSafe(vector);
    if (cellSafe == null)
    {
        return 0;
    }
    return cellSafe.roomId;
}
public EditorRoom RoomFromPos(IntVector2 vector, bool forEditor)
{
    ushort num = this.RoomIdFromPos(vector, forEditor);
    if (num == 0)
    {
        return null;
    }
    return this.rooms[(int)(num - 1)];
}
public CellArea AreaFromPos(IntVector2 vector, bool forEditor)
{
    foreach (CellArea cellArea in this.areas)
    {
        if ((!cellArea.editorOnly || forEditor) && cellArea.VectorIsInArea(vector))
        {
            return cellArea;
        }
    }
}

```

```

    }
}
return null;
}
public void FinalizeCompile(BaldiLevel toFinalize)
{
    for (int i = 0; i < this.mapSize.x; i++)
    {
        for (int j = 0; j < this.mapSize.z; j++)
        {
            if (toFinalize.cells[i, j].roomId != 0)
            {
                if (toFinalize.rooms[(int)(toFinalize.cells[i, j].roomId - 1)].type == "mystery")
                {
                    toFinalize.secretCells[i, j] = true;
                    toFinalize.entitySafeCells[i, j] = false;
                    toFinalize.eventSafeCells[i, j] = false;
                    toFinalize.coverage[i, j] = (PlusCellCoverage.North | PlusCellCoverage.East | PlusCellCoverage.South | PlusCellCoverage.West | PlusCellCoverage.Up |
                    PlusCellCoverage.Down | PlusCellCoverage.Center);
                }
                if (toFinalize.rooms[(int)(toFinalize.cells[i, j].roomId - 1)].type == "eventspawner")
                {
                    toFinalize.excludedFromRoomGroupCells[i, j] = true;
                }
            }
        }
    }
}
public BaldiLevel Compile()
{
    BaldiLevel baldiLevel = new BaldiLevel(this.mapSize.ToByte());
    baldiLevel.levelTitle = this.elevatorTitle;
    baldiLevel.timeLimit = this.timeLimit;
    baldiLevel.skybox = this.skybox;
    baldiLevel.spawnPoint = this.spawnPoint.ToData();
    baldiLevel.spawnDirection = (PlusDirection)this.spawnDirection;
    baldiLevel.randomEvents = new List<string>(this.randomEvents);
    baldiLevel.minLightColor = this.minLightColor.ToData();
    baldiLevel.lightMode = (PlusLightMode)this.lightMode;
    baldiLevel.initialRandomEventGap = this.initialRandomEventGap;
    baldiLevel.minRandomEventGap = this.minRandomEventGap;
    baldiLevel.maxRandomEventGap = this.maxRandomEventGap;
    baldiLevel.seed = this.seed;
    baldiLevel.usesMap = this.usesMap;
    this.UpdateCells(false);
    for (int i = 0; i < this.mapSize.x; i++)
    {
        for (int j = 0; j < this.mapSize.z; j++)
        {
            baldiLevel.cells[i, j] = new PlusStudioLevelFormat.Cell(this.cells[i, j]);
        }
    }
    for (int k = 0; k < this.rooms.Count; k++)
    {
        RoomInfo roomInfo = new RoomInfo(this.rooms[k].roomType, new TextureContainer(this.rooms[k].textureContainer));
        roomInfo.name = this.rooms[k].name;
        if (this.rooms[k].activity != null)
        {
            roomInfo.activity = new ActivityInfo
            {
                type = this.rooms[k].activity.type,
                position = this.rooms[k].activity.position.ToData(),
                direction = (PlusDirection)this.rooms[k].activity.direction
            };
        }
        baldiLevel.rooms.Add(roomInfo);
    }
    for (int l = 0; l < this.lights.Count; l++)
    {
        LightGroup lightGroup = this.lightGroups[(int)this.lights[l].lightGroup];
        baldiLevel.lights.Add(new LightInfo
        {
            color = lightGroup.color.ToData(),
            position = this.lights[l].position.ToByte(),
            prefab = this.lights[l].type,
            strength = lightGroup.strength
        });
    }
    for (int m = 0; m < this.premadeRooms.Count; m++)

```

```

{
    baldiLevel.tileObjects.Add(new TileObjectInfo
    {
        prefab = LevelStudioPlugin.Instance.premadeRoomDoors[this.premadeRooms[m].room],
        position = this.premadeRooms[m].GetDoorPos().ToByte(),
        direction = (PlusDirection)this.premadeRooms[m].direction.GetOpposite()
    });
    this.premadeRooms[m].direction.GetOpposite().ToIntVector2();
}
for (int n = 0; n < this.doors.Count; n++)
{
    string prefab = this.doors[n].type;
    IntVector2 intVector;
    Direction opposite;
    bool smartDoorPosition = this.GetSmartDoorPosition(this.doors[n].position, this.doors[n].direction, out intVector, out opposite);
    bool flag = false;
    DoorIngameStatus doorIngameStatus = LevelStudioPlugin.Instance.doorIngameStatus[this.doors[n].type];
    switch (doorIngameStatus)
    {
        case DoorIngameStatus.AlwaysDoor:
        case DoorIngameStatus.AlwaysDoorNoSmart:
            flag = false;
            break;
        case DoorIngameStatus.AlwaysObject:
            flag = true;
            break;
        case DoorIngameStatus.Smart:
            flag = !smartDoorPosition;
            break;
    }
    if (this.doors[n].type == "standard")
    {
        if (this.RoomFromPos(intVector, false).roomType == "mystery")
        {
            prefab = "mysterydoor";
            flag = false;
        }
        else if (this.RoomFromPos(this.doors[n].position + this.doors[n].direction.ToIntVector2(), false).roomType == "mystery")
        {
            intVector = this.doors[n].position + this.doors[n].direction.ToIntVector2();
            opposite = this.doors[n].direction.GetOpposite();
            prefab = "mysterydoor";
            flag = false;
        }
    }
    if (flag)
    {
        baldiLevel.tileObjects.Add(new TileObjectInfo
        {
            prefab = prefab,
            position = this.doors[n].position.ToByte(),
            direction = (PlusDirection)this.doors[n].direction
        });
    }
    else if (doorIngameStatus == DoorIngameStatus.AlwaysDoorNoSmart)
    {
        baldiLevel.doors.Add(new DoorInfo
        {
            prefab = prefab,
            position = this.doors[n].position.ToByte(),
            direction = (PlusDirection)this.doors[n].direction,
            roomId = this.GetCellSafe(this.doors[n].position.x, this.doors[n].position.z).roomId
        });
    }
    else
    {
        baldiLevel.doors.Add(new DoorInfo
        {
            prefab = prefab,
            position = intVector.ToByte(),
            direction = (PlusDirection)opposite,
            roomId = this.GetCellSafe(intVector.x, intVector.z).roomId
        });
    }
}
for (int num = 0; num < this.windows.Count; num++)
{
    baldiLevel.windows.Add(new WindowInfo
    {

```



```

prefab = this.windows[num].type,
position = this.windows[num].position.ToByte(),
direction = (PlusDirection)this.windows[num].direction
});
}
for (int num2 = 0; num2 < this.exits.Count; num2++)
{
baldiLevel.exits.Add(new ExitInfo
{
type = this.exits[num2].type,
position = this.exits[num2].position.ToByte(),
direction = (PlusDirection)this.exits[num2].direction,
isSpawn = this.exits[num2].isSpawn
});
}
for (int num3 = 0; num3 < this.items.Count; num3++)
{
ushort num4 = (ushort)Mathf.Max((int)this.GetCellSafe(Mathf.RoundToInt((this.items[num3].position.x - 5f) / 10f),
Mathf.RoundToInt((this.items[num3].position.y - 5f) / 10f)).roomId, 1);
baldiLevel.rooms[(int)(num4 - 1)].items.Add(new ItemInfo
{
item = this.items[num3].item,
position = this.items[num3].position.ToData()
});
}
for (int num5 = 0; num5 < this.itemSpawns.Count; num5++)
{
ushort num6 = (ushort)Mathf.Max((int)this.GetCellSafe(Mathf.RoundToInt((this.itemSpawns[num5].position.x - 5f) / 10f),
Mathf.RoundToInt((this.itemSpawns[num5].position.y - 5f) / 10f)).roomId, 1);
baldiLevel.rooms[(int)(num6 - 1)].itemSpawns.Add(new ItemSpawnInfo
{
weight = this.itemSpawns[num5].weight,
position = this.itemSpawns[num5].position.ToData()
});
}
for (int num7 = 0; num7 < this.objects.Count; num7++)
{
ushort num8 = (ushort)Mathf.Max((int)this.GetCellSafe(Mathf.RoundToInt((this.objects[num7].position.x - 5f) / 10f),
Mathf.RoundToInt((this.objects[num7].position.z - 5f) / 10f)).roomId, 1);
baldiLevel.rooms[(int)(num8 - 1)].basicObjects.Add(new BasicObjectInfo
{
prefab = this.objects[num7].prefab,
position = this.objects[num7].position.ToData(),
rotation = this.objects[num7].rotation.ToData()
});
}
for (int num9 = 0; num9 < this.structures.Count; num9++)
{
if (this.structures[num9] is RandomStructureLocation)
{
baldiLevel.randomStructures.Add(((RandomStructureLocation)this.structures[num9]).CompileIntoRandom(this, baldiLevel));
}
else
{
baldiLevel.structures.Add(this.structures[num9].Compile(this, baldiLevel));
}
}
for (int num10 = 0; num10 < this.npcs.Count; num10++)
{
baldiLevel.npcs.Add(new NPCInfo
{
npc = this.npcs[num10].npc,
position = this.npcs[num10].position.ToByte()
});
}
for (int num11 = 0; num11 < this.posters.Count; num11++)
{
baldiLevel.posters.Add(new PosterInfo
{
poster = this.posters[num11].type,
direction = (PlusDirection)this.posters[num11].direction,
position = this.posters[num11].position.ToByte()
});
}
for (int num12 = 0; num12 < this.tileBasedObjects.Count; num12++)
{
baldiLevel.tileObjects.Add(new TileObjectInfo
{
prefab = this.tileBasedObjects[num12].type,

```

```

direction = (PlusDirection)this.tileBasedObjects[num12].direction,
position = this.tileBasedObjects[num12].position.ToByte()
});
}
for (int num13 = 0; num13 < this.potentialStickers.Count; num13++)
{
    baldiLevel.potentialStickers.Add(new WeightedID
    {
        weight = this.potentialStickers[num13].weight,
        id = this.potentialStickers[num13].id
    });
}
for (int num14 = 0; num14 < this.premadeRooms.Count; num14++)
{
    baldiLevel.premadeRoomPlacements.Add(new RoomPlacementInfo
    {
        room = this.premadeRooms[num14].room,
        position = this.premadeRooms[num14].position.ToByte(),
        direction = (PlusDirection)this.premadeRooms[num14].direction,
        doorSpawnId = this.premadeRooms[num14].doorId,
        textureOverride = null
    });
}
baldiLevel.storeItemCount = this.storeItemCount;
for (int num15 = 0; num15 < this.potentialStoreItems.Count; num15++)
{
    baldiLevel.potentialStoreItems.Add(new WeightedID
    {
        weight = this.potentialStoreItems[num15].weight,
        id = this.potentialStoreItems[num15].id
    });
}
if (baldiLevel.potentialStoreItems.Count == 0)
{
    baldiLevel.storeItemCount = 0;
}
return baldiLevel;
}
public bool WallFree(IntVector2 pos, Direction dir, bool ignoreSelf)
{
    return this.WallFree(pos, dir, ignoreSelf, false);
}
public bool WallFree(IntVector2 pos, Direction dir, bool ignoreSelf, bool includeUnbreakableWindows)
{
    PlusStudioLevelFormat.Cell cellSafe = this.GetCellSafe(pos);
    if (cellSafe == null)
    {
        return false;
    }
    if ((cellSafe.walls & dir.ToBinary()) == 0)
    {
        if (!includeUnbreakableWindows)
        {
            return false;
        }
        bool flag = false;
        foreach (WindowLocation windowLocation in this.windows)
        {
            if (!LevelLoaderPlugin.Instance.windowObjects[windowLocation.type].breakable)
            {
                if (windowLocation.position == pos && windowLocation.direction == dir)
                {
                    flag = true;
                    break;
                }
                if (windowLocation.position == pos + dir.ToIntVector2() && windowLocation.direction == dir.GetOpposite())
                {
                    flag = true;
                    break;
                }
            }
        }
        if (!flag)
        {
            return false;
        }
    }
    int num = 0;
    for (int i = 0; i < this.structures.Count; i++)

```

```

{
    if (this.structures[i].OccupiesWall(pos, dir))
    {
        num++;
    }
}
for (int j = 0; j < this.posters.Count; j++)
{
    if (this.posters[j].OccupiesWall(pos, dir))
    {
        num++;
    }
}
if (ignoreSelf)
{
    num--;
}
return num <= 0;
}
public void Write(BinaryWriter writer)
{
    writer.Write(18);
    StringCompressor stringComp = new StringCompressor();
    stringComp.AddStrings(from x in this.lights
    select x.type);
    stringComp.AddStrings(from x in this.doors
    select x.type);
    stringComp.AddStrings(from x in this.exits
    select x.type);
    stringComp.AddStrings(from x in this.windows
    select x.type);
    stringComp.AddStrings(from x in this.items
    select x.item);
    stringComp.AddStrings(from x in this.objects
    select x.prefab);
    stringComp.AddStrings(from x in this.npcs
    select x.npc);
    stringComp.AddStrings(from x in this.tileBasedObjects
    select x.type);
    stringComp.AddStrings(from x in this.posters
    select x.type);
    stringComp.AddStrings(from x in this.rooms
    select x.roomType);
    stringComp.AddStrings(from x in this.rooms
    select x.textureContainer.floor);
    stringComp.AddStrings(from x in this.rooms
    select x.textureContainer.wall);
    stringComp.AddStrings(from x in this.rooms
    select x.textureContainer.ceiling);
    stringComp.AddStrings(from x in this.rooms
    where x.activity != null
    select x.activity.type);
    stringComp.AddStrings(from x in this.structures
    select x.type);
    stringComp.AddStrings(from x in this.markers
    select x.type);
    stringComp.AddStrings(from x in this.premadeRooms
    select x.room);
    this.structures.ForEach(delegate(StructureLocation x)
    {
        x.AddStringsToCompressor(stringComp);
    });
    this.markers.ForEach(delegate(MarkerLocation x)
    {
        x.AddStringsToCompressor(stringComp);
    });
    stringComp.AddString("null");
    stringComp.FinalizeDatabase();
    stringComp.WriteStringDatabase(writer);
    writer.Write((byte)this.mapSize.x);
    writer.Write((byte)this.mapSize.z);
    writer.Write(this.areas.Count);
    for (int i = 0; i < this.areas.Count; i++)
    {
        this.areas[i].Write(writer);
    }
    writer.Write(this.rooms.Count);
    for (int j = 0; j < this.rooms.Count; j++)
    {

```

```

stringComp.WriteStoredString(writer, this.rooms[j].roomType);
writer.Write(this.rooms[j].name);
stringComp.WriteStoredString(writer, this.rooms[j].textureContainer.floor);
stringComp.WriteStoredString(writer, this.rooms[j].textureContainer.wall);
stringComp.WriteStoredString(writer, this.rooms[j].textureContainer.ceiling);
if (this.rooms[j].activity == null)
{
    stringComp.WriteStoredString(writer, "null");
}
else
{
    stringComp.WriteStoredString(writer, this.rooms[j].activity.type);
    writer.Write(this.rooms[j].activity.position.ToData());
    writer.Write((byte)this.rooms[j].activity.direction);
}
}
writer.Write((ushort)this.lightGroups.Count);
for (int k = 0; k < this.lightGroups.Count; k++)
{
    writer.Write(this.lightGroups[k].color.ToData());
    writer.Write(this.lightGroups[k].strength);
}
writer.Write(this.lights.Count);
for (int l = 0; l < this.lights.Count; l++)
{
    stringComp.WriteStoredString(writer, this.lights[l].type);
    writer.Write(this.lights[l].position.ToByte());
    writer.Write(this.lights[l].lightGroup);
}
writer.Write(this.doors.Count);
for (int m = 0; m < this.doors.Count; m++)
{
    stringComp.WriteStoredString(writer, this.doors[m].type);
    writer.Write(this.doors[m].position.ToByte());
    writer.Write((byte)this.doors[m].direction);
}
writer.Write(this.windows.Count);
for (int n = 0; n < this.windows.Count; n++)
{
    stringComp.WriteStoredString(writer, this.windows[n].type);
    writer.Write(this.windows[n].position.ToByte());
    writer.Write((byte)this.windows[n].direction);
}
writer.Write(this.exits.Count);
for (int num = 0; num < this.exits.Count; num++)
{
    stringComp.WriteStoredString(writer, this.exits[num].type);
    writer.Write(this.exits[num].position.ToByte());
    writer.Write((byte)this.exits[num].direction);
    writer.Write(this.exits[num].isSpawn);
}
writer.Write(this.items.Count);
for (int num2 = 0; num2 < this.items.Count; num2++)
{
    stringComp.WriteStoredString(writer, this.items[num2].item);
    writer.Write(this.items[num2].position.ToData());
}
writer.Write(this.itemSpawns.Count);
for (int num3 = 0; num3 < this.itemSpawns.Count; num3++)
{
    writer.Write(this.itemSpawns[num3].weight);
    writer.Write(this.itemSpawns[num3].position.ToData());
}
writer.Write(this.objects.Count);
for (int num4 = 0; num4 < this.objects.Count; num4++)
{
    stringComp.WriteStoredString(writer, this.objects[num4].prefab);
    writer.Write(this.objects[num4].position.ToData());
    writer.Write(this.objects[num4].rotation.ToData());
}
writer.Write(this.structures.Count);
for (int num5 = 0; num5 < this.structures.Count; num5++)
{
    stringComp.WriteStoredString(writer, this.structures[num5].type);
    this.structures[num5].Write(this, writer, stringComp);
}
writer.Write(this.npcs.Count);
for (int num6 = 0; num6 < this.npcs.Count; num6++)
{

```

```

stringComp.WriteStoredString(writer, this.npcs[num6].npc);
writer.Write(this.npcs[num6].position.ToByte());
if (this.npcs[num6].properties == null)
{
    writer.Write(true);
}
else
{
    bool flag = this.npcs[num6].properties.IsAtDefaultSettings();
    writer.Write(flag);
    if (!flag)
    {
        this.npcs[num6].properties.Write(writer);
    }
}
}
writer.Write(this.posters.Count);
for (int num7 = 0; num7 < this.posters.Count; num7++)
{
    stringComp.WriteStoredString(writer, this.posters[num7].type);
    writer.Write(this.posters[num7].position.ToByte());
    writer.Write((byte)this.posters[num7].direction);
}
WallLocation[] array = (from x in this.walls
    where x.wallState == WallState.AddWall
    select x).ToArray<WallLocation>();
writer.Write(array.Length);
for (int num8 = 0; num8 < array.Length; num8++)
{
    writer.Write(array[num8].position.ToByte());
    writer.Write((byte)array[num8].direction);
}
WallLocation[] array2 = (from x in this.walls
    where x.wallState == WallState.RemoveWall
    select x).ToArray<WallLocation>();
writer.Write(array2.Length);
for (int num9 = 0; num9 < array2.Length; num9++)
{
    writer.Write(array2[num9].position.ToByte());
    writer.Write((byte)array2[num9].direction);
}
WallLocation[] array3 = (from x in this.walls
    where x.wallState == WallState.Oneway
    select x).ToArray<WallLocation>();
writer.Write(array3.Length);
for (int num10 = 0; num10 < array3.Length; num10++)
{
    writer.Write(array3[num10].position.ToByte());
    writer.Write((byte)array3[num10].direction);
}
writer.Write(this.markers.Count);
for (int num11 = 0; num11 < this.markers.Count; num11++)
{
    stringComp.WriteStoredString(writer, this.markers[num11].type);
    this.markers[num11].Write(this, writer, stringComp);
}
writer.Write(this.tileBasedObjects.Count);
for (int num12 = 0; num12 < this.tileBasedObjects.Count; num12++)
{
    stringComp.WriteStoredString(writer, this.tileBasedObjects[num12].type);
    writer.Write(this.tileBasedObjects[num12].position.ToData());
    writer.Write((byte)this.tileBasedObjects[num12].direction);
}
writer.Write(this.spawnPoint.ToData());
writer.Write((byte)this.spawnDirection);
writer.Write(this.elevatorTitle);
writer.Write(this.timeLimit);
writer.Write(this.initialRandomEventGap);
writer.Write(this.minRandomEventGap);
writer.Write(this.maxRandomEventGap);
writer.Write(this.randomEvents.Count);
for (int num13 = 0; num13 < this.randomEvents.Count; num13++)
{
    writer.Write(this.randomEvents[num13]);
}
}
writer.Write(this.skybox);
writer.Write(this.minLightColor.ToData());
writer.Write((byte)this.lightMode);
writer.Write(this.seed);

```

```

writer.Write(this.potentialStickers.Count);
for (int num14 = 0; num14 < this.potentialStickers.Count; num14++)
{
    writer.Write(this.potentialStickers[num14].id);
    writer.Write(this.potentialStickers[num14].weight);
}
writer.Write(this.usesMap);
writer.Write(this.premadeRooms.Count);
for (int num15 = 0; num15 < this.premadeRooms.Count; num15++)
{
    stringComp.WriteStoredString(writer, this.premadeRooms[num15].room);
    writer.Write(this.premadeRooms[num15].position.ToByte());
    writer.Write((byte)this.premadeRooms[num15].direction);
    writer.Write(this.premadeRooms[num15].doorId);
}
writer.Write(this.storeItemCount);
writer.Write(this.potentialStoreItems.Count);
for (int num16 = 0; num16 < this.potentialStoreItems.Count; num16++)
{
    writer.Write(this.potentialStoreItems[num16].id);
    writer.Write(this.potentialStoreItems[num16].weight);
}
this.meta.Write(writer);
}
public static EditorLevelData ReadFrom(BinaryReader reader)
{
    byte b = reader.ReadByte();
    if (b > 18)
    {
        throw new Exception(string.Concat(new string[]
        {
            "Attempted to read file with newer version number than the one supported! (Got: ",
            b.ToString(),
            " expected: ",
            18.ToString(),
            " or below)"
        }));
    }
    StringCompressor stringCompressor = StringCompressor.ReadStringDatabase(reader);
    string text = "WIP";
    if (b == 2)
    {
        text = reader.ReadString();
    }
    EditorLevelData editorLevelData = new EditorLevelData(new IntVector2((int)reader.ReadByte(), (int)reader.ReadByte()));
    editorLevelData.elevatorTitle = text;
    editorLevelData.lightGroups.Clear();
    editorLevelData.rooms.Clear();
    if (b < 17)
    {
        editorLevelData.legacyFlags |= StudioLevelLegacyFlags.BeforeNPCCustom;
    }
    int num = reader.ReadInt32();
    for (int i = 0; i < num; i++)
    {
        reader.ReadString();
        editorLevelData.areas.Add(new RectCellArea(default(IntVector2), default(IntVector2), 0).ReadInto(reader));
    }
    int num2 = reader.ReadInt32();
    for (int j = 0; j < num2; j++)
    {
        EditorRoom editorRoom;
        if (b < 14)
        {
            editorRoom = new EditorRoom(stringCompressor.ReadStoredString(reader), new TextureContainer(stringCompressor.ReadStoredString(reader),
            stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader)));
        }
        else
        {
            editorRoom = new EditorRoom(stringCompressor.ReadStoredString(reader), reader.ReadString(), new
            TextureContainer(stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader), stringCompressor.ReadStoredString(reader)));
        }
        string text2 = stringCompressor.ReadStoredString(reader);
        if (text2 != "null")
        {
            new ActivityLocation
            {
                type = text2,
                position = reader.ReadUnityVector3().ToUnity(),
            }
        }
    }
}

```

```

direction = (Direction)reader.ReadByte()
}.Setup(editorRoom);
}
editorLevelData.rooms.Add(editorRoom);
}
ushort num3 = reader.ReadUInt16();
for (int k = 0; k < (int)num3; k++)
{
editorLevelData.lightGroups.Add(new LightGroup
{
color = reader.ReadUnityColor().ToStandard(),
strength = reader.ReadByte()
});
}
int num4 = reader.ReadInt32();
for (int l = 0; l < num4; l++)
{
editorLevelData.lights.Add(new LightPlacement
{
type = stringCompressor.ReadStoredString(reader),
position = reader.ReadByteVector2().ToInt(),
lightGroup = reader.ReadUInt16()
});
}
int num5 = reader.ReadInt32();
for (int m = 0; m < num5; m++)
{
editorLevelData.doors.Add(new DoorLocation
{
type = stringCompressor.ReadStoredString(reader),
position = reader.ReadByteVector2().ToInt(),
direction = (Direction)reader.ReadByte()
});
}
int num6 = reader.ReadInt32();
for (int n = 0; n < num6; n++)
{
editorLevelData.windows.Add(new WindowLocation
{
type = stringCompressor.ReadStoredString(reader),
position = reader.ReadByteVector2().ToInt(),
direction = (Direction)reader.ReadByte()
});
}
int num7 = reader.ReadInt32();
for (int num8 = 0; num8 < num7; num8++)
{
editorLevelData.exits.Add(new ExitLocation
{
type = stringCompressor.ReadStoredString(reader),
position = reader.ReadByteVector2().ToInt(),
direction = (Direction)reader.ReadByte(),
isSpawn = reader.ReadBoolean()
});
}
int num9 = reader.ReadInt32();
for (int num10 = 0; num10 < num9; num10++)
{
editorLevelData.items.Add(new ItemPlacement
{
item = stringCompressor.ReadStoredString(reader),
position = reader.ReadUnityVector2().ToUnity()
});
}
if (b >= 8)
{
int num11 = reader.ReadInt32();
for (int num12 = 0; num12 < num11; num12++)
{
editorLevelData.itemSpawns.Add(new ItemSpawnPlacement
{
weight = reader.ReadInt32(),
position = reader.ReadUnityVector2().ToUnity()
});
}
}
int num13 = reader.ReadInt32();
for (int num14 = 0; num14 < num13; num14++)
{

```

```

editorLevelData.objects.Add(new BasicObjectLocation
{
    prefab = stringCompressor.ReadStoredString(reader),
    position = reader.ReadUnityVector3().ToUnity(),
    rotation = reader.ReadUnityQuaternion().ToUnity()
});
}
int num15 = reader.ReadInt32();
for (int num16 = 0; num16 < num15; num16++)
{
    string type = stringCompressor.ReadStoredString(reader);
    StructureLocation structureLocation = LevelStudioPlugin.Instance.ConstructStructureOfType(type);
    structureLocation.ReadInto(editorLevelData, reader, stringCompressor);
    editorLevelData.structures.Add(structureLocation);
}
int num17 = reader.ReadInt32();
for (int num18 = 0; num18 < num17; num18++)
{
    NPCPlacement npcplacement = new NPCPlacement
    {
        npc = stringCompressor.ReadStoredString(reader),
        position = reader.ReadByteVector2().ToInt()
    };
    npcplacement.properties = LevelStudioPlugin.Instance.ConstructNPCPropertiesOfType(npcplacement.npc, null);
    if (b >= 17 && !reader.ReadBoolean())
    {
        npcplacement.properties.ReadInto(reader);
    }
    editorLevelData.npcs.Add(npcplacement);
}
int num19 = reader.ReadInt32();
for (int num20 = 0; num20 < num19; num20++)
{
    editorLevelData.posters.Add(new PosterPlacement
    {
        type = stringCompressor.ReadStoredString(reader),
        position = reader.ReadByteVector2().ToInt(),
        direction = (Direction)reader.ReadByte()
    });
}
if (b < 7)
{
    editorLevelData.meta = new PlayableLevelMeta
    {
        name = editorLevelData.elevatorTitle,
        modeSettings = LevelStudioPlugin.Instance.gameModeAliases["standard"].CreateSettings(),
        gameMode = "standard",
        contentPackage = new EditorCustomContentPackage(true)
    };
}
if (b == 0)
{
    editorLevelData.spawnPoint = reader.ReadUnityVector3().ToUnity();
    editorLevelData.spawnDirection = (Direction)reader.ReadByte();
    return editorLevelData;
}
int num21 = reader.ReadInt32();
for (int num22 = 0; num22 < num21; num22++)
{
    editorLevelData.walls.Add(new WallLocation
    {
        wallState = WallState.AddWall,
        position = reader.ReadByteVector2().ToInt(),
        direction = (Direction)reader.ReadByte()
    });
}
int num23 = reader.ReadInt32();
for (int num24 = 0; num24 < num23; num24++)
{
    editorLevelData.walls.Add(new WallLocation
    {
        wallState = WallState.RemoveWall,
        position = reader.ReadByteVector2().ToInt(),
        direction = (Direction)reader.ReadByte()
    });
}
if (b >= 18)
{
    int num25 = reader.ReadInt32();

```



```

for (int num26 = 0; num26 < num25; num26++)
{
    editorLevelData.walls.Add(new WallLocation
    {
        wallState = WallState.Oneway,
        position = reader.ReadByteVector2().ToInt(),
        direction = (Direction)reader.ReadByte()
    });
}
if (b >= 9)
{
    int num27 = reader.ReadInt32();
    for (int num28 = 0; num28 < num27; num28++)
    {
        string type2 = stringCompressor.ReadStoredString(reader);
        MarkerLocation markerLocation = LevelStudioPlugin.Instance.ConstructMarkerOfType(type2);
        markerLocation.ReadInto(editorLevelData, reader, stringCompressor);
        editorLevelData.markers.Add(markerLocation);
    }
}
if (b >= 11)
{
    int num29 = reader.ReadInt32();
    for (int num30 = 0; num30 < num29; num30++)
    {
        editorLevelData.tileBasedObjects.Add(new TileBasedObjectPlacement
        {
            type = stringCompressor.ReadStoredString(reader),
            position = reader.ReadMystIntVector2().ToStandard(),
            direction = (Direction)reader.ReadByte()
        });
    }
}
editorLevelData.spawnPoint = reader.ReadUnityVector3().ToUnity();
editorLevelData.spawnDirection = (Direction)reader.ReadByte();
if (b < 3)
{
    return editorLevelData;
}
editorLevelData.elevatorTitle = reader.ReadString();
if (b >= 4)
{
    editorLevelData.timeLimit = reader.ReadSingle();
}
editorLevelData.initialRandomEventGap = reader.ReadSingle();
editorLevelData.minRandomEventGap = reader.ReadSingle();
editorLevelData.maxRandomEventGap = reader.ReadSingle();
int num31 = reader.ReadInt32();
for (int num32 = 0; num32 < num31; num32++)
{
    editorLevelData.randomEvents.Add(reader.ReadString());
}
if (b <= 4)
{
    return editorLevelData;
}
editorLevelData.skybox = reader.ReadString();
if (b <= 5)
{
    return editorLevelData;
}
editorLevelData.minLightColor = reader.ReadUnityColor().ToStandard();
editorLevelData.lightMode = (LightMode)reader.ReadByte();
if (b <= 6)
{
    return editorLevelData;
}
if (b >= 10)
{
    editorLevelData.seed = reader.ReadInt32();
}
if (b >= 12)
{
    editorLevelData.potentialStickers.Clear();
    int num33 = reader.ReadInt32();
    for (int num34 = 0; num34 < num33; num34++)
    {
        editorLevelData.potentialStickers.Add(new WeightedID

```

```

{
    id = reader.ReadString(),
    weight = reader.ReadInt32()
});
}
}
if (b >= 13)
{
    editorLevelData.usesMap = reader.ReadBoolean();
}
if (b >= 15)
{
    int num35 = reader.ReadInt32();
    for (int num36 = 0; num36 < num35; num36++)
    {
        PremadeRoomLocation premadeRoomLocation = new PremadeRoomLocation();
        premadeRoomLocation.room = stringCompressor.ReadStoredString(reader);
        premadeRoomLocation.position = reader.ReadByteVector2().ToInt();
        premadeRoomLocation.direction = (Direction)reader.ReadByte();
        premadeRoomLocation.doorId = reader.ReadInt32();
        editorLevelData.premadeRooms.Add(premadeRoomLocation);
    }
}
if (b >= 16)
{
    editorLevelData.storeItemCount = reader.ReadInt32();
    int num37 = reader.ReadInt32();
    for (int num38 = 0; num38 < num37; num38++)
    {
        editorLevelData.potentialStoreItems.Add(new WeightedID
        {
            id = reader.ReadString(),
            weight = reader.ReadInt32()
        });
    }
    editorLevelData.meta = PlayableLevelMeta.Read(reader, true);
    return editorLevelData;
}
public IntVector2 mapSize = new IntVector2(50, 50);
public PlusStudioLevelFormat.Cell[, ] cells;
public List<CellArea> areas = new List<CellArea>();
public List<LightGroup> lightGroups = new List<LightGroup>
{
    new LightGroup()
};
public List<LightPlacement> lights = new List<LightPlacement>();
public List<EditorRoom> rooms = new List<EditorRoom>();
public List<DoorLocation> doors = new List<DoorLocation>();
public List<WindowLocation> windows = new List<WindowLocation>();
public List<ExitLocation> exits = new List<ExitLocation>();
public List<ItemPlacement> items = new List<ItemPlacement>();
public List<ItemSpawnPlacement> itemSpawns = new List<ItemSpawnPlacement>();
public List<BasicObjectLocation> objects = new List<BasicObjectLocation>();
public List<StructureLocation> structures = new List<StructureLocation>();
public List<NPCPlacement> npcs = new List<NPCPlacement>();
public List<TileBasedObjectPlacement> tileBasedObjects = new List<TileBasedObjectPlacement>();
public List<PosterPlacement> posters = new List<PosterPlacement>();
public List<WallLocation> walls = new List<WallLocation>();
public List<MarkerLocation> markers = new List<MarkerLocation>();
public List<PremadeRoomLocation> premadeRooms = new List<PremadeRoomLocation>();
public string elevatorTitle = "WIP";
public string skybox = "daystandard";
public Color minLightColor = new Color(0f, 0f, 0f);
public LightMode lightMode = LightMode.Cumulative;
public float timeLimit;
public Vector3 spawnPoint = new Vector3(5f, 5f, 5f);
public Direction spawnDirection;
public float initialRandomEventGap = 30f;
public float minRandomEventGap = 45f;
public float maxRandomEventGap = 180f;
public List<string> randomEvents = new List<string>();
public int seed;
public List<WeightedID> potentialStickers = new List<WeightedID>();
public bool usesMap = true;
public List<WeightedID> potentialStoreItems = new List<WeightedID>();
public int storeItemCount = 6;
public PlayableLevelMeta meta;

```

```
public Dictionary<string, TextureContainer> defaultTextures = new Dictionary<string, TextureContainer>();  
public const byte version = 18;  
}  
}
```